

Sorting Algorithms Learning Manual

A patient explanation, starting with the smallest actions

English edition for independent study

80 learning entries including families variants and supporting techniques

5 October 2026

Imagine four cards on a table: 4, 1, 3, and 2. We want them to read 1, 2, 3, 4. Every card must survive, including cards with equal numbers. The goal is easy to say. The questions are what to look at next, when to change positions, and how to know we are done. A sorting algorithm is a set of steps that answers those questions. We will learn the small actions first, then see how each method combines them.

Searchable text linked catalog and bookmarks original process diagrams cited sources

A gentle first route

The package has one starting file: 00_从这里开始.html. Double-click it and open it with a browser already on your Mac, such as Safari, Chrome, or Edge. Choose English at the top. It presents the complete material in smaller pieces. This PDF is the searchable and printable reference; you do not need to read it all in one sitting.

The guided route begins with four cards, comparing, and moving. It then teaches insertion, selection, bubble, merge, quick, and counting sort. The full catalog groups efficiency under comparable conditions. The beginner route follows what is easiest to understand first. Those orders serve different purposes.

Open an animation when the starting file tells you to. At first use Next for one action. Read why it happens, then watch the bars. Use Previous action to revisit it. Once you can explain an action, use Play slowly; the same button becomes Pause. Start again returns to the beginning. A held item or a record in the lower tray still exists and must return to the final row.

Before clicking Next, predict the comparison, movement, or new empty slot. Check your prediction with one step. Then repeat the method with four numbered paper cards. Explain in your own words why its stopping rule is safe. Each introductory lesson includes a question with an explained answer.

Six basic methods have offline animations. All 80 learning entries have complete procedures and numeric examples, with original diagrams where useful. Begin an advanced entry with its small-action foundations. If a tree, heap, queue, or binary search is unfamiliar, open the foundation supplement first. Timing symbols can wait until the actions make sense.

Keep both HTML files and both PDFs in one folder. Move the whole folder so local links keep working. After downloading the ZIP, extract it before opening the starting file. Offline lessons need no installation, account, or network connection. External references and optional VisuAlgo pages do need the internet.

Scope and counting rules

The complete catalog appears before the algorithm lessons. Its 80 numbered entries are teaching units, not 80 unrelated inventions. Alternate names share one entry. An important variant receives its own lesson only when its operations differ, and is labeled as a variant. Related implementations may be taught as a family.

Coverage includes common comparison sorts, integer and string distribution methods, adaptive and cache-oriented methods, less familiar tree and heap sorts, external schedules, parallel sorts and networks, restricted-operation sorts, and playful methods. Replacement selection is labeled as a run-generation phase. Indirect sorting and small-network design are supporting techniques. Grailsort and WikiSort are different implementations in one family, not synonyms.

The catalog is broad rather than exhaustive. It does not develop AKS network construction, Columnsort, Cubosort, the many model-specific LearnedSort variants, all theoretical integer-sorting results, every cache optimization, or every tape schedule. It also does not count each choice of pivot, gap sequence, or threshold as a new algorithm. Those topics need additional mathematical or machine assumptions.

Stalin sort discards out-of-order items and therefore fails the requirement to keep every input record. Miracle sort lacks an effective step that guarantees progress. Neither is counted as a full sorter here. Topological sorting orders dependencies, and Quickselect finds a rank rather than fully sorting; they solve different tasks and are excluded from the numeric-sorting ranking.

Examples use ascending order and are newly constructed for this manual. Small numbers expose operations; they are not performance measurements. Advanced families receive concrete mechanisms and prerequisites, while a production implementation still needs the boundary and engineering details in the cited original materials.

Complete catalog

Every entry and its alternate names appears here first. Labels distinguish algorithms from variants and supporting techniques. The right column gives the lesson page.

1 Methods using key structure and distribution 14

01 Counting sort Algorithm · Counting-based histogram sort; related to, but distinct from, pigeonhole sort	14
02 Pigeonhole sort Algorithm · Hole-based distribution sort; not another name for the general bucket-sort family	15
03 LSD radix sort Important variant · Least-significant-digit radix sort; LSD is a radix-sort direction	16
04 MSD radix sort Important variant · Most-significant-digit radix sort	18
05 Binary quicksort Important variant · radix-exchange sort; binary MSD radix sort	19
06 American flag sort Important variant · In-place multiway MSD radix sort	21
07 Bucket sort Algorithm family · Bin sort; a family of distribution strategies	22
08 Flashsort Algorithm · FlashSort; in-place classification and permutation sort	23
09 Trie sort Algorithm family · Prefix-tree sorting; digital-tree sorting forms	25
10 Burtsort Algorithm family · Burst-trie string sorting; includes copying and sampled variants	26
11 Spreadsort Algorithm family · Hybrid radix/comparison sorting; integer, float, and string variants	27
12 Multikey quicksort Algorithm · Three-way radix quicksort; 3-way string quicksort; aliases of the same method	28

2 Methods that use existing order 30

13 Natural mergesort Algorithm family · Natural merge sort; run-adaptive mergesort	30
14 TimSort Algorithm · Timsort; an adaptive merge/insertion hybrid	32
15 Powersort Important variant · Power-based natural merge scheduling; multiway Powersort is a related variant	34
16 Peeksort Important variant · Midpoint-peeking natural mergesort	35
17 Insertion sort Algorithm · Straight insertion sort; linear insertion sort	37
18 Binary insertion sort Important variant · Insertion sort with binary position search	39
19 Library sort Important variant · Gapped insertion sort	40
20 Patience sort Algorithm · Patience sorting; card-pile sorting	42
21 Smoothsort Important variant · Dijkstra's adaptive in-place heapsort	43

22 Strand sort Algorithm · Repeated monotone-subsequence extraction and merging	45
23 Cartesian tree sort Algorithm · Cartesian-tree priority-queue sort; Levkopoulos–Petersson adaptive heapsort	46
3 General comparison sorts and conditional optimizations	49
24 Top-down merge sort Algorithm · Recursive merge sort; a standard merge-sort implementation	49
25 Bottom-up merge sort Important variant · Iterative merge sort; merge-sort variant	51
26 Heapsort Algorithm · Binary heapsort; heap-based selection sorting	52
27 Weak heapsort Algorithm · Weak-heap sort; classic implementation with n reverse bits	54
28 Tournament sort Algorithm · Winner-tree sort; tournament-tree sorting	55
29 Introsort Algorithm · Introspective sort; quicksort–heapsort hybrid	57
30 Pattern-defeating quicksort (pdqsort) Algorithm · pdqsort; pattern-defeating quicksort	58
31 Splaysort Algorithm · Splay sort; self-adjusting tree sorting	60
32 Sorting with a min-max heap Important variant · min-max-heap extraction sort	62
33 Block merge sorting: Grailsort / WikiSort Algorithm family · Block sort; Grailsort and WikiSort are related implementations, not aliases	63
34 Quicksort with Lomuto partitioning Algorithm · Quicksort; Lomuto single-scan partitioning	65
35 Quicksort with Hoare partitioning Important variant · Original Hoare partition scheme; two-pointer quicksort variant	67
36 Three-way quicksort Important variant · 3-way quicksort; fat-pivot quicksort; Dutch-national-flag partitioning	69
37 Dual-pivot quicksort Important variant · Two-pivot quicksort; Yaroslavskiy-style dual-pivot partitioning	70
38 Samplesort Algorithm family · Sample sort; multiway sampled partitioning family	72
39 Tree sort and balanced-tree variants Algorithm family · Binary-search-tree sort; BST sort; balanced-tree variants	73
40 Shellsort and gap-sequence variants Algorithm · Diminishing-increment sort; Shell sort	74
41 Funnelsort Algorithm · Funnel sort; a cache-oblivious sorting algorithm	76
42 Merge-insertion sort Algorithm · Ford–Johnson algorithm	77
43 Indirect sorting Supporting technique · index sorting; pointer sorting; tag sort in the index sense	79
4 External parallel and restricted-input methods	81
44 External multiway merge sort Algorithm family · k -way external merge sort	81

45	Balanced multiway external merge Important variant · balanced k-way merge	82
46	Polyphase merge sort Important variant · polyphase external merge	84
47	Replacement selection for run generation Supporting phase · replacement-selection run generation	85
48	External distribution sort Algorithm family · external bucket or range-partition sort	86
49	Parallel merge sort Important variant · parallel mergesort	88
50	Parallel quicksort Important variant · parallel quick sort	89
51	Parallel samplesort and IPS4o Algorithm family · in-place parallel super scalar samplesort	90
52	GPU radix sort Important variant · parallel GPU radix sorting	92
53	Bitonic sorting network Algorithm · bitonic sort; sometimes Batcher sort	93
54	Odd-even merge sorting network Algorithm · Batcher odd-even mergesort; not odd-even transposition	95
55	Fixed small sorting networks Supporting technique · small compare-exchange networks	96
56	Shear sort Algorithm · snake-order mesh sorting	98
57	Heap sorting for bounded displacement Algorithm · k-displaced sorting; bounded-displacement heap method	99
5	Higher-work and restricted-operation methods	101
58	Comb sort Algorithm · Comb sort; CombSort11 is a gap-rule variant	101
59	Circle sort Algorithm · Circlesort; distinct from other circular-sort proposals	102
60	Cycle sort Algorithm · Write-minimizing comparison cycle sort; known-destination variants differ	103
61	Rank / enumeration sort Algorithm · Ranking sort; enumeration sort (not exhaustive permutation sort)	105
62	Selection sort Algorithm · Straight selection sort	106
63	Double selection sort Important variant · Min-max selection sort; bidirectional selection sort	107
64	Gnome sort Algorithm · Garden gnome sort; originally also stupid sort, an ambiguous name	108
65	Bubble sort Algorithm · Sinking sort; exchange sort is an ambiguous name	109
66	Cocktail shaker sort Important variant · Bidirectional bubble sort; shaker sort; double-direction bubble sort	111
67	Odd-even transposition sort Algorithm · Brick sort; odd-even sort (not odd-even mergesort)	113
68	Direct exchange / interchange sort Algorithm · All-pairs exchange sort; terminology varies across sources	114

69 Rotation-based in-place merge sorting Algorithm family · In-place merge; this section uses single-item insertion rotations	115
70 Pancake sort Algorithm · Sorting by prefix reversal	116
71 Burnt pancake sort Important variant · Signed prefix-reversal sorting	118
72 Stooge sort Playful method · Overlapping two-thirds recursive sort (descriptive name)	119
73 Slowsort Playful method · Slowsort; not a synonym for Bogosort in this text	120
6 Special models and playful methods	122
74 Bead / gravity sort Physical model · Abacus sort; gravity sort	122
75 Spaghetti sort Physical model · Spaghetti Analog Gadget (SAG)	124
76 Sleep sort Playful method · Delay-based output (descriptive name)	125
77 Exhaustive permutation sort Playful method · Permutation sort; sometimes deterministic Bogosort	126
78 Bogosort Playful method · Stupid sort; monkey sort; random permutation sort	127
79 Bozosort Playful method · Random-transposition sort	128
80 Bogobogosort Playful method · Recursive-check Bogobogosort; definitions vary	130

Start with four cards and the smallest actions

1 What result are we trying to get?

Think of 1+1 as putting one card beside one more card: there are now two cards. Separate the number of cards from the number written on a card. A card marked 7 is still one card. Input [4,1,3,2] contains four records.

Ascending order means that, as you move right, the next number never gets smaller. [1,2,3,4] passes that test. [1,3,2,4] fails because 2 follows 3. Equal numbers may stand together, so [1,2,2,4] is ordered too.

2 Why every original card must stay

Output [1,2,3] looks increasing, but it does not correctly sort [4,1,3,2]: the card marked 4 has disappeared. We need both correct order and every original card, exactly once.

A card may hold a name and a score. When sorting by score, the name must move with it. The whole card is a record. The score used to decide order is its key. Those words will mean exactly this in later lessons.

3 Looking and moving are different actions

Ask whether 4 is larger than 1. The answer is yes. That action is a comparison; the cards have not moved.

If the rule requires the smaller card on the left, exchange their positions: [4,1,3,2] becomes [1,4,3,2]. That action is a swap. A comparison need not cause a swap: 1 followed by 4 is already in the right direction. A complete algorithm must tell us which cards to compare next.

4 Why one successful swap is not enough

The first pair in [1,4,3,2] is ordered, but 4 still has smaller numbers to its right. Ordering one pair and ordering the whole row are different achievements.

Many methods make one small promise at a time: the smallest item is now first, or the first three items are ordered among themselves. The next action builds on that promise. Green in a diagram may mark a locally ordered part; the caption says whether those items also occupy final positions.

5 Insert a card without losing another one

Suppose [1,4] is already ordered and we want to add 3. It belongs between them. Hold 3 aside, move 4 one place right, then put 3 into the empty place. The result is [1,3,4].

Computer positions follow the same logic. Writing 3 straight over 4 destroys the old value unless we first save or move it. Temporary storage and shifting are ways of keeping every record safe. A held card is still part of the input.

6 Position numbers are addresses

We need a short way to say which place to use. Computer arrays usually count from zero. In [4,1,3,2], position 0 holds 4, position 1 holds 1, position 2 holds 3, and position 3 holds 2.

A position number says nothing about key size: position 0 may hold the largest key. Prose uses first, second, and so on; index formulas state the zero-based convention. A range is a consecutive group of positions. A prefix starts at the beginning, and a suffix reaches the end.

7 Equal numbers can still be different records

Two cards may both show 2 while belonging to different people. Write 2a and 2b to remember their identities; the letters do not change the numeric keys. Input [2a,1,2b] can become [1,2a,2b] or [1,2b,2a] while ordering its keys.

Keeping the original order of equal-key cards is called stability. The first result is stable for this input. Key order and equal-record order are separate requirements; each lesson says what its version preserves.

8 Combine two ordered rows

Take [1,4] and [2,3]. Each front card is the smallest one remaining in its own row, so compare the two fronts. Take 1 before 2. Then compare 4 with 2 and take 2. Compare 4 with 3 and take 3. One row is empty, so append the remaining 4. Output: [1,2,3,4].

This action is called merging. After taking a card, update only that row's front. We do not search the whole input again. Both input rows must already be ordered; merging two unordered rows this way is not a sorting guarantee.

9 Why solve smaller tasks first?

To sort four cards, sort the first pair and the second pair, then merge the ordered pairs. To sort a pair, we may treat it as two single-card groups. A single card is already ordered, so splitting stops there.

Running the same method on a smaller group is recursion. Imagine pausing a large job, finishing its smaller jobs, then returning to where you paused. A stopping rule is essential; empty or single-item groups usually stop immediately.

10 A stack and a queue are rules for taking things

In a pile of plates, the last plate added is the first removed. That rule is a stack. Paused recursive jobs can be remembered this way, so the program knows where to return.

In a waiting line, the first person to arrive is served first. That rule is a queue. Neither structure sorts values by itself. It determines which stored item is taken next.

11 A tree organizes relationships

Draw a circle containing 4, with circles 1 and 6 below it and lines joining them. The upper circle is a parent and the lower circles are children. Add circles 2 and 3 beneath 1. The linked shape is a tree. Its top node is the root. Nodes reached farther down a path are descendants.

The picture alone makes no promise about number order. A search tree requires smaller values on the left and larger ones on the right. A min-heap requires each parent to be no larger than its children. These are different rules, and neither picture is automatically a fully sorted row.

12 How a heap keeps the smallest item available

If every parent is no larger than its children, a path downward can never decrease. The root must be a smallest item. This structure is a min-heap. After removing the root, put the last item there, compare it with the smaller child, and exchange downward when needed until the rule holds again.

To add an item, place it at the end and compare it with its parent. Move it upward while it is smaller, stopping at the root or a valid parent relation. Siblings need not be ordered. Heap lessons explain how array positions represent the tree and how to find the children.

13 Buckets and digits

A bucket is a group. Groups for 0–9, 10–19, and 20–29 have a useful order: every first-group key precedes every second-group key. Yet a single bucket may still be unordered, so its contents may need sorting.

In decimal notation, 21 means two tens and one unit. Its units digit is 1 and its tens digit is 2. Grouping by one digit differs from grouping by the complete number. Radix methods explain how several digit passes combine into correct full-key order.

14 Find a place by repeatedly halving

To insert 5 into [1,3,6,8,10], first inspect the middle item, 6. Since 5 is smaller, ignore 8 and 10 and inspect the left part. The final gap is between 3 and 6. Removing roughly half the remaining candidates at each step is binary search.

This saves comparisons during the search. To put 5 into a tightly packed array, 6, 8, and 10 still have to move right. Finding a place quickly does not make those moves disappear.

15 What do we count when we say fast?

Count actual actions: cards inspected, key comparisons, swaps, records moved, or file blocks read and written. Some tasks have expensive comparisons; others have very large records that cost more to move. One count does not always capture both.

We use n for the number of records. $n=4$ means four cards; $n=1000$ means a thousand. A pass that touches each item once performs roughly n actions. Doubling the input usually doubles this kind of work.

16 Why n squared and $\log n$ appear

If almost every item interacts with almost every other item, work grows like n times n , written n^2 . Ten items give a scale near a hundred such actions; a thousand give a scale near a million. Actual pairwise methods may use roughly half that many comparisons.

If candidates repeatedly halve, a thousand shrink to one in about ten steps: 1000, 500, 250, 125, and so on. That halving count is $\log_2 n$, the base-two logarithm. Touching the whole input at each of about ten levels gives work near $n \log n$. The symbols name patterns of actions we can already understand.

17 Best, worst, and average need different questions

An already ordered row is favorable for some methods. A reversed row may be unfavorable, but not every algorithm has its worst case on reversed data. The worst input is whichever makes that particular method do the most work.

An average needs a rule for choosing inputs. Choosing every permutation with equal probability differs from using daily logs that are usually nearly ordered. A randomized method may also vary because of its own choices. Without those conditions, an average number is not a guarantee.

18 Room, termination, and how to read a lesson

An extra row, a count table, or notes about unfinished jobs all use extra space. In-place usually means no second full record array; it does not automatically remove a recursive stack.

A method must say when it finishes. Correct sorting orders the keys, preserves every record, and completes under its stated conditions. While following an example, ask what changed, why no card was lost, and what small promise now holds. Read the cost formulas after you can carry out the actions yourself.

Compare efficiency under the same conditions

We have already met n , repeated halving, and n^2 through concrete actions. The table now puts those growth patterns together. It gives scales of work, not seconds. Unless stated otherwise, one worker handles data already in memory and each comparison or small-record move has similar cost. Files, many workers, and special key types need separate conditions.

Growth	Plain meaning	Scale at n equal to 1000
n	A full scan; doubling the input roughly doubles work	1000
$n \log_2 n$	Scan per level while tasks repeatedly halve	about 10000
$n \log_2^2 n$	For example a network with squared-logarithmic depth	about 100000
n^2	Most items may interact with most other items	1000000
$n!$	Try all permutations; each added item multiplies the count	far beyond useful enumeration

O gives an asymptotic upper bound, Θ a matching growth scale, and Ω a lower bound. These symbols usually hide constants and do not give seconds. n counts records; k can mean a key-range size or a displacement bound; r is a radix; d or w counts digits or bits. Each lesson defines its local symbols. Unless stated otherwise, $\log$ means base two.

Best case means the most favorable input and worst case the least favorable. An average needs an input distribution; an expectation may instead be over the algorithm's own random choices. With high probability is not a guarantee for every random outcome. A lesson avoids a single average bound when it cannot justify one.

Assume all keys are different. Comparing two items has two possible answers: the first is smaller or the second is smaller. One question can distinguish at most two answer paths; two questions at most four; t questions at most 2^t . There are $n!$ possible orders, where $n!$ means $1 \times 2 \times \dots \times n$. To distinguish them all, the longest path must allow $2^t \geq n!$, requiring at least $\log_2(n!)$ comparisons in the worst case, or $\Omega(n \log n)$. An individual comparison need not split possibilities evenly. Counting and radix sorts use key structure beyond these two-answer questions.

Real string comparisons may inspect many characters. Large records cost more to move, and caches, branches, allocation, and disk transfers matter. Two $O(n \log n)$ sorts can therefore have different elapsed times. A tiny trace explains correctness, not processor speed.

External-memory analysis counts blocks of B records transferred with memory capacity M . Parallel analysis reports total work W and span S , the longest chain of dependent actions. On p workers, time is at least $\max(W/p, S)$, before scheduling and communication costs. Physical devices and timer tricks need their own models.

Conditional efficiency tiers

Fixed conditions	Priority tiers from less work to more work	Limits
Integer range $k=O(n)$ and fixed digits	Near-linear counting or radix tier → general $n \log n$ comparison tier → quadratic tier	Huge ranges or long keys change this order; buckets need distribution assumptions
Few inversions or few long ordered runs	$O(n+I)$ insertion or near-linear adaptive merging → ordinary $n \log n$ tier	I counts inversions; different meanings of nearly sorted are not interchangeable
Arbitrary large arrays using comparisons	Protected $n \log n$ tier → methods exposed to quadratic worst cases → fixed quadratic tier → superpolynomial tier	Plain quicksort can be fast on average; methods within a tier need actual measurement
Expensive writes or large records	Evaluate indirect sorting and low-write cycle sorting separately	More comparisons can still mean less total time
Data exceeds memory	Fewer wide merge or balanced distribution passes → more whole-file passes	Hold memory capacity disk system balance and stream limits fixed
Multicore GPU or sorting circuits	Evaluate fixed work span workers and bandwidth	Parallel layers are not serial work; no ranking across different hardware
Physical and playful random models	Model-dependent special methods stand apart; factorial trials appear later	Sleep sort has no general correctness guarantee; random retry has no finite worst-case bound

The lessons begin with methods that can be near-linear under key or order assumptions, then develop general $n \log n$ methods. Separate machine models follow. Usually slower methods and playful algorithms come later. Within a group, guaranteed or low-work tiers precede conditional, degeneration-prone, or research-oriented methods. Order within one tier follows learning dependencies, not an assertion that one always runs faster. The catalog and PDF bookmarks link to every entry.

How to study an entry

Each lesson first explains any small actions it needs, then connects them into a complete procedure. Move paper cards while following the example. After every swap, shift, group, or output, check that every card still exists. A complex method may take a second reading; the aim is to understand each action rather than memorize a name.

The cost section states the assumptions behind efficiency. Try duplicates, negative values, reverse order, and an empty input after reproducing the supplied trace. A stability claim applies to the described version, not automatically to every implementation with the same name.

Each lesson provides concrete operations, a complete small example, and necessary implementation cautions. References point to original papers, author implementations, university material, and NIST definitions. Each language edition has its own usable reference numbering.

Diagram key: orange outlines and C mark comparisons or classification; blue fills and M mark records just moved or swapped; green fills and S mark an ordered region or final positions explicitly described in the text. A locally ordered S region need not be globally final. White cells are unmarked. Each row matches a state in the written example; the text explains any intermediate comparisons omitted from the drawing.

State trace C compare M move S ordered

Compare 4 and 1; green [2,3] is only locally ordered



Blue marks the swapped pair; this is only a legend



1 Methods using key structure and distribution

Begin with the near-linear tier for small integer domains and fixed-width keys, then examine distribution- and prefix-dependent methods. Methods within a tier are peers. Key range, total character count, and bucket balance change the order. Binary quicksort is an MSD variant; multikey quicksort compares characters and is not automatically linear for arbitrary long text.

01 Counting sort

Type Algorithm Aliases or related names Counting-based histogram sort; related to, but distinct from, pigeonhole sort

First understand the small actions

An item and its value are different things. In [3,1,2,1], four items must remain four items. The two ones cannot be collapsed. A record may also contain a name; its sorting value is called its key, and the name must move with it.

A counting table has labeled slots. Slot 1 stores how many keys equal 1, slot 2 stores how many equal 2, and so on. These slots contain quantities, not the original records.

The idea in plain words

Suppose the keys can only be a few small integers. We can count their copies instead of repeatedly comparing pairs. Once we know that there are two ones, we know that every two must come after two occupied positions. Counting sort turns that observation into exact destinations.

Carry it out one action at a time

1. Find the minimum and maximum keys. Make a zero counter for every integer in between. Negative keys are fine: subtract the minimum to obtain a nonnegative slot number.
2. For each input record, increase its counter. Next, add the counters from left to right. A running total tells us how many records have this key or a smaller one, so it marks the end of this key's output region.
3. Prepare an output array. Read the original records backward. If a key's running total is t , place the record at index $t-1$ and reduce that total by one. The reduction reserves the preceding position for another copy of the key.
4. Stop after the first original record has been placed. For plain integers, an alternative is to write each labeled value as many times as its count. That simpler reconstruction does not carry names or other record fields.

Walk through the numbers

1. Label the equal records: [3,1A,2,1B]. Counts for 1,2,3 become [2,1,1], then running totals [2,3,4]. Ones own indices 0–1, the two ones index 2, and the three owns index 3.
2. Read 1B first. Its total is 2, so write it at index 1: [empty,1B,empty,empty]. Reduce the ones' total to 1. Read 2 and write it at index 2: [empty,1B,2,empty].
3. Place 1A at index 0, then 3 at index 3. The answer [1A,1B,2,3] keeps A before B. Removing the labels gives [1,1,2,3].

How much work and room it needs

The main jobs are scanning the records, scanning the counters, and writing the records once. Let n be the record count and $k = \max - \min + 1$, including unused key values. Best, usual, and worst work for this full version are $\Theta(n+k)$. Additional storage is $\Theta(n+k)$. It is stable: equal-key records keep their original order. Ten widely separated integers can still require an impractically large table.

Why it is fast or slow

A key points directly to its counter and destination region. The advantage disappears when most of the table is empty but still has to be allocated and scanned.

When it is useful

Good for scores, ages, small category numbers, and stable passes inside radix sorting.

Details that make the procedure work

Return immediately for an empty or singleton input. Keys need exact discrete slots. A presence-only bitmap loses repeated records. Compressing a sparse key range requires an ordered key map; constructing that map is additional work.

References [\[1\]](#)

02 Pigeonhole sort

Type Algorithm Aliases or related names Hole-based distribution sort; not another name for the general bucket-sort family

First understand the small actions

A container can hold whole records, including names or other fields. Several records may have the same key, so one labeled hole needs a list rather than a single replaceable slot.

Appending means adding at the end. If records enter a hole in input order and later leave in that order, equal keys keep their original sequence.

The idea in plain words

Put a row of labeled boxes on the table. Each record goes straight into the box bearing its integer key. Empty the boxes in label order, and the records come out ordered. This is pigeonhole sort. Counting sort calculates destination positions from quantities; pigeonhole sort first moves the actual records into containers.

Carry it out one action at a time

1. Find the minimum and maximum keys. Create one empty hole for every integer in that range. A hole must be able to store all records with its label.
2. Read the input from left to right. Append a complete record to its matching hole. Do not replace a previous record just because its key is equal.
3. Visit the holes from the smallest label upward. Copy every record from the current hole to the output before visiting the next hole. The ordered labels establish the order between different keys.
4. Finish after emptying the largest-label hole. To use negative keys as array indices, subtract the minimum. A plain-integer version may store counts instead of record lists, but that is a simplified key-only form.

Walk through the numbers

1. For [4,2A,3,2B], prepare holes 2,3,4. Reading 4 makes hole 4=[4]. Reading 2A makes hole 2=[2A]. Reading 3 makes hole 3=[3]. Reading 2B extends hole 2 to [2A,2B].
2. The output starts empty. Empty hole 2 to obtain [2A,2B], then hole 3 to obtain [2A,2B,3], then hole 4 to obtain [2A,2B,3,4]. Each move carries the record's accompanying information.

How much work and room it needs

Every record enters and leaves a hole once, and every hole is visited. With n records and k integer labels from minimum through maximum, best, usual, and worst work are $\Theta(n+k)$. Record containers need $\Theta(n+k)$ extra space; a count-only integer version needs $\Theta(k)$. Ordered appending and collection make it stable.

Why it is fast or slow

Direct labels save searching. Huge numbers of empty containers waste both storage and scanning, and separate containers can be more costly than a flat counting table.

When it is useful

Useful for grouping whole records by a narrow integer domain, and for understanding distribution sorting.

Details that make the procedure work

A general bucket can contain several different keys; a pigeonhole represents exactly one. A sparse hash table saves empty holes but does not usually give ordered labels. Those labels must still be arranged. Empty input returns immediately, and every duplicate must survive.

References [\[2\]](#) [\[1\]](#)

03 LSD radix sort

Type Important variant Aliases or related names Least-significant-digit radix sort; LSD is a radix-sort direction

First understand the small actions

Digits have different importance. In 21, the tens digit is 2 and the units digit is 1. Use a common width: 7 can be written 07 without changing its value.

A stable digit pass puts records into digit-labeled groups in their incoming order and collects the groups from 0 upward. Equal digits keep that incoming order. Stability is the mechanism that connects the passes.

The idea in plain words

Sorting units alone does not sort whole numbers. The useful trick is to sort tens afterward without disturbing the units order among equal tens. Tens then decide the large groups, while units decide the order inside each group. Repeating this from right to left is LSD radix sort.

Carry it out one action at a time

1. Choose a common number of digit positions d and a radix, the number of values one digit can have. Decimal has ten; byte-based programs often use 256. Use decimal here so the passes are visible.
2. Perform a stable pass using only units. Append records to digit groups in their current order and collect groups in digit order. This is an intermediate order, not a complete numeric order.
3. Use that result as input to a stable tens pass. Records with equal tens keep the units order established previously.
4. Continue leftward, always using the preceding pass's output. Finish after the most important digit.
A practical stable counting pass counts digits, calculates regions, writes records to a buffer in preserved order, and makes that buffer the next input.

Walk through the numbers

1. Start [21,13,12,11,22]. Units-1 receives 21 then 11; units-2 receives 12 then 22; units-3 receives 13. Collection produces [21,11,12,22,13]. It still places 21 before 11, so this is not the answer.

- Now inspect tens in that new sequence. Tens-1 receives 11,12,13; tens-2 receives 21,22. Collection gives [11,12,13,21,22]. The units ordering inside each tens group survives.
- If the second pass reversed its tens-1 group into [13,12,11], it would destroy the first pass's work. That is why stability is required, not merely convenient.

State trace C compare M move S ordered

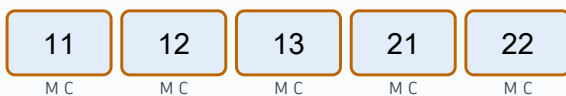
Input: inspect units first



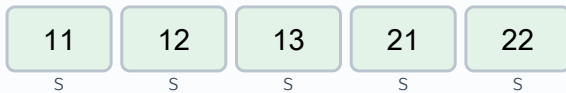
Stable units pass: intermediate order



Stable tens pass preserves units order



Both digits processed: fully sorted



How much work and room it needs

Each pass reads and writes all records and manages a digit table. For n records, d positions, and radix r , best, usual, and worst work are $\Theta(d(n+r))$ for the complete procedure. Fixed d and r make this linear in n . Extra space is $\Theta(n+r)$. Stable passes give stable output. Already sorted inputs normally still take all passes.

Why it is fast or slow

Small digit decisions replace repeated whole-key comparisons. Very long keys require many full-data passes, and an oversized radix table wastes memory.

When it is useful

Good for large fixed-width integer batches and equal-width identifiers.

Details that make the procedure work

Negative values need an order-preserving encoding rather than ordinary sign characters. Fixed-width signed integers can flip their sign bit. Variable-length strings need a suitable end marker or representation. Keep every duplicate record throughout every pass.

References [\[3\]](#) [\[4\]](#)

04 MSD radix sort

Type Important variant Aliases or related names Most-significant-digit radix sort

First understand the small actions

For equal-width positive integers, a smaller leading digit guarantees a smaller whole value. Strings instead use lexicographic order: inspect the first unequal character; if there is none, the shorter string comes first.

A region such as $[0,2)$ contains indices 0 and 1, excluding 2. Recursion means performing the same procedure on a smaller region, with explicit stopping conditions.

The idea in plain words

Why examine every digit of every key when some keys separate immediately? MSD radix sort begins with the most important digit. Different leading digits settle the order between groups; only ties need the next digit. A group containing one record can stop without reading the remainder of its key.

Carry it out one action at a time

1. Count the current digit values in the current region. Running totals give each group's consecutive destination region. Use an output buffer so writing a record cannot overwrite one still waiting to be read.
2. Distribute records in their incoming order, then copy the region back. Equal digits are now together. Preserving incoming order makes this buffered version stable.
3. For each group, return if it is empty, contains one record, or has no digit left. Otherwise move one digit rightward and repeat the same counting and distribution within that group. Never mix groups with different earlier digits again.
4. For variable-length strings, use a special end-of-string category smaller than every actual character. After the shared '2', key '2' ends while '21' continues. The ending key comes first, and its category must not recurse further.

Walk through the numbers

1. For $[31,12,32,11,22]$, tens-1 collects $[12,11]$, tens-2 $[22]$, and tens-3 $[31,32]$. The intermediate array is $[12,11,22,31,32]$, with regions 0–1, 2, and 3–4.
2. The singleton $[22]$ stops. Inside $[12,11]$, the units pass puts 11 before 12. Inside $[31,32]$, the units pass leaves the order unchanged.
3. The answer is $[11,12,22,31,32]$. There is no need to compare 11 with 32: their tens groups already establish the order.

State trace C compare M move S ordered

Input: inspect tens



Tens groups: 1 | 2 | 3



Sort units inside tens-1 only



All groups complete



How much work and room it needs

Count the characters actually inspected and the groups processed. With n records, maximum length d , radix r , and I grouping nodes, an upper bound is $O(nd+rI)$, commonly $O(nd)$ for a fixed small alphabet. Best is $\Theta(n+r)$ when one pass separates the keys. Usual cost depends on common prefixes. The buffered implementation uses $O(n+rd)$ extra space and is stable.

Why it is fast or slow

Separated groups never compare again, and singletons stop early. A long common prefix makes many records traverse many levels before they separate.

When it is useful

Useful for strings, long identifiers, and keys that separate near their beginning.

Details that make the procedure work

String '12' precedes '2' lexicographically, but not numerically. Normalize representations if numeric order is wanted. Keep repeated full keys and stop at their end. Small groups can use insertion sort: maintain an ordered prefix and insert the remaining records into it.

References [\[5\]](#) [\[6\]](#)

05 Binary quicksort

Type Important variant Aliases or related names radix-exchange sort; binary MSD radix sort

First understand the small actions

Binary writes numbers with 0 and 1. Three bit places have weights 4,2,1: 101 means $4+0+1=5$; 010 means 2; 111 means $4+2+1=7$; 001 means 1. Moving left doubles a place's weight, following $1+1=2$ and $2+2=4$.

A three-bit key beginning with 0 is at most 3; one beginning with 1 is at least 4. Compare lower bits only when higher bits agree. Left and right pointers simply remember the two positions being inspected.

The idea in plain words

Divide keys by their highest bit into a zero side and a one side, then divide each side by its next bit. It reads fixed bit positions rather than comparing whole keys with a numeric pivot. Cross-group order becomes known before internal order.

Carry it out one action at a time

1. Use equal-width nonnegative integers or an order-preserving equal-width encoding. Begin with the highest bit. Signed representations need transformation rather than blindly applying unsigned order.
2. Scan leftward-to-right with a left pointer, skipping current-bit zeros. Scan rightward-to-left with a right pointer, skipping current-bit ones. Stop scans at misplaced bits or crossing pointers.
3. If pointers have not crossed, the left one found a misplaced one and the right one a misplaced zero. Exchange their records, advance both inward, and continue.
4. After crossing, the left region contains zeros and the right ones at this bit. Recurse on each region using the next lower bit, rather than starting again at the highest.
5. Stop empty or single-item regions and stop when no bits remain. Equal-key records all survive. Read the full array as the result.

Walk through the numbers

1. Write [5,2,7,1] as [101,010,111,001]. At the highest bit, the left scan stops on 5 and the right scan on 1.
2. Swap them: [1,2,7,5]. Skip 2's highest zero and 7's highest one until the pointers cross. Regions are [1,2] | [7,5].
3. In the left region, second bits of 1 and 2 are zero and one, so they already separate into singletons. In the right region, 7's second bit is one and 5's is zero; exchange to [5,7].
4. All regions are singletons, so the last bit needs no scan. Result: [1,2,5,7]. Equal-key groups may instead continue through every bit, retaining every record.

How much work and room it needs

Across one bit level, unfinished groups scan at most n records. For w -bit keys, total work is bounded by $O(nw)$. Early singletons save later scans; an all-equal naive version still processes w levels. Only fixed key width permits treating w as a constant and calling the upper bound $O(n)$. Partitioning exchanges the input in place and depth-first recursion uses $O(w)$ stack. It is normally unstable. One-bit passes differ from radix implementations handling several bits at once.

Why it is fast or slow

Two buckets avoid a large count table, but single-bit passes may require more scans than wider-digit radix methods.

When it is useful

Fixed-width integer keys with limited extra array storage, and teaching bitwise partitioning.

Details that make the procedure work

For signed fixed-width integers, flipping the sign bit in the sorting representation can provide unsigned order matching numeric order; records keep their original values. Quick describes a partition resemblance, not an alternate numeric-pivot quicksort.

References [\[7\]](#) [\[3\]](#)

06 American flag sort

Type Important variant Aliases or related names In-place multiway MSD radix sort

First understand the small actions

Putting records into the right groups is only the first job. Their order inside a group may still need work. This method, like MSD radix sort, proceeds from important digits to less important ones.

An exchange saves one record temporarily so neither record is lost. A next pointer is simply an integer position: the next slot of a bucket not yet confirmed. It is not a second copy of the data.

The idea in plain words

A buffered digit sort needs another array of records. American flag sort avoids that large array. Count the groups first, reserve their consecutive regions, then exchange misplaced records into those regions. The groups become stripes; later passes finish the order inside each stripe.

Carry it out one action at a time

1. Count current digit values. In increasing digit order, assign each bucket a fixed start and end. Set its next position to its start.
2. Inspect the record at the current bucket's next position. If its digit belongs here, confirm the slot and advance next. Otherwise exchange it with the next unconfirmed slot of its destination bucket and advance that destination's next.
3. Inspect the record brought back into the current slot. It is still unconfirmed and may need another exchange. Continue until the bucket is full, then move to the next bucket. Boundaries and next positions identify completed areas without a marker on every record.
4. After distribution, process the next digit separately inside each bucket containing at least two records. Stop at a singleton, the final digit, or an end-of-string bucket. A list of pending regions can replace recursive function calls.

Walk through the numbers

1. For [21,12,11,22], both tens digits occur twice. Bucket 1 owns [0,2), bucket 2 owns [2,4), and their next positions are 0 and 2.
2. At index 0, 21 belongs to bucket 2. Exchange it with 11 at index 2: [11,12,21,22]. Bucket 2's next becomes 3. Recheck the current record, 11; it belongs to bucket 1, whose next becomes 1.
3. Confirm 12 at index 1, completing bucket 1. Confirm 22 at bucket 2's remaining index 3. Units passes on [11,12] and [21,22] finish the result [11,12,21,22].

How much work and room it needs

Each level counts, exchanges, and scans bucket tables. For n records, maximum key length d , radix r , and I processed groups, work is $O(nd+rI)$, or $O(nd)$ for fixed small radix. Best is $\Theta(n+r)$ when one pass separates keys; usual cost depends on prefixes. Depth-first tables and task information can use $O(rd)$ extra space without an n -record output buffer. Ordinary exchanges make it unstable.

Why it is fast or slow

It saves a large buffer, but exchanges can access memory less regularly than sequential writes. Managing tiny buckets can also cost more than their useful work.

When it is useful

Useful for digit-readable keys when memory is limited and equal-key order is unimportant.

Details that make the procedure work

Never advance beyond a bucket's end or reuse confirmed slots. In-place does not mean zero bookkeeping. Small buckets often use insertion sort. String endings form a category before characters and must stop further processing.

References [\[6\]](#) [\[8\]](#)

07 Bucket sort

Type Algorithm family Aliases or related names Bin sort; a family of distribution strategies

First understand the small actions

A bucket is a container, not a single counter. One bucket can contain several different values: the interval 10–19 includes both 11 and 13.

Ordered, nonoverlapping intervals settle the order between buckets. They do not settle the order within a bucket. Putting 13 and 11 together still leaves work.

The idea in plain words

When we know roughly where values fall, we can turn one large sorting job into many small ones. Send records to range buckets, sort each bucket, then join them. Bucket sort is fast only when the remaining local jobs are small enough; distribution alone is not sorting.

Carry it out one action at a time

1. Choose ordered intervals and define boundary ownership. Integers 0–99 could use tens buckets. Unequal-width intervals are also possible if dense areas need finer divisions. Every record must match exactly one interval.
2. Read records in input order and append each whole record to its bucket. Keep associated fields and every duplicate.
3. Order each bucket internally. For this example, treat its first item as ordered. Hold the next item, shift preceding larger items right, and insert it in the opened slot. Repeat to the bucket's end. Singletons need no internal work.
4. Collect the finished buckets from the smallest interval upward. Both the internal and inter-bucket order are now correct. Preserving arrival order and inserting after equal keys makes this version stable.

Walk through the numbers

1. For [42,13,35,11,41], use 10–19, 20–39, and 40–49. Records enter these buckets as [13,11] | [35] | [42,41].
2. In the first bucket, hold 11, shift 13 right, then insert 11: [11,13]. Leave [35] alone. In the last bucket, shift 42 to insert 41 before it: [41,42].
3. Joining these buckets gives [11,13,35,41,42]. Joining the original bucket contents without local sorting would give [13,11,35,42,41], which is incorrect.

How much work and room it needs

Distribution and collection each scan the records; local sorting adds work. Let n be record count, b bucket count, and m_i bucket i 's size. Insertion-sorted buckets give upper bound $O(n+b+\sum m_i^2)$, more exactly $\Theta(n+b+\sum I_i)$ for their inversion counts I_i . Best is $\Theta(n+b)$. Independent uniform keys with b proportional to n give expected $\Theta(n+b)$. One reversed bucket permits worst $\Theta(n^2+b)$. Extra space is $\Theta(n+b)$; stability depends on local sorting.

Why it is fast or slow

Good spread creates tiny tasks. Clustering leaves a large task nearly unchanged. A guaranteed $O(m \log m)$ local sorter protects the worst case but changes the algorithm's cost.

When it is useful

Useful for numerical ranges and distributions known in advance, including some spatial coordinates.

Details that make the procedure work

Assign the maximum to a valid final bucket. Hash buckets cannot generally be collected by bucket number to obtain numeric order. Too many buckets waste initialization work. Define an order for special values such as NaNs before using ordinary comparisons.

References [\[9\]](#) [\[10\]](#)

08 Flashsort

Type Algorithm Aliases or related names FlashSort; in-place classification and permutation sort

First understand the small actions

Array indices identify positions, starting at zero here. Region $[2,4)$ contains slots 2 and 3. Assigning those slots to a class does not order their contents.

Keep fixed class starts and moving ends. Placing a record at $\text{end}-1$ and decreasing end expands the completed suffix of that class. Save a displaced record before overwriting its slot; otherwise data would be lost.

The idea in plain words

Can we group values without storing separate arrays of records for every bucket? Flashsort reserves regions inside the original array and moves records around in cycles. One placement displaces another record, which is placed next. Numerical interpolation estimates a class, not an exact final position, so local finishing is essential.

Carry it out one action at a time

1. Find min and max. Equal values need no rearrangement. Otherwise choose m classes and compute $c(x) = \text{floor}((m-1)(x-\text{min})/(\text{max}-\text{min}))$; floor discards the fractional part downward. A class represents a range, not a record's final rank.
2. Count classes and accumulate exclusive ends. Retain their fixed starts. A class is unfinished when $\text{start}[c] < \text{end}[c]$. Hold the record at the first unfinished class's start as the cycle leader.
3. Place the held record at its class's $\text{end}-1$, saving the record displaced there, then decrease that end. The displaced record becomes the next held record. Repeat until a placement returns to the leader's original position, closing the cycle.
4. Find another unfinished class. A record already at its own class's last unfinished slot can simply be confirmed by decreasing end. When all ends equal starts, distribution is complete. Then insertion-sort each class by holding the next value, shifting larger predecessors right, and filling the opening.

Walk through the numbers

1. Use $[40,20,32,10,31]$, $\text{min}=10$, $\text{max}=40$, $m=3$. Class 0 contains 10,20; class 1 contains 31,32; class 2 contains 40. Counts $[2,2,1]$ give starts $[0,2,4]$ and ends $[2,4,5]$.
2. Hold 40 from index 0. Put 40 at 4 and retain 31; $\text{end}[2]=4$. Put 31 at 3 and retain 10; $\text{end}[1]=3$. Put 10 at 1 and retain 20; $\text{end}[0]=1$. Put 20 at 0 to close the cycle; $\text{end}[0]=0$. State: $[20,10,32,31,40]$.

3. Classes 0 and 2 are complete. Class 1 still has [2,3]; 32 already belongs in that last unfinished slot, so end[1] becomes 2. The classes are correct, but 20,10 and 32,31 remain reversed. Local insertions produce [10,20,31,32,40].



How much work and room it needs

Classification reads all records, cyclic placement settles them, and local ordering adds work. For n records and m classes, the first two phases take $\Theta(n+m)$. With m proportional to n and favorable spread, expected total work can approach $\Theta(n)$. Best is $\Theta(n+m)$, or $\Theta(n)$ for an all-equal shortcut. Insertion finishing permits worst $\Theta(n^2+m)$. Tables use $\Theta(m)$ space. The cycles normally make it unstable.

Why it is fast or slow

Classification shrinks global disorder into local tasks while cycles avoid a full record buffer. Outliers can stretch the range and crowd most keys into a few classes.

When it is useful

Useful for fairly smooth numerical distributions when equal-record order is not required.

Details that make the procedure work

This clear variant retains starts and ends; versions using fewer tables need more careful leader rules. Guard division by zero, rounded class indices, and bounds. The entry means Neubert's numerical classification algorithm, not other algorithms sharing the name.

References [\[11\]](#) [\[12\]](#)

09 Trie sort

Type Algorithm family Aliases or related names Prefix-tree sorting; digital-tree sorting forms

First understand the small actions

A string is a sequence of characters. Lexicographic order checks the first unequal character; if one string ends after an otherwise equal prefix, the shorter comes first. This differs from numeric order.

A tree has connected nodes and a starting root. Here every connection bears a character. Reading connections from the root spells a prefix. A node must also hold records ending there, not just links to children.

The idea in plain words

Many strings repeat the same beginning. A trie gives that beginning one shared path, with characters choosing branches. After insertion, exploring smaller-character branches first produces lexicographic order. The name 'prefix tree' describes exactly what is shared.

Carry it out one action at a time

1. Create an empty root. Each node needs character-labeled children and a list of ending records.
Direct character slots or labeled links can work, but output must visit child labels in sorted order.
2. Insert a string by following its characters from the root. Create a missing child when necessary. At the end, append the whole record to that node's ending list. Repeated keys share a path but retain separate ending records.
3. Output a node's ending list first, then visit children from the smallest label upward, doing the same at each child. Ending first places '2' before its extension '21'.
4. Return to the previous node when a branch finishes and visit its next child. Finish after all root branches. Recursive calls or an explicit task stack remember unfinished branches; remembering only the current path would lose them.

Walk through the numbers

1. Insert ['21','12','2','12']. '21' creates root→2→1 and an ending record. '12' creates root→1→2. '2' ends at the existing node 2. The second '12' appends another record at the existing terminal node.
2. Explore branch 1 and emit both '12' records. Explore branch 2, emit its ending '2', then its child 1's '21'. Output: ['12','12','2','21'].
3. For numeric order, normalize to equal width ['21','12','02','12']. Branch 0 emits '02' first, followed by both '12' records and '21': numbers [2,12,12,21].

How much work and room it needs

The main work reads characters, visits or creates nodes, and emits records. Let S be total input characters and n record count. A fixed small alphabet with direct children allows $O(S+n)$. Scanning all r slots at V dense nodes adds $O(rV)$; ordering sparse child labels also costs work. Typical storage is $O(S+n)$, possibly larger with dense nodes. Ordered terminal lists make it stable.

Why it is fast or slow

Shared prefixes avoid repeated full-key comparisons. Nodes, pointers, and scattered memory accesses can nevertheless be expensive.

When it is useful

Good for string sorting combined with prefix lookup.

Details that make the procedure work

An empty string ends at the root. A set-style trie needs counts or lists to preserve duplicates. Single-child paths can be compressed into multi-character edges, with the same ordering rules.

References [13] [14]

10 Burstsor

Type Algorithm family Aliases or related names Burst-trie string sorting; includes copying and sampled variants

First understand the small actions

A trie node represents a shared prefix and branches on the next character. A leaf bucket temporarily holds many records. A record pointer names the stored string; it need not copy the whole string.

A processor cache is small, fast nearby storage. Keeping a local sorting job small can avoid repeated slow memory access. The bucket threshold chooses when to split, not whether output is correct.

The idea in plain words

A complete trie may allocate too many nodes, while ordinary string comparisons repeatedly reread common beginnings. Burstsor keeps a small prefix tree with buckets below it. Only an oversized bucket becomes a deeper branching node: it bursts. The remaining small buckets are sorted locally.

Carry it out one action at a time

1. Begin with one root bucket. Follow existing prefix branches to a leaf and append each string's record pointer. The path already establishes a common prefix, so later local comparisons can skip it.
2. When the threshold is exceeded, replace the bucket with a next-character node. Read that character from each stored key and redistribute into child buckets. Keys ending here go into a terminal list; never try to subdivide nonexistent characters.
3. Split oversized children as needed. After all insertion, traverse character branches in order, outputting ending records before extensions.
4. At a retained leaf bucket, fully sort the unexamined suffixes before emitting records. A comparison merge can split to singleton pieces, then repeatedly take the smaller current head from two pieces. Bucket membership alone does not establish suffix order.

Walk through the numbers

1. Use ['21','12','11','22','12'] with teaching threshold 2. The first two enter the root bucket. The third bursts it into character-1 bucket ['12','11'] and character-2 bucket ['21'].
2. Adding '22' makes the latter ['21','22']. Another '12' overfills the first, which splits by its second character into path 11=['11'] and path 12=['12','12']. Keep both ending records rather than splitting forever.
3. Traversal emits '11', then both '12' records. The remaining prefix-2 leaf compares suffixes 1 and 2, emitting '21','22'. Answer: ['11','12','12','21','22'].

How much work and room it needs

There is prefix work and local suffix work. Let D count prefix operations, m_i bucket sizes, and c_i character work per suffix comparison. Comparison finishing gives $O(D + \sum c_i m_i \log m_i)$, potentially $O(nd \log n)$ for n records of maximum length d . Best and usual behavior cannot be stated from n alone: lengths, prefixes, and thresholds matter. Pointers, buckets, and nodes have a conventional $O(n+S)$ space bound for total characters S . Basic implementations normally make no stability promise.

Why it is fast or slow

It reduces repeated prefix reads and aims to keep local work in cache. Poor thresholds or data distributions can instead make splitting and allocation expensive.

When it is useful

Designed for large string collections where character reading and memory locality dominate.

Details that make the procedure work

Order-preserving distribution plus stable local sorting can make a stable version. C-burstersort copies suffixes; sampled variants estimate the tree first. Published speed ratios describe particular experiments, not a universal multiplier.

References [\[15\]](#) [\[16\]](#)

11 Spreadsort

Type Algorithm family Aliases or related names Hybrid radix/comparison sorting; integer, float, and string variants

First understand the small actions

For a nonnegative integer, shifting right s binary bits is division by 2^s with the fraction discarded. Shifting three bits groups consecutive values in ranges of eight; discarded bits still determine order inside a group.

A hybrid chooses different actions for different jobs. The comparison fallback must fully sort its region. One guaranteed method splits down to singletons, then merges two ordered pieces by repeatedly choosing their smaller current head.

The idea in plain words

One fixed digit size can be wasteful: too many buckets for a small job, or too little separation for a large one. Spreadsort examines the current range and population before choosing its bit grouping. It switches to comparisons when more distribution is not worth its overhead.

Carry it out one action at a time

1. Find the current minimum, maximum, and record count. Equal keys can stop. A sufficiently small region is sent directly to complete comparison sorting.
2. For a large region, choose a shift s that keeps the bucket table manageable. Number buckets by shifted key minus shifted minimum. Count records and accumulate consecutive destination regions.
3. Exchange misplaced records into their bucket's next unfilled slot, then deal with the displaced record. Advance the appropriate position counters until every region contains the correct bucket. Leading bits now settle the order between regions.
4. Recurse on large buckets with differing bits left. Switch an unsuitable or small bucket to a worst-case $O(m \log m)$ comparison sorter. Finish only when every region is internally ordered. Digit choice and thresholds belong to the implementation's strategy; arbitrary replacements need their own analysis.

Walk through the numbers

1. Take [17,2,18,1,9,8] and shift three bits, meaning divide by eight and discard the fraction. Buckets 0,1,2 each receive two records, so their regions are [0,2), [2,4), [4,6), with next positions [0,2,4].
2. At position 0, move 17 into bucket 2 by exchanging it with 9 at position 4. The array becomes [9,2,18,1,17,8], and bucket 2's next position is 5. The returned 9 belongs in bucket 1: exchange it with 18 at position 2, giving [18,2,9,1,17,8]. Bucket 1's next is now 3.

3. Recheck position 0. Exchange 18 with 8 at position 5: [8,2,9,1,17,18]. Then exchange 8 with 1 at position 3: [1,2,9,8,17,18]. Finally confirm 1 and 2 in bucket 0. The other buckets' next positions have reached their ends, so distribution is complete.
4. Bucket 1 still contains [9,8]. Local comparison sorting changes it to [8,9]; the other two regions need no change. The answer is [1,2,8,9,17,18]. A production version may comparison-sort six items immediately. This example exposes the distribution used on larger regions.

How much work and room it needs

Count distribution scans, movement, and fallback comparisons. For n integer records of bit length K , Boost lists best $O(n)$, average performance model $O(n\sqrt{\log n})$, and worst $O(n \cdot \min(\log n, K))$. The average model does not describe every distribution, and arbitrary threshold choices do not inherit the bound. Bucket and recursion storage depends on key length and configuration. Ordinary Spreadsort is unstable. Strings also require character-reading costs.

Why it is fast or slow

It exploits structure on large jobs, avoids bucket overhead on small jobs, and uses a protected fallback for difficult cases.

When it is useful

Large integer, floating, or string collections with suitable encodings and no equal-record order requirement.

Details that make the procedure work

Bit extraction and comparisons must agree on order. Floating keys need an ordered encoding and policies for NaNs and signed zeros. This named family is more specific than any generic bucket-then-sort design.

References [\[17\]](#) [\[18\]](#)

12 Multikey quicksort

Type Algorithm Aliases or related names Three-way radix quicksort; 3-way string quicksort; aliases of the same method

First understand the small actions

Lexicographic order compares first characters, then later characters only after equality. String "13" therefore precedes "2", although integer 13 exceeds 2.

Character position d begins at zero. Exchange complete string references, not characters inside a string.

Use an end marker below every real character when a string has ended. Prefix "2" then precedes "20".

Small, equal, and large describe the current character; only equals have established it and can advance.

The idea in plain words

Inspect one character position and partition three ways. Side groups continue at the same position; equals advance. Shared prefixes are established once along the equal branch instead of reread from the start in repeated full-string comparisons.

Carry it out one action at a time

1. Empty and singleton groups stop. Otherwise choose one string's character at d as saved pivot c , using the end marker if needed. Begin the entire sort at $d=0$.

2. Set *lt* and *i* at the group's first slot, *gt* at its last. Before *lt* is smaller; *lt*..*i*-1 equal; *i*..*gt* unchecked; after *gt* larger.
3. A character below *c* exchanges its whole reference with *lt*, then advances *lt* and *i*. An equal character advances *i* only. A larger character exchanges with *gt* and retreats *gt* without advancing *i*: inspect the incoming record again.
4. When unchecked space ends, sort smaller and larger groups at the same *d*. Their current-character order is unresolved internally. Advance the equal group to *d*+1 when *c* is real.
5. If *c* is the end marker, finish that equal group immediately while retaining all its records. Random pivots help avoid repeated uneven splits.
6. Tiny groups may use insertion: hold the next reference, compare strings from *d* until a difference or end, shift larger references, and insert. Do not reread an established prefix from zero.

Walk through the numbers

1. Use ["21","13","11","20","2"] at *d*=0, pivot character "2". Move smaller "13" left: ["13","21","11","20","2"]. Move smaller "11" left: ["13","11" | "21","20","2"]. Remaining first characters equal "2".
2. The left group shares "1", advancing to *d*=1. Against pivot "3" from "13", "11" has smaller "1". Exchange to obtain ["11","13"] and stop at singletons.
3. The original equal group advances to *d*=1: ["21","20","2"], pivot "1". Move "20" with its "0" left: ["20","21","2"]. The end marker of "2" is smaller still: ["20","2" | "21"].
4. Sort ["20","2"] at the same *d*=1. Its end marker precedes pivot "0", producing ["2","20"]. With "21", this group is ["2","20","21"].
5. Combine to get ["11","13","2","20","21"]. All five records occur once, in character order rather than integer order.

How much work and room it needs

Let *n* count strings and *L* be maximum length, with constant-time character access. Random pivots give expected $O(n \log n + nL)$, worst $O(n^2 + nL)$. Unequal-character partitions can degenerate, while equal-character advancement reads at most about *nL* positions. Identical short strings can give best $\Theta(n)$; identical length-*L* strings require $\Theta(nL)$. Reference exchanges need $O(1)$ temporary space. Naive stack is expected $O(\log n + L)$, worst $O(n + L)$. It is unstable. Copying bodies or decoding characters adds work.

Why it is fast or slow

Equal branches establish shared prefixes without restarting every comparison at character zero. End markers correctly order prefixes before longer strings.

When it is useful

Shared-prefix strings and lexicographic multi-field keys. Unequal-length decimal strings cannot directly replace numeric ordering.

Details that make the procedure work

Specify byte, Unicode code-point, or language collation order. Fixed-width zero-padded nonnegative decimal strings can also follow numeric order. Despite its radix alias, this method uses character comparisons rather than counting buckets.

References [\[19\]](#) [\[20\]](#) [\[21\]](#)

2 Methods that use existing order

Start with adaptive merging under a few-long-runs condition, then insertion under a few-inversions condition. These are different measures of existing order and do not create a fixed rank against one another. Later entries use space or randomness assumptions, or extraction strategies that may still do quadratic work. Worst-case protection and adaptivity are separate properties.

13 Natural mergesort

Type Algorithm family Aliases or related names Natural merge sort; run-adaptive mergesort

First understand the small actions

A run is a consecutive stretch already in order. Equal neighbors are allowed: $[1,1,4]$ is a nondecreasing run. A drop starts a new run.

Merging means repeatedly selecting the smaller of two current fronts, not merely joining the sequences end to end. Once one side ends, the other tail can be copied directly.

The idea in plain words

An input can be partly ordered even when the whole row is not. In $[1,4,2,3,0,5]$, three short stretches already rise. Natural mergesort notices those stretches and keeps them intact. It merges neighboring runs into longer ordered runs until one run covers the input. The word natural refers to using order that was already there.

Carry it out one action at a time

1. Scan left to right. Keep extending the current run while the next value is at least the current value. At a drop, record the run's boundaries and start another. Remember to record the final run when the input ends.
2. Pair adjacent runs throughout a round: first with second, third with fourth, and so on. For each pair, use a buffer and one position in each run. Compare their current records, write the smaller to the buffer, and advance only the selected position. On a tie, take the left record first.
3. When one run is exhausted, append the unused tail of the other. Write the merged records back over the two original runs and record one new run. An odd final run has no partner and passes to the next round unchanged.
4. Repeat the pairing rounds. Each round roughly halves the run count. Stop when a single run spans the array. This entry uses simple pairwise rounds; TimSort, Powersort, and Peeksort choose more carefully among neighboring merges.

Walk through the numbers

1. Scan $[1,4,2,3,0,5]$. The rise $1 \leq 4$ continues the first run, but $4 > 2$ ends it. Likewise, $2 \leq 3$ followed by $3 > 0$ gives the second run. The final rise gives $[0,5]$. State: $[1,4] \mid [2,3] \mid [0,5]$.
2. Merge the first pair. Select 1 against 2, then 2 against 4, then 3 against 4. The right run ends, so copy 4. The unpaired last run remains: $[1,2,3,4] \mid [0,5]$.
3. In the next merge, select 0 against 1. Values 1,2,3,4 then beat the right front 5 in turn. Copy the remaining 5. The single run is $[0,1,2,3,4,5]$, so sorting is complete.
4. Notice that an ordered run is not a final location. Zero started in the last run but eventually moved to the beginning of the whole output.

State trace C compare M move S ordered

Three runs: [1,4] | [2,3] | [0,5]



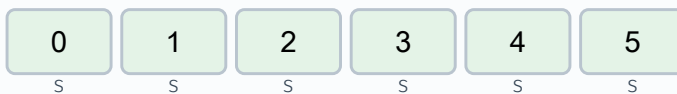
Merge first two; the run with 0 waits



Next merge selects the right run's 0 first



One run covers the input: finished



How much work and room it needs

Finding runs reads the input once. A merging round reads and writes each record at most once, costing $O(n)$. Starting with r runs takes about $\log_2 r$ rounds, for $O(n + n \log r)$ total work. Here n is the record count and r is the initial run count. One run gives best $\Theta(n)$. Unstructured typical data and the worst case need $O(n \log n)$. A conventional array merge uses $\Theta(n)$ extra buffer space; lists can merge by reconnecting nodes. Left-first equality makes the stated version stable.

Why it is fast or slow

Long existing runs reduce the number of rounds and avoid redoing order that was already correct. Simple pairing can still move a very long run repeatedly when its neighbors have very different lengths, which motivates better merge schedules.

When it is useful

A good starting point for adaptive merging, meaning sorting that uses existing order. It suits partly ordered arrays, linked lists, and run-based file processing.

Details that make the procedure work

A scanner may also recognize strictly decreasing runs and reverse them. Strictness matters: reversing equal records would reverse their original order. The example scans nondecreasing runs only. Never lose an unfinished tail during a merge. If writing backward, choose the right record first on equality to preserve the same stable order.

References [\[22\]](#) [\[23\]](#)

14 TimSort

Type Algorithm Aliases or related names Timsort; an adaptive merge/insertion hybrid

First understand the small actions

A run is a consecutive stretch in nondecreasing order. A stable merge repeatedly takes the smaller unused front, choosing the left front on equality.

A stack works like a pile of papers: add a paper on top and remove from the top. Here each paper describes a run's start and length; the actual records remain in the array.

Binary insertion finds a position by halving an ordered search interval, then shifts records backward to create an opening. TimSort uses it only to extend short runs.

The idea in plain words

A real input often contains order already, but its ordered stretches may have very different lengths. Repeatedly merging one long run with tiny neighbors can move the same long run many times. TimSort keeps the existing stretches, extends very short ones, and controls the order of neighboring merges. During a merge, it can also discover a whole winning block instead of comparing every item separately. These are three concrete operations to learn: find runs, schedule merges, and copy confirmed blocks.

Carry it out one action at a time

1. Find the next run at the first unprocessed position. Continue through equal or increasing neighbors. If it begins by decreasing, follow strictly decreasing neighbors and reverse that stretch by exchanging its outer items inward. Do not reverse equal items: their order would change. Either route produces a nondecreasing run.
2. Choose a short-run target called minrun, usually a few dozen records in real implementations. If the run is shorter, take following records one at a time and binary-insert them until the target or the input end is reached. Insert after older equal values and shift from right to left. The extended run remains ordered and stable.
3. Push the run's location and length onto the stack. For the corrected classic policy, call the top four lengths W, X, Y, Z , from older to newer. If $X \leq Y + Z$ or $W \leq X + Y$, merge Y with the shorter neighbor X or Z . If neither condition holds but $Y \leq Z$, merge Y and Z . Ignore a condition if one of its runs is missing. Replace the two descriptions with their merged description and check again until no rule requests another merge.
4. Only adjacent runs may merge. A full buffer and two front pointers provide the simplest implementation. A usual optimization buffers only the smaller run. If it is the left run, write forward; if it is the right run, write backward while comparing the larger remaining ends. Forward ties choose the left record; backward ties write the right record first. Both preserve the original order of equals.
5. When one side wins repeatedly, search for a whole winning block. Probe at distances 1,2,4,8,... until a record stops winning or the run ends. Binary-search inside the last bracket to find the exact boundary, then copy the confirmed block. This expanding search is called galloping. Resume ordinary merging when appropriate; production versions adjust the threshold for using it.
6. At the end of input, finish the pending neighboring merges even if no length rule currently fires. A classic final collapse repeatedly pairs the second-from-top run with its shorter available neighbor until only one run remains.

Walk through the numbers

1. Use teaching minrun=2 for [1,4,2,3,0,5]. The first run [1,4] gives length stack [2]. The next run [2,3] makes [2,2]. The newest two lengths satisfy $Y \leq Z$, so merge them.
2. Take 1, then 2 and 3, and copy 4. The first four slots become [1,2,3,4], and the stack is [4]. Adding [0,5] gives [4,2]. No immediate two-run rule fires. Input has ended, so the final collapse merges them into [0,1,2,3,4,5].
3. For a separate extension example, use target 4 on [2,5,1,4]. The natural run is only [2,5]. Insert 1 by shifting 5 and 2 right: [1,2,5,4]. Insert 4 before 5: [1,2,4,5]. Only then does the four-item run go onto the stack.
4. To see galloping, merge [1,2,3,4,8] with [5,6,7,9]. The current right value is 5. Left items at one-based distances 1,2,4 are 1,2,4, all eligible to come first. Distance 8 is beyond the run, which brackets the boundary between the fourth item and the end. Binary search finds 8 as the first item that cannot precede 5. Copy [1,2,3,4] together, then continue merging. A real threshold need not start galloping on this tiny case.

How much work and room it needs

Run discovery reads the array once. Extension performs insertion only inside short pieces. The length policy controls how many merge levels each record travels through, giving $O(n \log n)$ typical and worst time. One existing run gives best $\Theta(n)$. A run-sensitive upper bound is $O(n + n \log r)$, where n is the record count and r the initial run count. Galloping can reduce comparisons but cannot remove the work of writing records. Conventional extra space is $O(n)$, mainly a buffer for a shorter merge partner; the stack stores only descriptions. TimSort is stable.

Why it is fast or slow

Existing order avoids unnecessary sorting. Extending tiny runs reduces overhead, and the length rules prevent careless repeated movement of long runs. Galloping helps when one side supplies many consecutive winners; alternating winners offer much less opportunity.

When it is useful

Useful for general records, text, and partly ordered data requiring stability. Learn ordinary merging and binary insertion first; the stack inequalities are an advanced scheduling detail rather than the first idea to memorize.

Details that make the procedure work

This is the corrected classic scheduling policy. Its four-run check matters; incomplete older rules caused stack problems. Modern CPython retains run discovery, insertion, and galloping but uses Powersort for scheduling. Teaching targets 2 and 4 are not production choices. An empty input returns immediately, and a final short run retains its actual length.

References [\[24\]](#) [\[25\]](#) [\[26\]](#)

15 Powersort

Type Important variant Aliases or related names Power-based natural merge scheduling; multiway Powersort is a related variant

First understand the small actions

Merging runs of a and b records visits about $a+b$ records. A different order of merges can therefore change how often the same records move.

A stack stores pending left runs and boundary numbers. Push adds an entry on top; pop removes the top. Records remain in the array, not inside the scheduling stack.

Relative positions treat the whole array as a ruler of length one. A run midpoint is a location on that ruler, not the average of its values. Floor rounds down, for example $\text{floor}(2.7)=2$.

The idea in plain words

Suppose several ordered queues must become one queue. Joining two short queues before adding a long one can save repeated work. Powersort chooses neighboring merges from the lengths and positions of the original runs. Imagine dividing the input ruler into halves, then quarters, then eighths. For each boundary, ask when the neighboring run midpoints first land in different cells. That level is the boundary's power. A larger power represents a deeper, smaller merge to perform sooner; a smaller power represents a higher merge to leave until later.

Carry it out one action at a time

1. This basic version discovers maximal nondecreasing runs only. For a run starting at zero-based position s with length L , compute its relative midpoint $\mu=(s+L/2)/n$, where n is the total length. Keep the previous original run: the next boundary power is calculated from two neighboring original runs, not from an enlarged merged run.
2. Start with two equal ruler cells, numbered 0 and 1. If the midpoints share a cell, try four cells, then eight, and so on. With 2^p cells, a midpoint's cell number is $\text{floor}(2^p\mu)$. The first positive p producing different numbers is the power. Distinct midpoints eventually separate.
3. Keep a current run C in addition to the stack. Initially C is the first run. When the next run N arrives, calculate boundary power p from N and its original predecessor. While the stack's top power is greater than p , pop that entry and stably merge its saved left run with C . The merged run becomes C . This finishes deeper merges before introducing the new higher-level boundary.
4. When no top entry needs removal, push (C,p) , then set $C=N$. Continue scanning. Pending powers increase from the bottom toward the top; they describe boundaries, not the numerical priority of records.
5. At end of input, repeatedly pop a saved left run and merge it with C . Update C after each merge. When the stack is empty, C covers the whole input. Use ordinary stable merging: compare the two unused fronts, choose the smaller, favor the left on equality, and append the unfinished tail.
6. Avoid floating-point boundary mistakes by writing each midpoint as $(2s+L)/(2n)$. At a level p , compare the integer quotients $\text{floor}(2^p(2s+L)/(2n))$. A careful implementation may work through fraction bits instead of forming huge products, but it must preserve the same ruler-cell test.

Walk through the numbers

1. Use $[1,8,2,0,3,4,5,6]$. Scanning nondecreasing runs gives $R1=[1,8]$, $R2=[2]$, $R3=[0,3,4,5,6]$, with lengths 2,1,5 and starts 0,2,3. Since $n=8$, their relative midpoints are $1/8$, $2.5/8$, $5.5/8$.
2. $R1$ and $R2$ share the left half, but their quarter-cell numbers are 0 and 1. Their boundary power is 2. $R2$ is in the left half and $R3$ in the right half, so their boundary power is 1. The test uses positions, not a comparison between values 8 and 2.

3. Initially $C=R1$. On finding $R2$, push $(R1,2)$ and set $C=R2$. On finding $R3$, its new power 1 is smaller than the saved 2. Pop and merge $[1,8]$ with $[2]$, selecting 1,2,8. Push $([1,2,8],1)$ and make $R3$ the current run.
4. Input has ended. Pop the last left run and merge it with $[0,3,4,5,6]$. Select 0, then 1,2, then 3,4,5,6, and finally 8. Output is $[0,1,2,3,4,5,6,8]$. The merge lengths total $3+8=11$. Merging the last two original runs first would total $6+8=14$.

How much work and room it needs

Finding runs reads n records. The major remaining cost is the sum of all merge lengths, since each merge visits its participants. Powersort keeps that cost close to the best adjacent-run schedule. Typical and worst time are $O(n \log n)$; one run gives best $\Theta(n)$. More precisely, the bound is $O(n(1+H))$, with $H=\sum(L_i/n)\log_2(n/L_i)$, where L_i is run i 's length and r is the run count. H measures a length-based merge-level scale: it is zero for one run, $\log_2 r$ for r equal-length runs, and never exceeds $\log_2 r$. Conventional extra space is $O(n)$ for merging and $O(\log n)$ for scheduling. Stable. Near optimality permits an additive $O(n)$ difference rather than claiming the perfect schedule for every input.

Why it is fast or slow

Small neighboring regions combine before large ones. Run length affects position on the ruler, so the schedule avoids repeatedly dragging a long run into merges merely because it happened to be encountered first.

When it is useful

An advanced continuation after ordinary merging, especially for uneven run lengths. It can be combined with TimSort's extension and galloping operations. Beginners can return here after understanding natural mergesort.

Details that make the procedure work

The example deliberately recognizes nondecreasing runs only. An implementation that also reverses strict descents could recognize $[2,0]$ as a run, changing the boundaries and powers. Equal records must not be reversed. Multiway Powersort uses a multi-cell hierarchy and multiway merging rather than this binary schedule.

References [\[27\]](#) [\[28\]](#) [\[29\]](#)

16 Peeksort

Type Important variant Aliases or related names Midpoint-peeking natural mergesort

First understand the small actions

Recursion asks the same procedure to solve a smaller region, then combines the answers. It needs stopping cases so it does not keep calling itself forever.

A known ordered prefix touches the region's left end; a known ordered suffix touches its right end. Remembering their boundaries avoids checking the same confirmed run again.

A stable merge compares the unused fronts of neighboring ordered regions, chooses the smaller, and chooses the left record on equality.

The idea in plain words

Ordinary top-down mergesort cuts at the middle even if that cut splits a long run that was already ordered. Peeksort first looks around the middle to discover the run containing it. It moves the cut to the nearer run boundary, preserves the run, sorts the remaining regions, and merges them. This is a way to plan from the whole problem downward. Powersort plans in the other direction, collecting runs from left to right.

Carry it out one action at a time

1. Work on inclusive positions $[l, r]$. Return for an empty region, one item, or a known ordered run covering the whole region. Carry e , the end of a known left prefix, and s , the start of a known right suffix. Initially the outer single items can serve as these known runs.
2. Choose $m = \text{floor}((l+r)/2)$. If m lies in the known prefix, keep $[l, e]$ untouched, recursively sort $[e+1, r]$, and merge. If m lies in the known suffix, sort $[l, s-1]$ and merge it with $[s, r]$. Do not scan the already known run again.
3. Otherwise expand from m through nondecreasing neighbors. Going left is allowed while the left value is no greater than the right. On reaching the known prefix, compare only the joining pair: if the prefix's last value can precede the discovered run's first, extend the start directly to l without walking the prefix again. Do the symmetric join test at the known suffix and extend directly to r when it fits. Stop at a drop. This finds the complete midpoint run $[i, j]$ while skipping confirmed interiors.
4. Compare $m-i$ with $j-m$. If $m-i < j-m$, cut before i , giving $[l, i-1]$ and $[i, r]$. Otherwise cut after j , giving $[l, j]$ and $[j+1, r]$. If the run touches an outer end, use its internal boundary instead. A run covering the whole region is already a stopping case.
5. Recursively sort the two regions. Pass the discovered run to the region containing it: a cut before i makes it that region's known prefix; a cut after j makes it the other region's known suffix. Retain any other boundary information that belongs to each child region. This is how later calls avoid rescanning long runs.
6. Stably merge the two returned regions back into the original interval. A shared buffer is sufficient; recursive calls can change position boundaries rather than copy the entire input. The last outer merge completes the sort.

Walk through the numbers

1. For $[1, 8, 2, 0, 3, 4, 5, 6]$, the outer region is 0 through 7 and $m=3$, holding 0. To the left, $2 > 0$ stops expansion. To the right, $0 \leq 3 \leq 4 \leq 5 \leq 6$ continues to the end. The midpoint run is $[0, 3, 4, 5, 6]$ at positions $[3, 7]$. Only nondecreasing runs are recognized here.
2. The distances from m to the run ends are 0 and 4, so cut before position 3. The left problem is $[1, 8, 2]$. The right problem is a known complete run; passing its prefix end as 7 lets that call return immediately.
3. In the left region 0 through 2, midpoint 1 holds 8. Its run is $[1, 8]$ at positions $[0, 1]$. Since it touches the left boundary, cut after position 1. Keep $[1, 8]$, return immediately on singleton $[2]$, then merge by selecting 1, 2, 8.
4. The outer merge combines $[1, 2, 8]$ with $[0, 3, 4, 5, 6]$. Select 0, then 1, 2, then 3, 4, 5, 6, then 8. Final output is $[0, 1, 2, 3, 4, 5, 6, 8]$. The five-item right run was never broken into five singleton sorting tasks.

How much work and room it needs

Run discovery checks neighboring records, and merging visits every participating record. Remembered prefix and suffix boundaries avoid repeated discovery scans. Cuts near the middle control the merge levels. Best time is $\Theta(n)$ for one run; unstructured typical and worst time are $O(n \log n)$. The finer bound is $O(n(1+H))$, where n is the record count, L_i is run i 's length, and $H = \sum (L_i/n) \log_2(n/L_i)$. H is zero for one run, $\log_2 r$ for r equal-length runs, and at most $\log_2 r$. Merge cost is near the best adjacent-run plan, allowing additive $O(n)$. Conventional buffer space is $O(n)$, with $O(\log n)$ recursion information for standard scheduling. Stable.

Why it is fast or slow

Moving the cut to a run's edge preserves useful order instead of breaking and rebuilding it. Remembering discovered boundaries prevents a long run from being walked repeatedly merely to confirm the same fact.

When it is useful

An advanced step after top-down merging. If recursion is unfamiliar, learn natural mergesort's pairwise rounds first, then return to these region and boundary rules.

Details that make the procedure work

This description and example recognize maximal nondecreasing runs only. A version that reverses strict descents could discover [2,0] as a run and would need different example boundaries. The midpoint starts the search; it need not be the cut. Every recursive child must be smaller, and complete runs, empty regions, and singletons stop recursion.

References [\[30\]](#) [\[28\]](#)

17 Insertion sort

Type Algorithm Aliases or related names Straight insertion sort; linear insertion sort

First understand the small actions

An array is a row of numbered slots. Indices here start at zero: index 0 is the first slot from the left, index 1 the second, and so on. An index is a position, not its stored value. Moving a value means writing it into another slot. Keep a temporary copy before overwriting the slot that holds the value you are inserting.

An ordered prefix is the left part of the row in the correct internal order. Its items may still move when a smaller item arrives later.

The idea in plain words

Imagine holding some cards in order, then picking up one more card. You slide the larger cards to the right and place the new card in the opening. Insertion sort uses that same small operation throughout an array. It starts with one ordered item, adds the next item, and keeps growing the ordered part until nothing remains outside it.

Carry it out one action at a time

1. Treat the first item as ordered: a single item cannot be out of order with itself. Starting at the second slot, save the next value x in a temporary variable. Think of its old position as a hole, even if the old bits are still there.
2. Compare x with the item immediately left of the hole. If that item is greater, copy it one slot right into the hole. The hole has moved left. Repeat. Working backward preserves every value before its slot is overwritten.
3. Stop when the left neighbor is no greater than x , or when the hole reaches the start. Everything farther left is also small enough, because that part was already ordered. Write x into the hole.
4. The ordered prefix now contains one additional item. Take the next item and do the same thing. After the last insertion, the prefix is the whole array. Move strictly greater items only; leaving equal items in place preserves their arrival order.

Walk through the numbers

1. Start with [4,2,3,1]. The first prefix is [4]. Hold 2. Since 4 is greater, copy 4 right: [hole,4,3,1]. There is no item left of the hole, so write 2 there: [2,4,3,1].

2. Now hold 3. Move 4 right to make [2,hole,4,1]. The next comparison is with 2. It is smaller, so leave it alone and fill the hole with 3: [2,3,4,1].
3. Hold the final 1. Move 4, then 3, then 2 one slot right. Only after those moves do you write 1 in the first slot. The array becomes [1,2,3,4]. Each completed insertion adds exactly one record to the ordered prefix.
4. With labeled duplicates 2A and 2B, an arriving 2B does not push an existing 2A aside. Equality stops the search, so A stays before B.

State trace C compare M move S ordered

Hold 2 and compare with 4



Insert 2: first two internally ordered



Insert 3: shift 4 right



Insert 1 at the start: fully sorted



How much work and room it needs

Count the comparisons and the items shifted. Ordered input needs about one comparison per arriving item and no shifts: best $\Theta(n)$. On reversed input, the shifts add up to $1+2+\dots+(n-1)$, making the worst $\Theta(n^2)$. Random input usually has quadratic movement too. A more precise bound is $\Theta(n+I)$, where n is the number of records and I is the number of inversions: an earlier record greater than a later one. Each shift removes one such pair. Extra space is $\Theta(1)$ for a saved record and position counters. The strict-greater shifting rule is stable.

Why it is fast or slow

A nearly ordered row gives each new item only a short journey. In a reversed row, every new item travels all the way to the beginning and moves every earlier item again. That repeated movement, rather than a complicated operation, explains the difference.

When it is useful

Good for small arrays, nearly ordered data, and short pieces inside a larger sorting method. It can also maintain an ordered collection as new items arrive, provided the collection is not too large.

Details that make the procedure work

Empty and one-item inputs finish immediately. Track the prefix with a boundary index; there is no need to copy it. A linked list avoids moving whole array sections, but it still needs to find the insertion position. Finding a position quickly and physically making room are separate costs.

References [31] [32]

18 Binary insertion sort

Type Important variant Aliases or related names Insertion sort with binary position search

First understand the small actions

Binary search uses an ordered row to rule out half the possibilities at a time. Looking at the middle tells you which side could contain the insertion point.

Positions begin at zero here. The interval $[lo,hi)$ includes lo and excludes hi . When lo equals hi , there are no unchecked slots left.

The idea in plain words

Ordinary insertion sort may inspect many old items just to discover where the next one belongs. Binary insertion sort asks a better sequence of questions. It checks the middle of the ordered part, keeps the relevant half, and repeats. The new record still needs an opening in the array, so this method saves comparisons rather than eliminating movement.

Carry it out one action at a time

1. Keep the left prefix ordered, as in insertion sort, and save the next record x . If the prefix has i items, start with $lo=0$ and $hi=i$. Only this ordered prefix is searched.
2. Compute mid as the average of lo and hi rounded down. If x is smaller than the value at mid , set $hi=mid$: x must be inserted at that position or earlier. Otherwise set $lo=mid+1$. Taking the right branch on equality puts a new equal record after older equal records.
3. Repeat while $lo < hi$. Every comparison shrinks the interval, so it must eventually become empty. Its remaining boundary lo is the insertion position, which can be the start or the position just after the prefix.
4. Move the prefix items at and after that position one slot right, starting at the right end. Write x into the opening. Then extend the prefix and process the next record. An optional check against the prefix's last record can skip the search when x already belongs at the end.

Walk through the numbers

1. Use the full input $[1,3,5,7,4]$. The first four values can each join the ordered prefix at its end. When 4 arrives, search $[1,3,5,7]$ with $[lo,hi)=[0,4)$.
2. The middle position is 2, holding 5. Since $4 < 5$, change the interval to $[0,2)$. Its middle is now position 1, holding 3. Since $4 \geq 3$, move lo to 2. The interval $[2,2)$ is empty, so the opening belongs at position 2.
3. The search did not create that opening. Copy 7 to position 4, then 5 to position 3. Now $[1,3,hole,5,7]$ has room for the saved 4, giving $[1,3,4,5,7]$.
4. If another 3 arrived instead, the equality branch would continue to the right of the existing 3. That choice preserves duplicate records and their original order.

How much work and room it needs

A search through i ordered items takes about $\log_2 i$ comparisons because the remaining range halves each time. Across n insertions, typical and worst comparison counts are $\Theta(n \log n)$. However, an insertion can still shift i items. Typical and worst movement is $\Theta(n^2)$, so total array time is still $\Theta(n^2)$. Without an end check, ordered input also needs $\Theta(n \log n)$ comparisons. With the end check, best time becomes $\Theta(n)$. Here n is the record count. Extra space is $\Theta(1)$, and inserting after equal records makes this version stable.

Why it is fast or slow

It helps when comparing two records is expensive, such as comparing long text. It helps much less when moving records dominates the work. Quickly locating the place between 3 and 5 does not remove the need to move 5 and 7.

When it is useful

Useful for short ordered prefixes, expensive comparisons, and stable insertion inside hybrid methods. TimSort uses this operation to extend short runs.

Details that make the procedure work

Keep the same half-open interval convention throughout. Move items backward when opening a slot to avoid overwriting them. Do not report $O(n \log n)$ total time merely because the search uses that many comparisons.

References [\[33\]](#) [\[24\]](#)

19 Library sort

Type Important variant Aliases or related names Gapped insertion sort

First understand the small actions

A gap is an empty-slot flag, not a special numerical value. Read occupied slots from left to right to obtain the current ordered sequence.

Random arrival means shuffling the records before insertion. The final ordering rule stays the same; randomness spreads the pressure from insertions.

At a redistribution, record the positions of the existing items. These support positions stay occupied for the next round, even though shifting may change which records occupy them.

The idea in plain words

Picture a crowded bookshelf. Adding a book near the beginning requires moving many books. If there are gaps along the shelf, you can often move a short section into a nearby gap instead. Library sort maintains an ordered sequence in a partly empty array and occasionally spreads the records out again. Its speed guarantee comes from a particular randomized procedure, not from gaps alone.

Carry it out one action at a time

1. Determine the final count n and reserve about $(1+\epsilon)n$ slots, for a fixed positive spare fraction ϵ . Shuffle the arrival order. Work in rounds: a round starting with m records inserts up to m more, roughly doubling the population. Stop the final round early when all n records have arrived.
2. At the start of a round, spread the m existing records evenly over the active area sized for that round's target population. Save their slot numbers as support positions. Binary-search the current values at these occupied positions to bracket a new x . Within the bracket, inspect occupied slots in order to find the exact predecessor and successor. At the outer edges, use the active area's boundaries.
3. If a gap already lies between that predecessor and successor, put x there. Otherwise look for a nearby gap. For a gap on the right, shift the intervening consecutive records right, beginning next to the gap. For a gap on the left, shift them left, beginning next to that gap. Place x in the resulting opening between its ordered neighbors.
4. A shift followed by insertion never leaves an originally occupied support position empty. Their current values therefore remain safe to search during the round. At the scheduled population limit, redistribute over a larger active area and record fresh support positions. This activates more of the array reserved initially; a fixed array does not grow by itself.

5. After the last insertion, collect occupied slots from left to right into a packed output. To preserve original order among equal keys despite shuffling, compare the original position after comparing the key.

Walk through the numbers

1. Let the arrival order be $[4,1,7,3,2]$. Four starts the shelf. Make room to its left for 1, giving occupied records $[1,4]$. Seven follows them, so the record sequence is $[1,4,7]$. Empty slots do not enter this sequence.
2. To isolate the local insertion mechanism, show those three records spread as $[1,\text{gap},4,\text{gap},7,\text{gap}]$, with support positions $0,2,4$. This is an illustrative spread, not a claim that a formal round ends after three insertions. Three belongs between 1 and 4 and can use the gap directly: $[1,3,4,\text{gap},7,\text{gap}]$.
3. For 2, searching the support values $1,4,7$ identifies the interval between 1 and 4. Inspecting that interval locates the exact position between 1 and 3. There is no gap there, but position 3 is empty. Copy 4 from position 2 to 3, then 3 from 1 to 2. Write 2 at position 1: $[1,2,3,4,7,\text{gap}]$.
4. A later scheduled spread could look like $[1,\text{gap},2,\text{gap},3,\text{gap},4,\text{gap},7,\text{gap}]$. Collecting the records produces $[1,2,3,4,7]$. Ten reserved slots for five records illustrate $\epsilon=1$. The actual rounds follow their doubling schedule and stay within the reserved capacity.

How much work and room it needs

Searching support values costs about $\log n$ comparisons per insertion. Redistribution moves each existing record once. The uncertain part is how far an insertion must search for a gap and how many records it shifts. With randomized arrivals, fixed spare fraction, and the paper's rounds, congestion is controlled: expected total time is $O(n \log n)$, and the same bound holds with high probability. This means a shrinking probability of failure as size grows, not certainty. Adverse arrivals or rare unlucky outcomes can still require $\Theta(n^2)$ work. A standard version that always binary-searches generally has $O(n \log n)$ best work too. Extra space is $\Theta(n)$. The shuffled basic method is unstable; key-plus-original-position comparisons restore stability.

Why it is fast or slow

Nearby empty space turns a large move into a small one. Random arrivals help avoid repeatedly crowding the same region. Redistribution replenishes gaps where earlier insertions consumed them. Position searches still carry a logarithmic cost.

When it is useful

Useful for studying ordered arrays with spare space and randomized performance guarantees. In practice, the extra storage and gap management must justify their complexity; ordinary insertion sort is simpler for small inputs.

Details that make the procedure work

Do not binary-search gaps as though they were keys. Search occupied support positions, then inspect the bounded local region. Keep a fixed positive spare fraction, respect active boundaries, and handle a partial final round. Different indexing or redistribution schemes need their own analysis rather than borrowing the original theorem.

References [\[34\]](#)

20 Patience sort

Type Algorithm Aliases or related names Patience sorting; card-pile sorting

First understand the small actions

A pile lets you remove its top item first. Here each pile is nondecreasing from top to bottom, so its top is its smallest remaining item.

A priority queue holds candidates and can remove the smallest without keeping every candidate in a fully sorted row. A binary min-heap makes each parent no greater than its children, placing the minimum at the root.

Binary search is applied to the ordered list of pile tops, not to the records inside a pile.

The idea in plain words

Deal the values into piles. Put a new value on the leftmost pile whose top is at least as large; if none qualifies, start a new pile on the right. The piles each become ordered from top downward, and their top values stay ordered from left to right. This dealing stage is only preparation. To finish sorting, repeatedly take the smallest available pile top and reveal the next item below it.

Carry it out one action at a time

1. Read x and binary-search the top-value list for its first value $\geq x$. At a middle position, keep the left half including that position if its value qualifies; otherwise keep the right half. Put x on the selected pile, or create a new rightmost pile if the search finds none. Keep every item in the piles.
2. The top list stays ordered because every earlier top is smaller than x ; otherwise that earlier pile would have been selected. Every later top was at least the selected old top and therefore at least x . Within the selected pile, x is no greater than the item it covers.
3. After dealing, place one candidate from each nonempty pile into a minimum priority queue. Store its pile identity too. A binary heap can use an array: zero-based position i has parent $\text{floor}((i-1)/2)$ and children $2i+1, 2i+2$, if those positions exist.
4. To insert a candidate, append it and exchange it upward with a larger parent until the parent relation holds or it reaches the root. To remove the minimum, save the root, move the last entry to the root, then exchange it downward with the smaller child while that child is smaller. This repairs a path rather than sorting the whole queue.
5. Emit the removed candidate and take it off its pile. If that pile still has records, insert its newly exposed top into the queue. Otherwise it supplies no further candidate. An empty queue means every record has been emitted.

Walk through the numbers

1. Deal [4,2,3,1]. Four starts pile one. Two covers it, making top-to-bottom [2,4]. Three is greater than the only top 2, so it starts pile two [3]. One covers pile one's top. The piles are [1,2,4] and [3]; their top list is [1,3].
2. The queue begins with (1,pile one) and (3,pile two). Remove 1, output [1], and insert the newly exposed 2. Remove 2 next, output [1,2], and insert the exposed 4.
3. Candidates are now 3 and 4. Choose 3, output [1,2,3], and leave its empty pile out of the queue. Choose the final 4. Output is [1,2,3,4], and the queue is empty. Emptying the first pile completely before the second would incorrectly produce [1,2,4,3].
4. Under the basic $\geq$ rule, a new equal value covers an old equal value and comes out first. For stable sorting, use the first top $> x$ instead, and resolve equal queue candidates by their original positions.

How much work and room it needs

Each record is dealt using a binary search, then emitted with a queue removal; exposing it may also cause an insertion. With p piles, those searches and heap repairs travel about $\log(p+1)$ levels, giving $O(n \log(p+1))$ total work. Here n is the record count. Strictly decreasing input gives one pile and best $\Theta(n)$. Random distinct input typically, and the worst case, give $\Theta(n \log n)$. Strictly increasing input makes n piles. Full piles, candidates, and output need $\Theta(n)$ extra space. The basic $\geq$ version is unstable; the stated strict-greater and position-tie variant is stable.

Why it is fast or slow

Records under a smaller pile top can wait without joining the current comparison. Fewer piles mean fewer candidates and shorter searches. If nearly every record starts its own pile, that advantage disappears.

When it is useful

Good for learning multi-sequence merging, priority queues, and how input order affects work. Pile count is also connected to longest increasing subsequences, but computing a subsequence length is not the same as producing sorted output.

Details that make the procedure work

Keeping only pile tops would discard records. Ordered tops do not mean the entire input is sorted. Each queue entry needs a pile identity so removal can expose the correct next item. Empty input immediately yields empty output.

References [\[35\]](#) [\[36\]](#) [\[37\]](#)

21 Smoothsort

Type Important variant Aliases or related names Dijkstra's adaptive in-place heapsort

First understand the small actions

A tree connects a parent record to children. In a max-heap, every parent is at least its children, so the root is largest. This does not put every record in sorted order.

A forest is several trees side by side. Here each tree occupies a consecutive array block, with its root in that block's last slot.

Leonardo sizes are $L(0)=L(1)=1$ and $L(k)=L(k-1)+L(k-2)+1$: 1,1,3,5,9,15,... . A larger block consists of a left child block, a right child block, and one root. Orders 0 and 1 remain distinct despite both having size one.

The idea in plain words

Like heapsort, Smoothsort settles the largest remaining record at the right end. Instead of one fixed-shape heap, it organizes the unfinished prefix into a forest of special-sized heaps. Each heap root is largest within its block, and the roots are kept nondecreasing from left to right. The rightmost root is therefore the largest record in the whole unfinished prefix. If the data already fits these relations, little repair is needed. The following clear version fully repairs every step; optimized implementations can postpone repairs that later joins make unnecessary.

Carry it out one action at a time

1. Build the prefix left to right and track each block's order. The first singleton has order 1. If the last two heap orders are $k, k-1$, combine them and the new record into order $k+1$, putting the new record at the block's end as root. Otherwise append a singleton: use order 0 when the previous last order is 1, and order 1 otherwise. No size-two Leonardo heap is required.

2. First learn the internal repair, sift. An order- k block for $k \geq 2$ contains an order- $(k-1)$ left block, an order- $(k-2)$ right block, then the root. If the root is at t , the right child root is $t-1$ and the left is $t-1-L(k-2)$. Choose the larger child. If it exceeds the root, exchange them and continue inside that child heap. Stop when the root dominates its children or the block is a singleton.
3. The forest also needs root-order repair, called trickle. For current root position t and heap order k , the preceding heap's root is at $t-L(k)$; do not scan to find it. Exchange the roots only if that preceding root exceeds the current root and both current children, when children exist. Then move the repair position to the preceding heap and continue leftward. The large root moved right still dominates that right heap's children.
4. If the preceding root does not pass all those tests, stop moving left and sift inside the current heap. If there is no preceding heap, sift there as well. This repairs the internal heap whose root was reduced. After every addition, fully trickle the newly formed rightmost heap to restore both heap relations and ordered roots.
5. Once construction finishes, leave the rightmost root in place as the final maximum and shorten the unfinished prefix by one. Delete a singleton heap description. For order $k \geq 2$, removing the root exposes its order- $(k-1)$ left heap and order- $(k-2)$ right heap. Their internal heaps remain valid, but the exposed roots may violate forest order. Trickle the left exposed heap first, then the right.
6. Repeat maximum removal, deletion or splitting, and root repair. The final suffix grows by one slot per removal. Never exchange a record in that settled suffix. An empty unfinished prefix ends the algorithm.

Walk through the numbers

1. Build $[3,4,1,5,2]$. Values 3 and 4 form singleton orders $[1,0]$, with roots already ordered. Adding 1 joins them into an order-2 heap of size three, $[3,4,1]$. Root 1 is smaller than child 4, so exchange them: $[3,1,4,5,2]$. The first heap's root is now 4.
2. Add singleton 5, making orders $[2,1]$ and roots $4 \leq 5$. Adding 2 joins them into an order-3 heap of size five. Its child roots are 4 and 5. Exchange root 2 with larger child 5: $[3,1,4,2,5]$. The final 5 is now settled.
3. Removing 5's root description exposes $[3,1,4]$ and singleton $[2]$. Root 4 exceeds the following root 2. Trickle exchanges them, giving $[3,1,2,4,5]$. The decreased left root 2 is below its child 3, so sift yields $[2,1,3,4,5]$. The next rightmost maximum 4 is settled.
4. Delete singleton 4 and settle root 3. Its heap splits into singletons 2 and 1. Repair their reversed root order to obtain $[1,2,3,4,5]$, then settle 2 and 1. A block such as $[3,1,4]$ is a heap, not an already sorted run.

How much work and room it needs

Repair can walk downward through heap levels or leftward through forest roots, with $O(\log n)$ work for a difficult operation. Unstructured typical and worst total time are $O(n \log n)$. On increasing input, the required relations already hold, so repairs usually stop immediately; amortized shape updates give best $\Theta(n)$. Here n is the record count. Standard implementations encode the shape in a constant number of machine words large enough to represent n , using $O(1)$ extra space. An explicit educational list of heap orders uses $O(\log n)$ instead. Normally unstable: distant exchanges can reorder equal records.

Why it is fast or slow

Already suitable heap and root relationships are retained rather than rebuilt blindly. Disordered data still requires movement through logarithmic levels. This adaptation does not mean every possible measure of near-sortedness guarantees linear time.

When it is useful

An advanced study of adaptive in-place heap sorting. Learn ordinary parent-child sifting before forest-root tinkling. This entry can wait until simpler sorting methods are familiar.

Details that make the procedure work

The fully repaired version exposes the required relations; optimized code may perform fewer construction exchanges. Distinguish the two singleton orders and never invent a size-two Leonardo heap. Scanning every root for every maximum loses the linear best case. These child positions come from consecutive Leonardo blocks, not the ordinary binary-heap formula $2i+1$.

References [\[38\]](#) [\[39\]](#)

22 Strand sort

Type Algorithm Aliases or related names Repeated monotone-subsequence extraction and merging

First understand the small actions

A subsequence keeps the original order of selected items but may skip items between them. Selecting 4 and 5 from [4,1,3,2,5] is allowed even though they are not neighbors.

To merge two ordered sequences, compare their first unused items. The smaller one is the smallest remaining item overall, because neither sequence hides a smaller item behind its front.

The idea in plain words

Walk through the input and collect a chain that keeps going upward. Leave any item that cannot follow the chain for a later walk. Each collected chain is already ordered, so merge it into the ordered result. Strand sort uses these possibly separated subsequences; natural mergesort instead discovers consecutive stretches.

Carry it out one action at a time

1. Begin with an empty result. If unprocessed items remain, remove their first item and use it to start a new strand. A selected item must leave the remainder so it cannot be selected again.
2. Scan the other remaining items from left to right. If an item is at least the strand's last value, remove it and append it. Otherwise retain it in the remainder. Keep retained items in their original order.
3. Merge the strand into the old result. Keep one pointer at each sequence's first unused record. Output the smaller front and advance only that pointer. When one sequence ends, copy the other tail. On equal values, choose the old result first.
4. Repeat with the remainder until it is empty. For an array, build the next remainder during the scan rather than deleting each selected element with a separate tail shift. A linked list can detach and reconnect nodes instead.

Walk through the numbers

1. Start with [4,1,3,2,5]. Four begins the first strand. Values 1,3,2 are too small to follow 4, so retain them. Five fits. Strand [4,5] becomes the first result, and [1,3,2] remains.
2. The next strand starts with 1. Append 3, but retain 2 because it is smaller than the current last value.
 3. Merge [1,3] with [4,5]: select 1 against 4, then 3 against 4, then copy 4 and 5. The result is [1,3,4,5], with remainder [2].
3. The last strand is [2]. During its merge, select 1 against 2, then select 2 against 3. The strand ends, so copy 3,4,5. Final output: [1,2,3,4,5].

4. Allowing equality within a strand keeps duplicate records. Choosing older result records first on a merge tie, together with order-preserving extraction, preserves their original order.

How much work and room it needs

Each round scans the remainder and merges with the growing result. If the first strand takes everything, only one scan and output are needed: best $\Theta(n)$. A reversed input gives one-item strands. The repeated scans have lengths $n-1, n-2, \dots$, and merging repeatedly revisits the old result, giving worst $\Theta(n^2)$. Typical work depends on the available subsequences; no universal average bound is assumed. Here n is the record count. Conventional output and remainder storage use $\Theta(n)$ extra space; lists can reuse nodes. The stated extraction and tie rules make this version stable.

Why it is fast or slow

Long strands remove many records per round. Short strands force repeated visits to nearly the same records and repeated merges of an already large result. There is no rule ensuring that every round halves the remaining work.

When it is useful

Helpful for learning subsequence extraction and merging, or for lists that naturally contain a few long increasing strands. General array sorting usually benefits from a method with a guaranteed $O(n \log n)$ worst case.

Details that make the procedure work

Some original descriptions require a strictly greater value before appending. That also sorts correctly but puts equal values into more strands. This entry uses at-least instead. Empty input needs no strand, and every skipped record must remain available. During merging, advance the side you actually selected.

References [\[40\]](#)

23 Cartesian tree sort

Type Algorithm Aliases or related names Cartesian-tree priority-queue sort; Levkopoulos–Petersson adaptive heapsort

First understand the small actions

A node is a record with left and right links to other nodes. A missing child has an empty link. The top node is the root.

Inorder visits the left subtree, the node, then the right subtree. Here it preserves the original input order. Separately, min-heap order requires a parent value to be no greater than its children.

A stack removes the most recently added entry. A priority queue removes the smallest entry. Tree construction uses the former; sorted extraction uses the latter.

The idea in plain words

Could we build a tree that puts the smallest record of every subtree at that subtree's top? A Cartesian tree does this while also preserving the input: walking left, node, right reproduces the original sequence. These two rules coexist, but neither means that the tree can simply be traversed to obtain sorted output. For sorting, output the root, expose its children as candidates, and repeatedly choose the smallest exposed candidate.

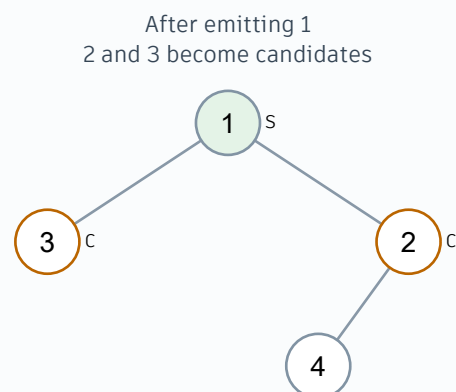
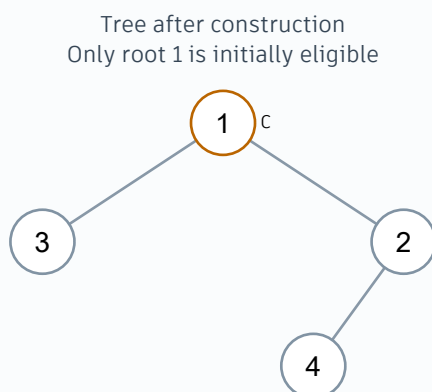
Carry it out one action at a time

1. Create a separate node for each input record, initially with empty child links. Keep a stack whose values are nondecreasing from bottom to top; it represents the current tree's rightmost path. For a new node x , start last as empty. While the top value exceeds x , pop it and assign that popped node to last. The final popped node is the root of the whole portion moved aside.
2. Set x 's left child to last. If a stack top remains, replace its right link with x . If the stack is empty, make x the overall root. Push x . The previous right portion has not been lost: it now hangs from x 's left link. Since x is the new final input record, that placement preserves inorder input order.
3. Keep the old links within popped nodes. They already connect the moved subtree, so attaching its root is enough; do not rebuild every node individually. Each node is pushed once and popped at most once. After the input ends, the saved root and child links describe the entire Cartesian tree.
4. For sorted extraction, initialize a minimum priority queue with the root only. Candidates are unprinted nodes whose parents have been printed. Each is the minimum of its still-unprinted subtree. All hidden records lie under such a candidate, so the smallest remaining record must be among the candidates.
5. Remove the smallest candidate, append its record to output, and insert its existing children. A separate binary min-heap can implement the queue: append a new entry and exchange upward past larger parents; on removal, replace the root with the last entry and exchange downward with the smaller child until heap order returns. This candidate heap is not the Cartesian tree.
6. Continue until the queue is empty. Every node is emitted once. For stable duplicates, consistently compare (key, original position) during both construction and extraction. A smaller original position wins when keys tie.

Walk through the numbers

1. Build [3,1,4,2]. Reading 3 makes it the root and gives stack [3]. Reading 1 pops 3, sets 1.left=3, and makes 1 the new root because the stack is empty. Stack: [1].
2. Reading 4 pops nothing because $1 \leq 4$. Set 1.right=4 and push 4, giving [1,4]. Reading 2 pops 4. Set 2.left=4, then replace the remaining top node's right link: 1.right=2. Stack: [1,2]. The old node 4 survives beneath 2.
3. The final tree has root 1, left child 3, right child 2, and 2's left child 4. Inorder visits 3, then 1, then 4, then 2: [3,1,4,2]. It reconstructs the input, demonstrating why inorder is not the sorting step.
4. Start candidates [1]. Emit 1 and expose 3 and 2. Choose 2, output [1,2], and expose 4. Candidates are now 3 and 4. Emit 3, then 4, to finish [1,2,3,4]. Both children join the candidate queue; traversing the entire left subtree first would be unjustified.

Cartesian tree and eligible frontier for [3,1,4,2]



How much work and room it needs

Construction pushes and pops each node at most once, taking $\Theta(n)$. Extraction inserts and removes each candidate once, with a heap path of about $\log(q+1)$ levels. Its upper bound is $O(n \log(q_{\max}+1))$, where $q_{\max}$ is the largest candidate count and n is the record count. A chain keeps one candidate and gives best $\Theta(n)$. General typical and worst sorting time are $O(n \log n)$; linear tree construction does not imply linear sorting. Nodes, stack, queue, and output use $\Theta(n)$ extra space. Consistent key-plus-original-position comparisons make it stable; arbitrary equal-key handling does not.

Why it is fast or slow

The tree establishes which local minima may become eligible first. Only exposed candidates need comparison. A chain leaves almost no choice; many exposed branches still require logarithmic queue work.

When it is useful

An advanced example of preserving input order while organizing local minima. It is related to range-minimum structures as well. Learn nodes, stacks, and priority queues before using this entry as an implementation guide.

Details that make the procedure work

This is not a binary search tree: a left child need not have a smaller value. Keep a separate node for each duplicate. Repeatedly scanning intervals for their minimum can make construction $\Theta(n^2)$; the stack avoids that. The Cartesian tree may be very unbalanced, but the candidate binary heap is a different structure and does not inherit that height.

References [\[41\]](#) [\[42\]](#)

3 General comparison sorts and conditional optimizations

For large arbitrary arrays, guaranteed or aggregate-amortized $n \log n$ methods precede methods whose efficiency relies on pivots or input structure. Shellsort bounds depend on its gaps. Cache behavior, comparison counts, and large-record movement receive separate treatment toward the end; these measures do not imply a measured timing rank.

24 Top-down merge sort

Type Algorithm Aliases or related names Recursive merge sort; a standard merge-sort implementation

First understand the small actions

An array is a row of numbered slots. A range means that we work on only a consecutive part of that row.

In an ordered row, the first remaining item is the smallest remaining item. Two ordered rows therefore need only their two front items compared.

Recursion means asking the same procedure to handle a smaller range. An empty range or one item stops immediately.

The idea in plain words

Break a row in half, then break each half again until every piece has one item. Work back upward, joining two ordered pieces into a larger ordered piece. Joining is a sequence of small choices: compare the two available fronts and take the smaller.

Carry it out one action at a time

1. For the current range, stop at zero or one item. Otherwise split near its middle. Finish sorting the left range, then the right range, by these same instructions.
2. Once both ranges are ordered, remember a next-item position for each. Such a remembered position is called a pointer here; it need not be a computer memory address. Prepare temporary output slots for their combined length.
3. Compare the two next items. Copy the smaller complete record into the next output slot. Advance only the pointer that supplied it, then advance the output position. On equality, take the left record first.
4. Keep comparing while both ranges have items. If one is exhausted, copy the remaining items from the other in their existing order. Their order is already known.
5. Copy the temporary result back into the original range. Return to the larger task, which can now treat this whole range as an ordered input.

Walk through the numbers

1. Start with [4,1,3,2]. Split into [4,1] and [3,2], then into [4], [1], [3], [2]. Singletons require no action.
2. Join [4] with [1]: 1 is smaller, so write 1, then the remaining 4. This gives [1,4]. Joining [3] with [2] similarly gives [2,3]. The row is now [1,4 | 2,3].
3. For the final join, compare 1 with 2 and write 1. The left front becomes 4. Compare 4 with 2 and write 2: output [1,2].
4. Compare 4 with the new right front 3 and write 3. The right side is empty, so append 4. Copy [1,2,3,4] back into the row.

State trace C compare M move S ordered

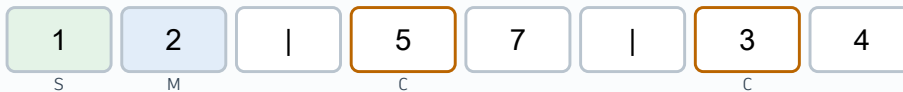
Sorted halves: left | right



Output 1 | unread left | unread right



Output 1,2 | unread left | unread right



Output 1,2,3,4 | unread left



Right exhausted; append 5,7



How much work and room it needs

Let n count records. Joining reads and writes each item in the current range. All ranges at one splitting level contain n items in total. Repeated halving creates about $\log_2 n$ levels. The basic version therefore takes $\Theta(n \log n)$ time in best, typical, and worst cases; Θ means a matching growth rate. Temporary records need $O(n)$ slots, and suspended calls need $O(\log n)$ stack space. It is stable when the left side wins ties. A boundary check that skips already ordered joins can make ordered input take $\Theta(n)$.

Why it is fast or slow

An item passes through each level once. We never compare every item in one side with every item in the other. Copying records and allocating the temporary area are the main additional costs.

When it is useful

A dependable stable sort for general records when a temporary array is affordable.

Details that make the procedure work

An ordered piece is not necessarily in final positions: $[1,4 \mid 2,3]$ still needs work. Move whole records, including associated information. A practical version allocates one reusable temporary array rather than a new large array at every call.

References [\[23\]](#) [\[43\]](#)

25 Bottom-up merge sort

Type Important variant Aliases or related names Iterative merge sort; merge-sort variant

First understand the small actions

One item is already ordered. Joining two one-item pieces creates an ordered pair; joining pairs creates ordered groups of four.

A merge pointer remembers the next unused item of a piece. The output position remembers the next empty temporary slot.

A loop repeats a round after changing its piece size. This version needs no recursive calls.

The idea in plain words

Build upward from single items. Join neighboring pieces of size one, then two, then four, doubling the size after each complete sweep. Each sweep uses the order established by the previous sweep.

Carry it out one action at a time

1. Start with piece size $w=1$. Moving left to right, select up to w items for a left piece and up to w following items for a right piece. A short final piece is allowed. If no right piece exists, leave the left piece alone.
2. Join the two pieces by comparing their next unused items. Copy the smaller into temporary output and advance its source pointer. Take the left record first on a tie. When one piece ends, copy the remaining items of the other.
3. Copy the joined result back, then advance to the next pair of pieces. After the whole sweep, groups of up to $2w$ items are ordered.
4. Double w and repeat while $w < n$, where n is the number of input items. Empty and single-item arrays require no sweep.

Walk through the numbers

1. Use [4,1,3,2,5]. With $w=1$, join [4] and [1] into [1,4], then [3] and [2] into [2,3]. The last [5] has no partner. Result: [1,4 | 2,3 | 5].
2. With $w=2$, compare the fronts of [1,4] and [2,3]. Take 1, then 2, then 3, then remaining 4. Now the row is [1,2,3,4 | 5].
3. With $w=4$, join [1,2,3,4] and [5]. The left front is smaller on four successive comparisons, so write 1,2,3,4, then the remaining 5. The whole row is [1,2,3,4,5]. The next doubled size exceeds n , so stop.

How much work and room it needs

Let n be the item count. A sweep reads and writes each participating item, and doubling needs about $\log_2 n$ sweeps. Basic best, typical, and worst time are $\Theta(n \log n)$, a matching growth rate. A reusable temporary array needs $O(n)$ slots. Loop counters need $O(1)$ space, with no recursive stack. The method is stable if left records win ties. Checks that skip already ordered boundaries can improve favorable inputs.

Why it is fast or slow

The same small front comparisons used by recursive merging appear here. Loop control saves recursive-call overhead, but temporary-record traffic remains.

When it is useful

Useful when iterative control or straightforward sequential sweeps are preferred.

Details that make the procedure work

Do not assume the array length is a power of two. Clip every piece boundary at the array end. The fifth item in this example shows why. A piece becomes an input for the next round only after its entire join has finished.

References [44] [45]

26 Heapsort

Type Algorithm Aliases or related names Binary heapsort; heap-based selection sorting

First understand the small actions

Read the array as a tree filled level by level. Its first item is the root, followed by its children, then their children. No separate tree objects are needed.

With zero-based positions, i has children $2i+1$ and $2i+2$ if those positions exist. Root position is 0.

A maximum heap requires each parent to be at least its children. Following a downward path cannot increase the value, so the root is largest. Siblings and neighboring array slots need not be ordered.

Sinking means exchanging an undersized parent with its larger child and continuing from that new position.

Floor rounds down to a whole number: $\text{floor}(5/2)=2$. Array positions must be whole numbers.

The idea in plain words

Arrange the row into a maximum heap, exchange its largest root with the last active item, and exclude that last slot. Sink the new root within the smaller heap, then repeat. Finished largest values accumulate at the right, leaving an ascending result.

Carry it out one action at a time

1. First define sinking in a heap of m items. At position i , find existing children. If there are two, compare them and select the larger, c . Stop if the parent is at least c : it is also at least the other child. Otherwise exchange parent and c , set $i=c$, and repeat. A childless position stops.
2. Build the heap by sinking parents from $\text{floor}(n/2)-1$ down to 0. Childless bottom items already satisfy the rule. Processing upward ensures each parent's child branches already satisfy it, so its own single downward path is enough.
3. After construction, exchange root 0 with $m-1$. The largest active item has reached its final rightmost place. Reduce m by one and leave that completed suffix outside the heap.
4. If $m>1$, sink the replacement root. Other branches remained ordered internally; only the moved root can violate the rule.
5. Keep exchanging, shrinking, and sinking until one item remains. Empty and singleton inputs need no construction. Child tests must use the active size m rather than the original length.

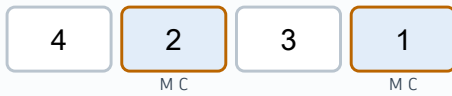
Walk through the numbers

1. For $[4,1,3,2]$, the last parent is position 1. Its value 1 has just child position 3 containing 2. Exchange them: $[4,2,3,1]$. The new position 3 has no children.
2. Root 4 has children 2 and 3. Select larger child 3; since $4 \geq 3$, stop. The heap is ready, but $[4,2,3,1]$ is not an ascending row.
3. Exchange root 4 with final 1: $[1,2,3 \mid 4]$, active size 3. Select child 3 over 2 and exchange it with root 1: $[3,2,1 \mid 4]$. That path ends.
4. Exchange root 3 with active last 1: $[1,2 \mid 3,4]$, size 2. Its only child 2 rises by exchange: $[2,1 \mid 3,4]$.

5. Exchange 2 with 1: [1 | 2,3,4], size 1. Stop. Every item right of the bar was final when placed there; all original records remain.

State trace C compare M move S ordered

Build max heap: exchange 1 and child 2



Fix 4 at end; promote 3 to repair root



Fix 3 at end; promote 2

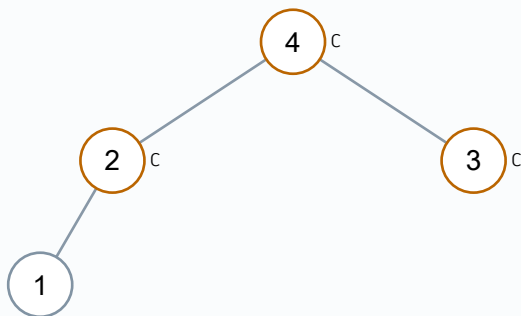


Exchange 2 and 1; done

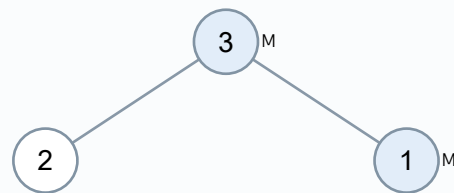


Heap relationships: nodes show the same array, not additional records

Compare root and children
Array is [4,2,3,1]



Heap after removing 4
Heap [3,2,1]; output suffix [4]



How much work and room it needs

Let n count items. Bottom-up construction costs $\Theta(n)$: many parents need very short paths, while only a few need long ones. Each later extraction sinks along at most about $\log_2 n$ levels. About n extractions give typical and worst $\Theta(n \log n)$, also best $\Theta(n \log n)$ for distinct keys. The stated early-stop rule can give $\Theta(n)$ on all-equal input. Iterative control and one exchange temporary use $O(1)$ extra space. The sort is unstable because root-to-end exchanges can change equal-record order.

Why it is fast or slow

The next maximum comes from one path rather than a complete rescan. Widely separated parent and child positions can reduce cache locality.

When it is useful

Useful when worst-case protection and little extra storage matter; also a fallback inside quicksort hybrids.

Details that make the procedure work

Bottom-up construction differs from the bottom-up heapsort variant, which first promotes larger children along a path and then reinserts the saved root to reduce comparisons. More-child heaps are shorter but inspect more children per level.

References [46] [47]

27 Weak heapsort

Type Algorithm Aliases or related names Weak-heap sort; classic implementation with n reverse bits

First understand the small actions

This tree uses different positions from an ordinary binary heap. Root 0 has only child 1. Every position $i \geq 1$ has physical children $2i$ and $2i+1$; physical parent j is $\text{floor}(j/2)$.

Each position owns a reverse bit $r[i]$, either zero or one. At zero, $2i$ is its logical left child and $2i+1$ its right; at one, the names exchange. Slots do not move. Root $r[0]$ remains zero, and root has no left child.

A subtree includes a child and every position reachable below it. A maximum weak heap bounds a node's logical right subtree, not its left subtree. Root's sole right subtree contains everything else, so root remains largest.

The operation `join` compares two prescribed positions and may exchange values and flip a bit. It does not turn any arbitrary pair into a complete weak heap.

Notation $a[i]$ means the item in slot i of the row named a . Brackets select a position; they do not mean multiplication.

The idea in plain words

Weak heaps save some key comparisons by relaxing ordinary heap rules and recording child roles in bits. Construction and extraction must select the right positions for each comparison. One `join` compares just two values; ancestor searches and bit handling do the additional bookkeeping.

Carry it out one action at a time

1. Find the distinguished ancestor of original position j . Climb while the current position is its parent's logical left child; stop at the first right-child edge and return that edge's parent. Numerically, a left edge means $(j \bmod 2) = r[\text{floor}(j/2)]$, where $\bmod 2$ is the remainder after division by two. On a left edge replace j by its parent; otherwise return the parent. Preserve the original j for the later `join`.
2. `join(i,j)` compares $a[i]$ and $a[j]$. If $a[j] > a[i]$, exchange their values and flip $r[j]$ between zero and one. Otherwise do nothing. Bits belong to slots and do not travel with values. Flipping the roles below j is part of correctness.
3. Initialize all bits to zero. For original j from $n-1$ down to 1, find its distinguished ancestor i and perform `join(i,j)`. Already built smaller weak heaps are progressively connected until root holds the maximum. Empty input stops directly.
4. Exchange root with the active last item and decrease active size m . If $m \leq 1$, stop restoration immediately; position 1 is no longer an active candidate.
5. Otherwise descend from position 1 along existing logical left children. At j that child is $2j+r[j]$. Descend only when the child position is below m .
6. From the bottom, perform `join(0,j)`, then move to physical parent $\text{floor}(j/2)$, repeatedly until $j=0$. This upward sequence compares path candidates with root and restores its maximum. Extract again. Do not substitute repeated distinguished-ancestor searches for this physical-parent walk.

Walk through the numbers

1. For [4,1,3,2], bits begin at zero. Position 3 is parent 1's logical right child, so its distinguished ancestor is 1. Join values 1 and 2: [4,2,3,1], and set $r[3]=1$.
2. Position 2 is parent 1's logical left child. Climb to 1, then follow its right-child edge to root 0. Value 3 does not exceed root 4. Joining position 1's value 2 with root also changes nothing.
3. Extract 4: [1,2,3 | 4], size 3. Descend from 1 to left child 2. $\text{join}(0,2)$ exchanges 1 and 3 and flips $r[2]$: [3,2,1 | 4]. Move to physical parent 1; $\text{join}(0,1)$ compares 3 and 2 without exchange.
4. Extract 3: [1,2 | 3,4], size 2. Position 1 has no active left child. $\text{join}(0,1)$ gives [2,1 | 3,4] and flips $r[1]$.
5. The final exchange gives [1,2,3,4], size 1. Stop without visiting position 1 again.

How much work and room it needs

Let n count records. For nonempty input, construction uses exactly $n-1$ key comparisons; ancestor bookkeeping also gives $\Theta(n)$ total construction work. Classic extraction repeatedly follows logarithmic-length paths, giving best, typical, and worst $\Theta(n \log n)$ time, although exchange counts vary. Extra storage is n bits plus $O(1)$ controls; separate byte storage costs n bytes. This is not strict constant-space sorting. It is unstable.

Why it is fast or slow

A join avoids comparing two children before comparing with their parent. Ancestor searches, bit flips, and scattered access can offset the saved key comparisons.

When it is useful

A specialized option for expensive comparisons, weak-heap priority queues, and comparison-count research.

Details that make the procedure work

Sources may mirror logical left and right. Child formulas, ancestor tests, and extraction paths must use one matched convention. Flipping a bit changes child roles rather than reversing the array.

References [\[48\]](#) [\[49\]](#) [\[50\]](#)

28 Tournament sort

Type Algorithm Aliases or related names Winner-tree sort; tournament-tree sorting

First understand the small actions

Think of a knockout tournament in which the smaller value wins. Losing a match does not delete the original record.

Leaves are bottom positions without children. Each record has one leaf; match positions above store the identity of a winning leaf, not another output record.

When one leaf withdraws, only matches on its path to the top depended on it. Other remembered results remain valid.

Use an active flag for withdrawals rather than inventing a numeric value that might also occur in the input.

The idea in plain words

Compare pairs, then their winners, until the top identifies the smallest record. Output it, withdraw its leaf, and replay only its path upward. The top then identifies the next smallest. Continue until every record has been output once.

Carry it out one action at a time

1. For n records, choose P as the smallest power of two at least n , chosen from $1, 2, 4, 8, 16, \dots$. A convenient tree uses root position 1, children $2v$ and $2v+1$, and leaves P through $2P-1$. Assign records to the first n leaves; mark unused leaves inactive. Empty input stops.
2. Define a match. With no active child, mark the parent inactive. With one active child, copy its winning record identity. With two, compare their records and retain the smaller identity. On equal keys, the lower original input position wins, preserving arrival order.
3. Build matches from parent $P-1$ backward to 1. Children therefore have valid results before their parent is processed. Root finally names the champion.
4. Output the complete record named by root, then mark its particular leaf inactive. Clearing only root would leave stale lower matches that could promote the same record again.
5. Starting at that leaf's parent, recompute each match using the current child winners. Move upward by integer-dividing the tree position by two, stopping after root.
6. If root still names an active record, output and withdraw it, then update again. Stop after n outputs or an inactive root. Match identities themselves are never additional records to output.

Walk through the numbers

1. For $[4, 1, 3, 2]$, $P=4$. Leaves 4,5,6,7 refer to original record positions 0,1,2,3. Match position 2 compares 4 and 1, retaining record 1. Match 3 compares 3 and 2, retaining record 3. Root compares those winners and names record 1, value 1.
2. Output $[1]$ and deactivate leaf 5. Match 2 now automatically promotes its remaining 4. Root compares that 4 with the unchanged other winner 2 and names record 3. Other branches need no replay.
3. Output 2 and deactivate leaf 7. Match 3 promotes remaining 3; root compares 4 and 3. Output is $[1, 2]$, and the new champion is 3.
4. Output 3, deactivate leaf 6, and make match 3 inactive. Root automatically promotes remaining 4. Output 4; all leaves are inactive. Result: $[1, 2, 3, 4]$.

How much work and room it needs

Let n count records. The padded leaf count P is below $2n$ for nonempty input. Construction visits $O(n)$ positions. Each of n withdrawals updates about $\log_2 n$ matches, so the basic complete winner tree takes $\Theta(n \log n)$ best, typical, and worst time. Tree identities and usual output storage require $O(n)$ extra space. Consistent original-position tie-breaking makes it stable.

Why it is fast or slow

Remembered comparisons replace a full rescan for each new minimum with one path. Extra tree storage and indirect reads are the trade-off.

When it is useful

Useful for repeated minimum selection and joining ordered streams: replace an output stream head with its next record and replay that leaf's path.

Details that make the procedure work

A related loser tree stores opponents defeated along the path instead. Replacement selection additionally freezes records that cannot continue the current external run; it generates runs and is not another name for this complete sort.

References [\[51\]](#) [\[52\]](#)

29 Introsort

Type Algorithm Aliases or related names Introspective sort; quicksort–heapsort hybrid

First understand the small actions

Quicksort separates a range around a reference value, then works on both sides. One-item splits can repeatedly rescan nearly the whole row.

A depth budget is an integer counter for one chain of subdivisions. It is not a timer or one shared budget spent by every branch.

Insertion holds the next item, shifts larger preceding items right, and fills the opening. A maximum heap puts each parent above smaller children so its first item is largest.

The idea in plain words

Begin with quicksort. If one chain of partitions becomes too deep, switch that unfinished range to heapsort. Use insertion for tiny ranges. Introsort chooses among these familiar actions using explicit size and budget tests.

Carry it out one action at a time

1. Stop immediately for empty or singleton input, without computing a budget. For other inputs of n items, a common starting budget is $2\text{floor}(\log_2 n)$. Compute it by repeatedly integer-dividing n by two until one remains, counting the steps and doubling the count. $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ gives budget eight; $10 \rightarrow 5 \rightarrow 2 \rightarrow 1$ gives six. Choose a fixed tiny-range threshold.
2. At a tiny range, insert its items: starting with the second, hold x , move preceding values larger than x one slot right, then write x into the opening. Repeat to the end. Otherwise, if the budget is zero, use the heap steps below.
3. With budget left, choose a pivot. A common sample is the first, middle, and last values: compare them to find the middle-valued one, not their arithmetic average. One valid teaching partition moves it to the end, scans other items, exchanges values at most the pivot into a growing front region, and finally exchanges the pivot into that region's boundary.
4. Subtract one from the budget. Both unfinished sides receive that reduced value. Processing one branch does not spend the other branch's allowance. Calling the smaller side and looping over the larger side limits stack use.
5. The fallback heap applies only to the current range. Use local positions starting at zero, with children $2i+1$ and $2i+2$. Starting with the last parent and working backward, select its larger child; exchange if the parent is smaller, and continue downward. Stop at a sufficiently large parent or a childless position.
6. The heap root is now largest. Exchange it with the last active item of this range, shrink the active part, and sink the replacement root by the same larger-child rule. Repeat until one item remains, then return to the surrounding task.

Walk through the numbers

1. A practical implementation usually inserts $[4,1,3,2]$ directly. Insert 1 by shifting 4: $[1,4,3,2]$. Insert 3 by shifting 4: $[1,3,4,2]$. Insert 2 by shifting 4 and 3: $[1,2,3,4]$.
2. To expose the fallback separately, use the same input with an artificial budget of one and disable the tiny-range shortcut. Pivot 2 admits 1 into the front by exchanging it with 4: $[1,4,3,2]$. The 3 stays larger. Put pivot 2 into its boundary slot: $[1,2,3,4]$.
3. Left singleton $[1]$ stops. Right range $[3,4]$ inherits budget zero. Its heap construction exchanges parent 3 and child 4, giving $[1,2,4,3]$. Other ranges remain untouched.

4. Exchange that heap's root 4 with its last 3: [1,2,3,4]. The remaining heap is a singleton. These artificial settings illustrate switching, not normal four-item library behavior.

How much work and room it needs

Let n count records. Partition ranges at one depth contain at most n items in total. Only $O(\log n)$ depths are allowed, so partition work is $O(n \log n)$. Fallback ranges do not overlap, and their combined heap work is also $O(n \log n)$. Typical time is $\Theta(n \log n)$, worst $O(n \log n)$. Basic nonadaptive versions also have about $\Theta(n \log n)$ best time. Standard control uses $O(\log n)$ stack space and $O(1)$ exchange storage. It is unstable.

Why it is fast or slow

Ordinary cases keep quicksort's fast scans. Difficult cases cannot continue bad partitions indefinitely; the heap supplies a dependable upper bound.

When it is useful

A general-purpose choice when unusual input must not trigger quadratic work.

Details that make the procedure work

The allowance follows partition depth, not swap count. Heap-sort only the unfinished current range. Versions that finish with one insertion sweep must still keep unfinished small pieces bounded in size.

References [\[53\]](#) [\[54\]](#)

30 Pattern-defeating quicksort (pdqsort)

Type Algorithm Aliases or related names `pdqsort`; `pattern-defeating quicksort`

First understand the small actions

Keep three actions in mind: partition around a value, insert a held item by shifting larger predecessors, and extract a heap maximum to the end. `pdqsort` combines them with observations and switching rules.

An input pattern is an existing arrangement such as ascending, descending, or periodic order. Some pivot policies repeatedly choose poorly on a pattern. Exchanging a few positions disturbs it without changing the records.

A limited insertion attempt has a small movement allowance. If it fails, all held records must be returned before normal sorting continues.

The idea in plain words

While quicksorting, ask whether a split is badly uneven and whether a region already looks easy. Finish easy regions with a short insertion attempt; disturb recurring bad patterns; fall back to heapsort after too many bad splits. Conditional equal-key filtering avoids repeatedly processing duplicates.

Carry it out one action at a time

1. Use insertion on small ranges: hold the next x , shift larger predecessors right, then fill the opening. For larger ranges choose a middle-valued sample item. Large samples may take medians of three groups and then their median; this selects a pivot rather than sorting the row.
2. One ordinary partition temporarily places saved pivot p at left endpoint lo . Set $i=lo+1$ and $j=hi$. While $i \leq j$, advance i over values below p and retreat j over values at least p , checking $i \leq j$ before reading. If they have not crossed, exchange the misplaced pair, advance i , and retreat j . After crossing, exchange the pivot with position j , then continue on either side. The observation records internal pair exchanges, not sample rearrangement or final pivot placement. Author code refines

the bounds and offers a branchless form.

3. Record whether internal misplaced-pair exchanges occurred. Sample rearrangement and final pivot placement are different from that signal. If the split is reasonably balanced and no internal exchanges occurred, try insertion on the sides with a small total movement allowance. Count how many positions held items move. Finish if the attempt completes; otherwise return the held item safely and resume partitioning.
4. A side below roughly one eighth of the range marks a bad partition. Exchange a few specified, separated positions in the larger side to disturb repeated sampling patterns. Reduce a bad-partition allowance initially proportional to $\log_2 n$. On exhaustion, heap-sort this unfinished range. Pattern disturbance alone is not a guarantee; the allowance and fallback supply it.
5. Equal filtering needs prior information. If an earlier partition established that every value here is at least the preceding pivot, and the new pivot equals that value, separate equals from strictly larger items. Leave all equal records in the completed front and continue only on larger ones. Without that lower-bound information, smaller values cannot be ignored.
6. For fallback, use local heap positions: i has children $2i+1$ and $2i+2$. Work backward from the last parent, selecting its larger child and exchanging downward while the parent is smaller. Exchange the resulting maximum root to the active end, shrink, and sink again until one item remains.
7. An optional branchless partition records misplaced left positions in one fixed-size buffer and misplaced right positions in another, then exchanges matched pairs. These buffers hold locations, not additional output records.

Walk through the numbers

1. The author implementation inserts tiny input $[4,1,3,2]$: insert 1 to get $[1,4,3,2]$, then 3 to get $[1,3,4,2]$, then 2 by shifting 4 and 3 to obtain $[1,2,3,4]$. Large-range pattern logic is not used.
2. Separately inspect an easy partition of ordered $[1,2,3,4,5]$. Temporarily move pivot 3 left: $[3,2,1,4,5]$. The scans cross without exchanging internal misplaced pairs. Final pivot placement restores $[1,2 \mid 3 \mid 4,5]$. Limited insertion on both sides confirms order without shifts.
3. For a bad-split example, a sixteen-item range whose pivot is its smallest value leaves zero items left and fifteen right. Spend one allowance and disturb specified positions on the large side. Continued failures eventually send the unfinished range to heapsort. Exact disturbance positions belong to the author's implementation.

How much work and room it needs

Let n count records. Recognized easy patterns can take $O(n)$ scans and short checks. Typical time is $O(n \log n)$. Repeated severely unbalanced splits spend the allowance; heapsort then bounds remaining work, giving worst $O(n \log n)$. Author implementations use $O(\log n)$ stack space and optional $O(1)$ fixed buffer space. The method is unstable. Thresholds and disturbance choices affect constants.

Why it is fast or slow

Easy regions stop early, while difficult patterns cannot consume unlimited partition work. Sampling and trial checks add overhead, and branchless handling mainly helps cheap comparisons.

When it is useful

A fast general-purpose array sort when stability is unnecessary; also a useful example of explicit control around familiar sorting actions.

Details that make the procedure work

A failed trial must preserve every record. Production code also needs precise sample layouts, bounds, and sentinel handling; reuse the tested author implementation rather than treating this explanation as its complete source code.

References [\[55\]](#) [\[56\]](#) [\[57\]](#)

31 Splaysort

Type Algorithm Aliases or related names Splay sort; self-adjusting tree sorting

First understand the small actions

A search tree follows smaller keys left and larger keys right until an empty link receives a node. It remembers order relationships rather than storing an already sorted array.

A right rotation raises left child x over parent p . Make p the right child of x and transfer x 's old right branch to p 's left. Its keys lie between x and p . A left rotation mirrors this.

The node above x is its parent p ; the next above is grandparent g . Their left/right relationships determine the rotation order.

Amortized cost bounds a whole sequence, allowing an expensive individual operation. It is not a per-operation promise.

The idea in plain words

After inserting or accessing a node, rotate it up to root: this is splaying. Recently used keys and their neighbors may become easier to reach. After all insertions, read the tree left, node, right. The tree changes shape without maintaining a fixed height bound.

Carry it out one action at a time

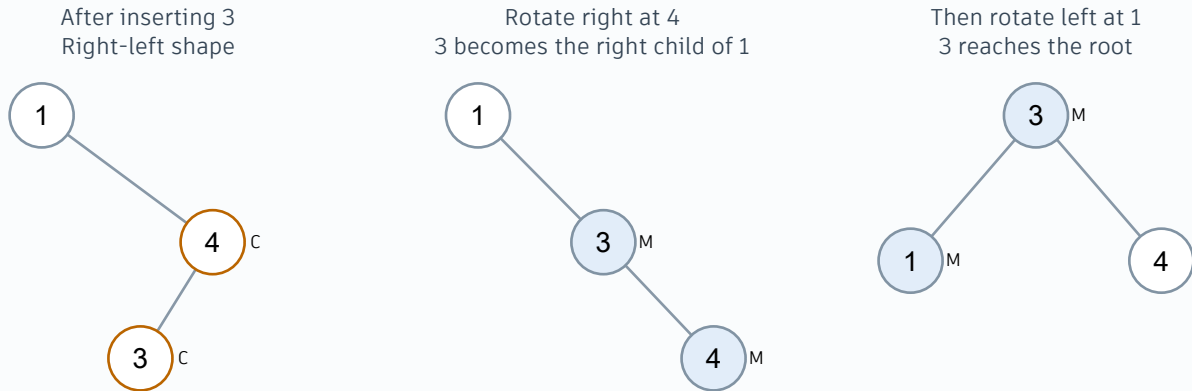
1. Insert by comparisons from root: smaller left, larger right, then create a node at an empty link. For equality, append to an existing node's arrival-order queue and use that node as x . Keep parent connections for the upward changes.
2. If x is root, stop. If p is root, perform one rotation: right at p for a left child, left for a right child. This zig puts x at root.
3. If x and p are both left children, rotate right at g to raise p , then right at p to raise x . For two right-child edges, rotate left at g , then left at p . This zig-zig raises the parent first; its order is deliberate.
4. If x is right of p while p is left of g , rotate left at p , then right at g . Reverse both directions for the opposite turn. This zig-zag raises x over its parent, then grandparent.
5. Reconnect the rotated part to any higher ancestor, inspect x 's new relatives, and repeat until x is root. Preserve every transferred middle branch. Rotations reconnect records; they do not remove them.
6. For final output, return at empty links. Completely read the left branch, output the node's queued records, then read the right branch. Calls or an explicit stack remember unfinished returns.

Walk through the numbers

1. Insert $[4,1,3,2]$. Root 4 receives left child 1. A right zig at 4 makes root 1 with right child 4.
2. Insert 3 by going right from 1 and left from 4. This right-left turn needs right rotation at 4, making 3 the right child of 1 with 4 to its right, then left rotation at 1. Root is 3 with children 1 and 4.
3. Insert 2 by going left from 3 and right from 1. Rotate left at 1, then right at 3. Root becomes 2 with left child 1 and right child 3; node 3 keeps right child 4. No records disappear.

4. Read left child 1, root 2, then right-branch nodes 3 and 4: [1,2,3,4].
5. To inspect zig-zig separately, suppose a local left chain is $4 \rightarrow 3 \rightarrow 2$. Right-rotate 4 to make 3 above children 2 and 4; right-rotate 3 to make 2 above the right chain $3 \rightarrow 4$. This is a separate shape demonstration.

Two splay rotations for the example insertion of 3



How much work and room it needs

Let n count records. One insertion and splay can follow $O(n)$ nodes. Across the full sequence, the splay rules give typical and whole-sort worst $O(n \log n)$; multiplying the worst individual cost by n loses that sequence guarantee. Monotone arrivals can cost $\Theta(n)$, inserting beside root and rotating once each time. Nodes need $O(n)$ space. Final trees can still be tall, so recursive reading may need $O(n)$ stack. Arrival-order equal queues or original-position ties can make output stable.

Why it is fast or slow

Keeping recent keys near root can shorten later nearby searches. Rotations, pointer reads, and allocation add costs beyond comparisons.

When it is useful

Useful for adaptive-sorting study and data already using a self-adjusting search tree.

Details that make the procedure work

Changing the zig-zig order creates a different adjustment rule without automatically retaining the splay guarantee. Preserve the middle subtree at every rotation. A tall tree remains possible despite good amortized performance.

References [58] [59] [60]

32 Sorting with a min-max heap

Type Important variant Aliases or related names min-max-heap extraction sort

First understand the small actions

Fill a binary tree by levels using array positions: root 0, children 1,2, then 3,4,5,6. Children of i are $2i+1$ and $2i+2$ when those positions exist. Its parent is $\text{floor}((i-1)/2)$, with floor discarding a fractional part.

An ordinary min-heap exposes one end. A min-max heap alternates roles by level: root minimum, next level maximum, next minimum. A minimum-level node is no larger than its descendants; a maximum-level node is no smaller. The root is smallest, and the larger root child is largest.

Grandchildren share a node's minimum or maximum role, while direct children have the opposite role. Repairs therefore inspect both generations and sometimes correct the intervening parent relation.

The idea in plain words

Keep both extrema available in one tree. Ascending output repeatedly removes the root and repairs the remaining tree. The extra rules are useful when an application already needs both ends, rather than automatically improving ordinary sorting.

Carry it out one action at a time

1. Represent the complete tree in an array. Depth zero is minimum; even depths are minimum and odd depths maximum. Find roles with level boundaries: position 0 is level 0, positions 1–2 level 1, 3–6 level 2, and 7–14 level 3. The next level starts at twice the current start plus one. While scanning positions backward, reduce the level only when crossing its start; do not repeatedly climb every node's parent chain. The binary digit count of $i+1$ minus one also gives its level, if the machine's bit-operation cost is accounted for.
2. Repair bottom-up from the last parent, $\text{floor}(n/2)-1$, through position zero. Children's subtrees are already repaired before a parent is processed.
3. At a minimum level, inspect existing children and grandchildren and choose their smallest. Stop if none is smaller than the current record. If the candidate is a direct child, exchange once and stop.
4. For a grandchild candidate, exchange, then check the lowered old record against its maximum-level parent. If it is too large, exchange with that parent. Continue minimum-level repair at the grandchild position. Reverse all inequalities at maximum levels.
5. After construction, emit the root, move the final array record there, shorten the tree, and repair the minimum root. Repeat until empty. Empty and single-item trees must not inspect nonexistent children.
6. Descending extraction instead uses the larger root child and maximum-level repair. The ascending version here writes a separate output array.

Walk through the numbers

1. For [4,1,6,2,5,3], root 4 is a minimum level and 1,6 are maximum levels. Node 6 already exceeds child 3. Node 1 must exceed children 2,5, so exchange with 5: [4,5,6,2,1,3].
2. Root candidates 5,6,2,1,3 have minimum 1. Exchange it with 4: [1,5,6,2,4,3]. Lowered 4 does not exceed parent 5, so no correction is needed.
3. Emit 1 and replace it with final 3, leaving [3,5,6,2,4]. Exchange with grandchild 2 to obtain [2,5,6,3,4]. Lowered 3 remains below parent 5.
4. Emit 2, replace with 4, and repair [4,5,6,3] to [3,5,6,4]. Emit 3; replacement 4 already lies below children 5,6. Emit 4; replacement 6 swaps with direct child 5 to form [5,6]. Emit 5 then 6. Output: [1,2,3,4,5,6].

How much work and room it needs

Bottom-up construction is $O(n)$. Each minimum removal repairs at most roughly $\log n$ levels, giving $O(n \log n)$ sorting work. General distinct-key cases have that scale; all-equal input may require fewer repairs. The heap reuses the input, but this extraction procedure needs $O(n)$ output space, with $O(1)$ iterative management. It is normally unstable. Children, grandchildren, and parent corrections add work compared with an ordinary heap; the benefit is exposing both extrema.

Why it is fast or slow

Access to both extremes is useful, but full sorting alone rarely needs it. Child-and-grandchild repair is more complex than ordinary heap repair.

When it is useful

Applications that already use a double-ended priority queue and later need ordered extraction.

Details that make the procedure work

Checking children alone can miss a smaller or larger grandchild. Direct-child and grandchild exchanges have different continuation rules, so do not combine them into one vague repair instruction.

References [\[61\]](#)

33 Block merge sorting: Grailsort / WikiSort

Type Algorithm family Aliases or related names Block sort; Grailsort and WikiSort are related implementations, not aliases

First understand the small actions

A block is a consecutive group of slots. Exchange equal-size blocks item by item, not just their fronts.

Workspace temporarily holds data. Here it is borrowed from the input, so its original records must return.

Distinct tags temporarily replace A-block fronts to show original block order. The real fronts are parked in the tag area. Tags and real front values answer different questions.

Rotation exchanges neighboring pieces while preserving internal order. Reverse each piece and then the whole: $[7,8 \mid 5,6]$ becomes $[8,7 \mid 5,6]$, then $[8,7 \mid 6,5]$, then $[5,6 \mid 7,8]$.

$|A|$ counts items in A. The symbol $\sqrt{}$ means square root: $\sqrt{16}=4$ because $4 \times 4 = 16$.

The idea in plain words

Borrow a few real records as tools, move the remainder in blocks, join neighboring ordered pieces, then return every borrowed record. The representative below uses WikiSort's two internal buffers. Only left-run A blocks receive tags; right-run B blocks retain actual first values. Related Grailsort has different scheduling and boundary rules.

Carry it out one action at a time

1. First form neighboring ordered runs A and B. Tiny runs can use insertion: hold the next item, shift larger preceding items right, then fill the opening. Increase run sizes in successive joining rounds. For a large join, choose block length near $\sqrt{|A|}$.
2. Find distinct keys for two reserved groups: workspace large enough for one A block, and one tag per remaining A block. Identify selected keys, then collect them by rotations working backward from later selected groups. Unselected records retain their relative order. Reserved areas do not roll.
3. Exchange ascending tags with A-block first items in original order. Real first values remain in corresponding tag slots. The minimum remaining tag identifies the next original A block; read its

corresponding real first value to decide insertion. B blocks receive no tags.

4. Exchange the leftmost rolling A block with the next equal-size B block. B moves left and A moves right; A blocks may become scrambled. Keep rolling while the next real A first value exceeds the last value of the B block crossed. Otherwise find the minimum-tag A block and exchange it into the leftmost rolling position.
5. Binary-search the preceding B block for its first value at least the real A first value. A smaller middle value eliminates the left half; otherwise retain the left half. Rotate that B suffix with A to insert A there. A goes before equal B values. Exchange its tag back with its real first value.
6. After the next A block lands, finish the previous A and intervening B items. Exchange that A block into workspace. Compare workspace and B heads; exchange the smaller head with the next output slot, then advance its source and the output. Choose A on ties. Displaced borrowed records remain safe. When one source ends, exchange the other remainder into place.
7. Finish the last A with remaining B. Insertion-sort scrambled workspace keys, then redistribute both reserved groups using searches and rotations with source-aware tie rules. Scarce keys require one tag buffer, larger blocks, and search-and-rotation local joins. Uneven edge pieces also need special rotations.

Walk through the numbers

1. The whole input [1,2,3,4,6,9,11,14 | 5,7,8,10,12,13,15,16] is unordered although each half is ordered. Reserve [1,2] as workspace and [3,4] as tags. A=[6,9][11,14]; B=[5,7][8,10][12,13][15,16]. All sixteen items survive.
2. Tag A as [3,9][4,14], parking real fronts [6,11]. Roll past [5,7]. Since $6 \leq 7$, insert and restore the first A before 7: [5][6,9][7].
3. The next front 11 passes [8,10] and belongs before 12 in [12,13]. After restoration the body is [5][6,9][7,8,10][11,14][12,13,15,16]. Tags return to [3,4].
4. Exchange [6,9] into workspace [1,2]. Workspace becomes [6,9], local output [1,2,7,8,10]. Exchanging out 6 gives [6,2,7,8,10], workspace [1,9]. Choose B heads 7 and 8 next: [6,7,2,8,10], then [6,7,8,2,10].
5. Now $9 < 10$. Exchange workspace 9 with output 2, obtaining [6,7,8,9,10] and restoring workspace [1,2]. Join the last pair into [11,12,13,14,15,16]. Return [1,2] and [3,4] to obtain 1 through 16. Real code directly handles tiny arrays; small blocks expose its tools.

How much work and room it needs

Let n count items. Established implementations keep each joining level to $O(n)$, giving typical and worst $O(n \log n)$. Searching $\sqrt{n}$ tags $\sqrt{n}$ times costs $O(n)$; insertion-sorting $\sqrt{n}$ workspace keys also costs $O(n)$. In-place configurations externally allocate $O(1)$ machine words; borrowed $O(\sqrt{n})$ records remain inside the input. Optional external buffers change this. Ordered-input skipping can approach $O(n)$. Established implementations are stable when extraction, ties, and restoration follow their rules.

Why it is fast or slow

Whole blocks avoid repeated long individual shifts. Extraction, tagging, restoration, and edges add considerable overhead.

When it is useful

Useful when stability must avoid a full second record array. Act out these operations, then use tested production code.

Details that make the procedure work

Tags describe original block order, not value-based insertion points. Restore every borrowed record. Scarce-key and uneven-block handling are necessary parts of the complete algorithm.

References [\[62\]](#) [\[63\]](#) [\[64\]](#)

34 Quicksort with Lomuto partitioning

Type Algorithm Aliases or related names Quicksort; Lomuto single-scan partitioning

First understand the small actions

Positions here start at zero. lo and hi are the first and last positions of the current range, both included.

A pivot is a temporarily chosen comparison value. This version chooses the last item without removing it.

Partitioning separates values into groups; it does not order every item within those groups.

Notation $a[i]$ means the item in slot i of the row named a . Brackets select a position; they do not mean multiplication.

The idea in plain words

Scan from left to right and collect items no larger than the last-item pivot near the front. Then put the pivot between the two groups. That pivot item is finished. Repeat the operation on the two remaining groups.

Carry it out one action at a time

1. Stop for fewer than two items. Save the last value as p and set $b=lo$. Position b is the first slot after the collected small-item group, which initially contains nothing.
2. Scan j from lo through $hi-1$, leaving the pivot slot untouched. If $a[j] \leq p$, exchange $a[j]$ with $a[b]$, then increase b . Otherwise leave it and continue scanning.
3. Why is the exchange safe? Already scanned items from b to just before j are larger than p . Moving one of them to j leaves the small-item group intact. When $b=j$, exchanging a slot with itself changes nothing.
4. After the scan, exchange $a[b]$ with the pivot at hi . Everything before b is at most p ; everything after b is larger. The pivot at b has its final position.
5. Apply these instructions separately to $[lo, b-1]$ and $[b+1, hi]$. Exclude b . Empty ranges and singletons stop. Calls can remember unfinished ranges, or an explicit list can do that job.

Walk through the numbers

1. Use $[4,1,3,2]$, with $p=2$ and $b=0$. Scan 4: it is larger, so leave it. Scan 1: exchange it with slot b , producing $[1,4,3,2]$, then set $b=1$.
2. Scan 3: it is larger than 2, so leave it. Exchange the final pivot 2 with slot 1 containing 4. The row becomes $[1,2,3,4]$. Pivot 2 is fixed; unfinished ranges are $[1]$ and $[3,4]$.
3. The singleton stops. In $[3,4]$, pivot 4 admits 3 into the small group through a self-exchange, and the final pivot exchange also changes nothing. All items are ordered. This lucky trace becomes globally ordered early; other partitions usually leave more work within both groups.

State trace C compare M move S ordered

Pivot 2 at end; skip $4 > 2$



$1 \leq 2$: exchange 1 and front 4



$3 > 2$; place pivot at boundary



Finish the left and right regions



How much work and room it needs

Let n count records. A partition scans its entire range. Roughly equal splits give about $\log_2 n$ levels and best $\Theta(n \log n)$ time. Random input or randomized pivots give expected $\Theta(n \log n)$. Repeated one-item splits scan $n + (n-1) + \dots$ items, giving worst $\Theta(n^2)$. All-equal input also degenerates with this $\leq$ rule. Stack space is usually $O(\log n)$, worst $O(n)$, or always $O(\log n)$ when only the smaller side recurses. The algorithm is unstable.

Why it is fast or slow

Simple scans and exchanges are cheap, but repeatedly choosing an extreme pivot rescans almost the whole input. Equal values need additional care.

When it is useful

A clear introduction to quicksort partitioning. General-purpose versions usually add pivot sampling, equal-key grouping, or a fallback.

Details that make the procedure work

Only the pivot item is final, not every item before it. Use this scheme's recursion boundaries. Changing $\leq$ to $<$ changes equal-key placement without automatically removing its bad equal-key case.

References [65] [66]

35 Quicksort with Hoare partitioning

Type Important variant Aliases or related names Original Hoare partition scheme; two-pointer quicksort variant

First understand the small actions

Use zero-based positions and inclusive endpoints lo and hi . The lower middle position is $\text{floor}((lo+hi)/2)$; floor discards the fractional part.

A scanning pointer is a remembered position. Moving it means explicitly adding or subtracting one.

Save the pivot value separately. Its original record may move, but the saved comparison value does not.

The idea in plain words

Look inward for items on the wrong side: the left scan stops on a value at least the pivot, and the right scan stops on one at most the pivot. Exchange the pair. When the scans meet or cross, split at their boundary. This boundary does not certify a final pivot position.

Carry it out one action at a time

1. Stop at fewer than two items. Save the lower-middle value as p . Set $i=lo-1$ and $j=hi+1$, just outside the range. Do not substitute the upper middle: a two-item ordered range can then fail to shrink.
2. Advance i once, then keep advancing while its value is smaller than p . Retreat j once, then keep retreating while its value is larger than p . Stop on equal values as well as misplaced ones.
3. If $i \geq j$, return j without exchanging. Values in $[lo, j]$ are at most p , and values in $[j+1, hi]$ are at least p . Equal values may appear on both sides.
4. Otherwise exchange the stopped values and repeat the scans. Each scan starts with its compulsory movement, so equal values cannot trap it at the same positions forever.
5. Sort $[lo, j]$ and $[j+1, hi]$ using these instructions. Include j in the left range. The lower-middle choice and scan rules make both subranges smaller. Stop at empty ranges and singletons.

Walk through the numbers

1. For $[4, 1, 3, 2]$, bounds are $0..3$. The lower-middle position 1 gives $p=1$. The left scan stops at 4. The right scan passes 2 and 3, then stops at 1.
2. Exchange them to get $[1, 4, 3, 2]$. Next, i moves to position 1 and j to position 0. They cross. Return boundary 0 and handle $[1]$ and $[4, 3, 2]$.
3. For the right range, lower-middle value 3 is the pivot. Exchange stopped values 4 and 2: $[1, 2, 3, 4]$. The next scans meet at position 2 containing 3, so return boundary 2. The unfinished groups are $[2, 3]$ and $[4]$.
4. The lower-middle pivot 2 splits $[2, 3]$ into singletons. The result is $[1, 2, 3, 4]$. A boundary happened to be final here; do not depend on that coincidence.

State trace C compare M move S ordered

Save middle value $p=1$; stop at 4 and 1



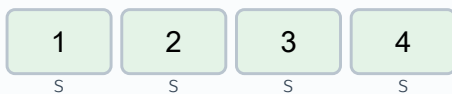
Exchange; next return boundary 0



Right region $p=3$: exchange 4 and 2



Finish recursive boundary splits



How much work and room it needs

Let n be the record count. A partition scans across its range once from two directions. Balanced distinct-key splits give best $\Theta(n \log n)$; suitable randomization gives expected $\Theta(n \log n)$. Bad splits allow worst $\Theta(n^2)$. All-equal input splits near the middle but still requires $\Theta(n \log n)$, rather than finishing in one pass. Stack space is typically $O(\log n)$, worst $O(n)$, or $O(\log n)$ with smaller-side recursion. The method is unstable.

Why it is fast or slow

An exchange corrects one item on each side. Fewer exchanges can help, but balanced subproblems remain essential.

When it is useful

A common array-quicksort mechanism when scan limits and recursive bounds are carefully implemented.

Details that make the procedure work

Some two-way schemes finally move the pivot and are called Hoare-style. Their bounds differ. This lesson uses the original boundary-returning scheme: never exclude j as if it were a final pivot.

References [\[67\]](#) [\[68\]](#)

36 Three-way quicksort

Type Important variant Aliases or related names 3-way quicksort; fat-pivot quicksort; Dutch-national-flag partitioning

First understand the small actions

Three groups mean smaller than, equal to, and larger than a chosen value. Equal records are retained individually.

Three positions divide the row into known-small, known-equal, unchecked, and known-large regions. Only the unchecked region needs decisions.

After an exchange, an incoming item may be unchecked. Advancing immediately can skip it.

Notation $a[i]$ means the item in slot i of the row named a . Brackets select a position; they do not mean multiplication.

The idea in plain words

Collect all pivot-equal items in the middle during one scan. They need no further sorting by this key. Continue only on smaller and larger groups, avoiding repeated work on duplicates.

Carry it out one action at a time

1. Save pivot value p . With zero-based inclusive bounds, set $lt=lo$, $i=lo$, $gt=hi$. The regions are $lo..lt-1$ small, $lt..i-1$ equal, $i..gt$ unchecked, and $gt+1..hi$ large. Initially only the unchecked region exists.
2. While $i \leq gt$, inspect $a[i]$. If it is smaller than p , exchange it with $a[lt]$, then advance both lt and i . The small region grows. Any equal item displaced from its boundary was already checked.
3. If it equals p , advance i only. This enlarges the equal region without removing the record.
4. If it is larger, exchange it with $a[gt]$ and decrease gt . Keep i unchanged: the incoming right-side item still needs checking.
5. When $i > gt$, nothing remains unchecked. Sort only the small and large ranges with these same instructions. The equal range $lt..gt$ is finished. Empty and singleton ranges stop.

Walk through the numbers

1. For $[3,1,3,2,3]$, take $p=3$. The first 3 advances i to 1. The 1 is small, so exchange positions 1 and 0: $[1,3,3,2,3]$, with $lt=1$ and $i=2$.
2. The next 3 advances i to 3. Exchange 2 with the equal-region boundary at $lt=1$: $[1,2,3,3,3]$, with $lt=2$ and $i=4$. The last 3 advances i beyond gt . All three 3s remain.
3. Sort $[1,2]$ separately using pivot 1. The 1 is equal; the 2 moves into the large region through a self-exchange. Its singleton needs no work. Result: $[1,2,3,3,3]$.
4. To see why a large-item exchange does not advance i , take $[2,3,1]$ with $p=2$. Exchange middle 3 with last 1: $[2,1,3]$. Check the new middle 1 again and move it left, obtaining $[1,2,3]$. Skipping that check would miss the 1.

State trace C compare M move S ordered

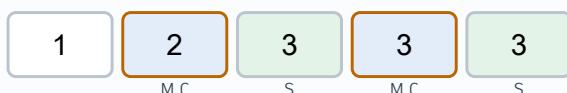
Pivot 3; inspect 1



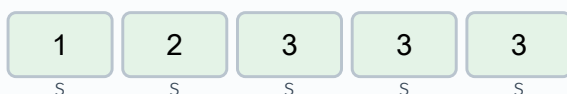
1 < 3; exchange to left



2 < 3; equal middle is finished



Sort left [1,2]; done

**How much work and room it needs**

Let n count items. Each step shrinks the unchecked region, so one partition costs $\Theta(n)$. All-equal input finishes in that pass: best $\Theta(n)$. Random distinct keys or randomized pivots give expected $\Theta(n \log n)$; extreme splits allow worst $\Theta(n^2)$. Stack is typically $O(\log n)$, worst $O(n)$, or $O(\log n)$ with smaller-side recursion. The method is unstable because distant exchanges can reorder equal records.

Why it is fast or slow

Duplicates leave further work immediately. With mostly different keys, the method retains quicksort's dependence on pivot quality.

When it is useful

Useful for categories, scores, and other repeated keys; also the foundation of character-based string partitioning.

Details that make the procedure work

Do not advance i in the larger-than branch. Keep all equal records rather than replacing the group by one representative. An equal-value group need not preserve the original order of its record identities.

References [69] [70]

37 Dual-pivot quicksort

Type Important variant Aliases or related names Two-pivot quicksort; Yaroslavskiy-style dual-pivot partitioning

First understand the small actions

Two pivots are saved values $p \leq q$. They define values below p , between p and q inclusive, and above q .

A classified region is not necessarily internally ordered. It only satisfies its value limits.

After exchanging with the unchecked right end, inspect the incoming item before moving on.

Notation $a[i]$ means the item in slot i of the row named a . Brackets select a position; they do not mean multiplication.

The idea in plain words

Place comparison values at both ends, classify the interior, then move the pivots to the group boundaries. Continue on the three remaining regions. This lesson uses a simple executable teaching partition; production Yaroslavskiy implementations refine its scans and pivot selection.

Carry it out one action at a time

1. Stop for fewer than two items. Compare the endpoints and exchange them if needed. Save their values as p and q . Set $lt=lo+1$, $gt=hi-1$, and $i=lt$, leaving both endpoint pivots outside the scan.
2. While $i \leq gt$, a value below p exchanges with $a[lt]$; advance lt and i . A value between p and q remains in the middle; advance i only.
3. A value above q exchanges with $a[gt]$; decrease gt but retain i . Inspect the incoming item again. It may belong to any of the three groups.
4. When no unchecked items remain, decrease lt and increase gt . Exchange the left pivot into lt and the right pivot into gt . Their two positions are now finished.
5. Sort $lo..lt-1$, $lt+1..gt-1$, and $gt+1..hi$ separately. If $p=q$, the entire middle equals that value and can be skipped. If p differs from q , the middle still needs sorting. Empty ranges and singletons stop.

Walk through the numbers

1. Use $[4,1,5,2,3]$. Exchange endpoints to get $[3,1,5,2,4]$, so $p=3$ and $q=4$. Start $lt=i=1$ and $gt=3$. The 1 is small; a self-exchange advances lt and i to 2.
2. The 5 exceeds q . Exchange positions 2 and 3: $[3,1,2,5,4]$, with $gt=2$. Keep $i=2$ and inspect incoming 2. It is below p , so its self-exchange advances lt and i to 3.
3. The scan ends. Decrease lt to 2 and exchange pivot 3 with value 2: $[2,1,3,5,4]$. Increase gt to 3 and exchange pivot 4 with value 5: $[2,1,3,4,5]$.
4. The pivots are fixed. Sort left range $[2,1]$ by exchanging its endpoints. The middle is empty and the right $[5]$ stops. Result: $[1,2,3,4,5]$.

How much work and room it needs

Let n count records. Each partition scans its range, but an item may need both pivot comparisons and exchanges. Suitable sampling or random distinct input commonly gives expected $O(n \log n)$. Worst time is $O(n^2)$; endpoint pivots alone do not prevent bad splits. With the stated equal-pivot shortcut, all-equal input gives best $\Theta(n)$. Stack is typically $O(\log n)$, worst $O(n)$ without control. The method is unstable. Different dual-pivot implementations have different exact comparison counts.

Why it is fast or slow

Three subproblems can reduce later levels, but classification costs more within each level. Sampling, equal values, cache behavior, and hardware decide whether that trade pays.

When it is useful

Useful for understanding multi-pivot partitioning and some standard-library sorting choices.

Details that make the procedure work

The middle is range-limited, not ordered. Continue sorting it unless $p=q$. These teaching steps do not claim to reproduce a particular runtime library line for line.

References [\[71\]](#) [\[72\]](#)

38 Samplesort

Type Algorithm family Aliases or related names Sample sort; multiway sampled partitioning family

First understand the small actions

A bucket collects records in a value range; collection alone does not order them. Ranges are arranged so every earlier-bucket key precedes every later-bucket key.

A sample copies a few input values to choose boundaries. Original records still participate once in normal classification; sample copies are not extra output.

Comparing with a middle boundary can eliminate half the candidate buckets.

The idea in plain words

Use a small ordered sample to choose several boundaries, distribute all records into their value ranges, sort each bucket, then read buckets in range order. Bucket membership is determined by comparisons, rather than using integer keys directly as array positions.

Carry it out one action at a time

1. Choose b buckets and a sample larger than b . Sort the sample, for example by holding each next value and inserting it after smaller predecessors. Select $b-1$ roughly evenly spaced sample values as splitters. Oversampling improves balance likelihood, not certainty.
2. Define where boundary-equal values belong, such as the bucket on the left. For each input record, compare with middle splitters to narrow the possible ranges until exactly one bucket remains.
3. A simple buffered version appends records to bucket lists in input order. An in-place variant counts bucket sizes, establishes contiguous ranges, and exchanges records into them; that requires additional boundary tracking.
4. Sort each bucket. For a tiny one, hold its next item, shift larger predecessors right, and fill the opening. Larger buckets may repeat sampling. Detect all-equal buckets and partitions that do not shrink. Finish equals immediately or switch to a guaranteed method, such as repeatedly splitting pieces and joining ordered fronts.
5. Output the ordered buckets from lowest range upward. No cross-bucket merge is needed because the boundaries already order their ranges.

Walk through the numbers

1. Use $[7,2,5,1,6,3]$ with teaching splitters 2 and 6. The ranges are $x \leq 2$, $2 < x \leq 6$, and $x > 6$. A sample $[7,2,5,1,6]$ sorts to $[1,2,5,6,7]$, from which 2 and 6 can be chosen. Original 2 and 6 still occur in the input.
2. Classify in order: 7 goes third; 2 first; 5 second; 1 first; 6 second; 3 second. The buckets are $[2,1]$, $[5,6,3]$, and $[7]$.
3. Insert 1 before 2 in the first bucket. In the second, 5 and 6 already precede one another; hold 3, shift 6 and 5, then write 3. The ordered buckets are $[1,2]$, $[3,5,6]$, $[7]$.
4. Read them consecutively: $[1,2,3,5,6,7]$. Sample values are not appended again.

How much work and room it needs

Let n count records and b count buckets. Balanced splitter search takes about $\log_2 b$ comparisons per item, plus movement. Good randomized samples and shrinking recursion give expected $O(n \log n)$. This family has no single best-case formula across all bucket policies and leaf sorters. Naive versions can take $O(n^2)$ or fail to shrink; guarded variants can guarantee $O(n \log n)$. Buffered lists commonly use $O(n+b)$ extra space. Stability requires order-preserving distribution and stable bucket sorting.

Why it is fast or slow

Many independent buckets reduce later problem sizes, but sampling and storage cost work. Skew can leave most items in one bucket.

When it is useful

Common in parallel and distributed sorting, and useful for separating distribution from actual internal ordering.

Details that make the procedure work

Give equal values one unambiguous bucket. All-equal detection must retain every record. Parallel elapsed time additionally needs worker and communication assumptions.

References [\[73\]](#) [\[74\]](#)

39 Tree sort and balanced-tree variants

Type Algorithm family Aliases or related names Binary-search-tree sort; BST sort; balanced-tree variants

First understand the small actions

A tree node is a small container holding a record and two links, left and right. An absent link is empty. The first node is the root.

The search-tree rule places smaller keys throughout the left branch and larger keys throughout the right branch, not just in immediate children.

Ordered reading visits the left branch, then the current node, then the right branch. Apply that rule at every node.

Rotation reconnects a few links. Raising a right child puts the old parent on its left; the raised child's old left branch becomes the old parent's right branch.

The idea in plain words

Insert records by following smaller-left and larger-right decisions until an empty link is found. The tree remembers key relationships. Reading left, node, right produces ordered output. A tall chain makes insertion slow; balanced variants change local connections to keep searches short.

Carry it out one action at a time

1. Create an empty tree. The first record becomes root. For each later x , compare at root, follow left if smaller and right if larger, and repeat. At an empty link create its node. Only that path is searched.
2. Never replace an existing equal record. One stable policy keeps an arrival-order queue at each equal-key node, outputting every queued record. Another compares key followed by original position.
3. For output, an empty link returns immediately. Otherwise completely read the left branch, output the node's records, and completely read the right branch. Suspended calls remember where to return after reading the left side; an explicit node stack can do the same.
4. An ordinary tree makes no shape adjustments. An AVL variant stores height: empty links have height zero, and a node has one plus its larger child height. Update heights toward root after insertion. If left and right differ by more than one, inspect the direction of the taller child's taller branch.
5. Matching directions need one rotation; a turn needs two, first at the child, then at its parent. For a left rotation at p , raise right child x , transfer x 's old left branch to p 's right link, make p the left child of x , and reconnect the old upper link to x . Right rotation mirrors this. Recompute changed heights.

6. Once insertion finishes, read the tree in order. Rotations preserve key ranges, so reading rules remain unchanged. Red-black trees, B-trees, and treaps support the same sorting framework through different balance rules.

Walk through the numbers

1. Insert [4,1,3,2]. Make 4 root. Put 1 left of 4. For 3, go left from 4, then right from 1. For 2, go left from 4, right from 1, then left from 3. All records exist, but creation order is not sorted output.
2. Read root 4's left branch first. Node 1 has no left branch, so output 1. Its right node 3 first reads left child 2, producing 2, then outputs 3. Return to output 4: [1,2,3,4].
3. Separately, inserting [1,2,3] into a plain tree forms a right chain. AVL detects root child heights zero and two. Since the taller child also leans right, rotate left at 1. Root becomes 2, with 1 left and 3 right.
4. Reading still yields [1,2,3]. An old left branch of 2, if present, contains values between 1 and 2 and correctly transfers to 1's right link.

How much work and room it needs

Let n count records and h be tree height. Insertion follows a path of up to h nodes. Ordinary distinct-key BST construction is best $\Theta(n \log n)$, expected $\Theta(n \log n)$ for random insertion, and worst $\Theta(n^2)$. AVL and red-black variants guarantee $O(n \log n)$ construction plus $\Theta(n)$ output. Grouping equals can make all-equal input $\Theta(n)$. Nodes and queues use $O(n)$ extra storage; reading uses $O(h)$ stack, worst $O(n)$ for a plain tree and $O(\log n)$ for a balanced one. Suitable equal-key queues or original-position ties make it stable.

Why it is fast or slow

Short branches discard many possibilities per comparison. A chain searches progressively farther. Balancing adds link updates but limits path length.

When it is useful

Useful when ordered searches and updates are already needed. For one array sort, node allocation and indirect access can be expensive.

Details that make the procedure work

Red-black trees use color rules, recoloring, and rotations; B-trees split full multi-key nodes and promote a middle key; treaps rotate according to random priorities and have probabilistic balance. Plain BSTs do not inherit these guarantees.

References [75] [76]

40 Shellsort and gap-sequence variants

Type Algorithm Aliases or related names Diminishing-increment sort; Shell sort

First understand the small actions

Insertion holds an item, shifts larger predecessors, and fills the opening. Shellsort first replaces a one-slot step by h slots.

For $h=4$, positions 0,4,8,... form one small row and 1,5,9,... another. These interleaved rows need no copies.

Ordering every small row makes the array h -sorted. Adjacent values can still be out of order.

At $h=1$ there is just the full row, so insertion guarantees complete order.

Notation $a[i]$ means the item in slot i of the row named a . Brackets select a position; they do not mean multiplication.

The idea in plain words

Let misplaced items travel long distances early, shorten the step, and finish with ordinary insertion. Early passes can save later one-slot shifts. The gap sequence determines the guarantee; Shellsort has no single universal time formula.

Carry it out one action at a time

1. Choose decreasing positive gaps ending in one. Knuth starts at 1 and repeatedly uses $3h+1$: 1,4,13,40,... . Use suitable gaps backward. A gap at least n has no multi-item row to process.
2. For gap h , scan i from h through $n-1$. Hold $x=a[i]$ and set opening $j=i$. While $j \geq h$ and $a[j-h] > x$, move that predecessor into j and subtract h from j .
3. Write x into the final opening. This inserts it into its interleaved row without disturbing other rows. Completing the scan orders every row for h .
4. Use the next smaller gap and scan again. New relationships need comparisons; earlier passes reduce long-distance disorder rather than certify final positions.
5. Stop after gap one. Empty and singleton inputs need no work. Return every held item and stop looking backward when $j < h$.

Walk through the numbers

1. Use [8,1,6,2,7,3,5,4] with gaps 4 and 1. At $i=4$, hold 7, shift 8 at position 0, and write 7: [7,1,6,2,8,3,5,4]. The 3 after 1 stays. Insert 5 before 6 across four slots: [7,1,5,2,8,3,6,4]. The 4 after 2 stays.
2. At gap one, insert 1 by shifting 7: [1,7,5,2,8,3,6,4]. Insert 5: [1,5,7,2,8,3,6,4].
3. Insert 2 by shifting 7 and 5: [1,2,5,7,8,3,6,4]. Leave 8. Insert 3 by shifting 8,7,5: [1,2,3,5,7,8,6,4].
4. Insert 6 by shifting 8 and 7: [1,2,3,5,6,7,8,4]. Insert 4 by shifting 8,7,6,5: [1,2,3,4,5,6,7,8]. The gap-four result was still globally unordered.

How much work and room it needs

Let n count items and h be the gap. Each pass checks roughly $n-h$ items and may shift repeatedly. Knuth $3h+1$ gaps give worst $O(n^{3/2})$; sorted input still scans about $\log n$ gaps, giving best about $\Theta(n \log n)$. No universal exact random-input formula is assumed. Original halving gaps allow worst $\Theta(n^2)$. Pratt uses all $2^p 3^q$ below n , for nonnegative p and q , giving worst $O(n \log^2 n)$ with more passes. Extra space $O(1)$; unstable.

Why it is fast or slow

Long steps move distant wrong items quickly. Poor gaps can leave important relationships untouched until the final pass.

When it is useful

Modest arrays requiring low memory and simple movements.

Details that make the procedure work

Ciura gaps 1,4,10,23,57,132,301,701 have several extension policies, so one bound does not cover every continuation. Indirect equal crossing occurs: [2a,2b,1,3] at gap two becomes [1,2b,2a,3].

References [77] [31] [78]

41 Funnel sort

Type Algorithm Aliases or related names Funnel sort; a cache-oblivious sorting algorithm

First understand the small actions

Cache is a small fast storage area near the processor. Main memory is larger but slower. Reading often transfers a block of neighboring items; nearby access can reuse that work.

Joining two ordered inputs compares their unused fronts, emits the smaller, and advances only its source.

Use a contiguous circular array for each first-in, first-out queue. Track read head, write tail, and count. Reads advance head and decrease count; writes advance tail and increase count. Wrap at capacity without overwriting unread records.

An upstream merger produces batches for a downstream consumer. Empty buffered space does not necessarily mean the source has ended.

The batch formula $k^{3/2}$ means k times $\sqrt{k}$. At $k=4$, $\sqrt{k}=2$, a batch size is 8, and queue capacity is 16. Short sources produce only their real remaining records.

The idea in plain words

Order small pieces and join them through recursively arranged batch-producing mergers.

Intermediate queues let small mergers reuse nearby data. Cache-oblivious means the program needs no actual cache capacity as a parameter; its structure still aims to use caches efficiently.

Carry it out one action at a time

1. Split n items into about $n^{1/3}$ contiguous pieces of about $n^{2/3}$ items and sort each by these rules. For concrete teaching boundaries, use insertion at $n \leq 2$; otherwise choose $\text{ceil}(n^{1/3})$ pieces, with lengths differing by at most one. Ceil rounds upward. Eight items form two four-item pieces; four form two pairs. Insertion holds the next item, shifts larger predecessors, and fills the opening.
2. Join k inputs with a k -merger. Two inputs use front comparisons directly. Larger k uses about $\sqrt{k}$ upstream mergers, each joining about $\sqrt{k}$ sources into an intermediate queue. One final $\sqrt{k}$ -merger joins those queues. Build smaller versions recursively to constant-size bases.
3. An original k -merger call produces at most about k^3 items. An intermediate queue holds about $2k^{3/2}$; an upstream $\sqrt{k}$ -merger produces about $k^{3/2}$ per call. Before the next downstream batch, refill each below-half queue whose source has more data. These capacities allow a batch without restarting upstream for every consumed item.
4. Upstream mergers use the same rule at smaller levels, writing queue tails and updating counts. Record true source exhaustion separately. An exhausted source may still have buffered records: consume them before withdrawing it.
5. Downstream compares available fronts, emits the smaller, advances that queue's head, and reduces count. Choose the originally earlier input stream on ties at every level. Inspect and refill between batches until the request completes or all sources end.
6. Each original record passes through the necessary stages, never twice into final output. Write the finished ordered sequence back into this range and supply it to the larger sorting task.

Walk through the numbers

1. Sort [7,1,6,2,8,3,5,4]. The top level has two four-item pieces. In the first, order [7,1] and [6,2] as [1,7] and [2,6], then compare fronts to produce [1,2,6,7]. The second similarly produces [3,4,5,8].

2. The final join takes 1,2, then compares left front 6 with right fronts 3,4,5, taking them first. It then takes 6,7 and remaining 8, producing [1,2,3,4,5,6,7,8].
3. Inspect a four-input merger separately with the same records as [1,7],[2,6],[3,8],[4,5]. Its two upstream mergers produce $Q1=[1,2,6,7]$ and $Q2=[3,4,5,8]$. This does not replace the eight-item top-level split.
4. Downstream takes $Q1$'s 1 and 2, advancing head from 0 to 2. It takes $Q2$'s 3,4,5, advancing that head to 3, then takes $Q1$'s 6,7 and $Q2$'s 8. These streams fit, so no refill occurs. Larger inputs show repeated consumption and replenishment.

How much work and room it needs

Let n count items. Ordinary comparison and transfer work is typical and worst $O(n \log n)$, also a best-case upper bound rather than a general linear-adaptation promise. Output commonly needs $O(n)$ slots; a k -merger has $O(k^2)$ buffer storage. With ideal cache capacity M items and blocks of B items, the original contiguous layout and batching give $O(1+(n/B)(1+\log_M n))$ transfers when $M=\Omega(B^2)$. These count blocks, not CPU comparisons. Earlier-stream ties at every level give stability.

Why it is fast or slow

Batching concentrates small-merger work around reusable records. Queue management and recursive control can outweigh cache benefits on small data.

When it is useful

Memory-hierarchy research and large-data comparison sorting. Learn two-way joining and circular queues first.

Details that make the procedure work

M and B are analysis parameters, not program settings. The transfer guarantee needs its ideal-cache and tall-cache assumptions. Merger structures as well as queues must retain the original contiguous layout; scattered nodes do not automatically inherit it.

References [\[79\]](#) [\[80\]](#)

42 Merge-insertion sort

Type Algorithm Aliases or related names Ford–Johnson algorithm

First understand the small actions

One pair comparison gives a fact we can reuse: the smaller belongs before the larger. Label the pair (b,a) , with b small and a large. These are names, not extra values. Pair $(3,7)$ records $3 \leq 7$.

When inserting 3 into an ordered chain and knowing its partner is 7, search only before 7. Binary insertion inspects a middle candidate and removes impossible half-ranges. Finding the gap does not remove the shifts needed to make room.

The insertion order targets comparison thresholds. Seven ordered items have eight insertion gaps, which three yes-or-no decisions can distinguish. Eight items have nine gaps, so three decisions may not suffice. The schedule aims for search lengths near 3,7,15, and so on.

The idea in plain words

Compare pairs first, sort their larger members while retaining partners, then insert smaller members only before their partners. A special insertion schedule keeps search prefixes within useful comparison thresholds.

Carry it out one action at a time

1. For an empty or one-record total input, return immediately without creating pairs or a main chain. Otherwise compare neighboring pairs and label each smaller b and larger a . With m pairs and one unmatched item, label that item $b_{(m+1)}$; it joins the same insertion groups but has an end-of-chain bound.
2. Sort the a members recursively with this method, carrying each b partner whenever its a moves. Relabel so $a_1 \leq a_2 \leq \dots$, preserving $b_i \leq a_i$. A larger-member subtask with zero or one a needs no further comparison. That does not end the whole sort: build the main chain and insert pending b members and any unmatched record.
3. Begin the main chain $[b_1, a_1, a_2, \dots]$. Since $b_1 \leq a_1$ and a_1 is the smallest larger member, b_1 can lead without another comparison. Other b members remain pending.
4. Process pending indices in descending groups: b_3, b_2 ; b_5, b_4 ; $b_{11}, b_{10}, \dots, b_6$; then b_{21} down to b_{12} . Skip nonexistent indices. Boundaries start 1,3; each new boundary is the previous plus twice the one before it, producing 5,11,21,....
5. Locate b_i 's partner a_i in the current chain and binary-search only the ordered prefix before it. Save b_i , shift later chain records right, and insert at the gap. Partner positions change after insertions; do not use stale indices. An unmatched item searches the whole chain.
6. When no pending records remain, the chain is ordered and complete. Use consistent equal-key rules; original-position tie keys can provide stability from the start.

Walk through the numbers

1. Pair $[7,3,6,1,8,2,5,4]$ into $(3,7), (1,6), (2,8), (4,5)$. Sorting larger members 7,6,8,5 into 5,6,7,8 carries partners to $(4,5), (1,6), (3,7), (2,8)$.
2. Start chain $[4,5,6,7,8]$. Pending $b_2=1$, $b_3=3$, $b_4=2$ follow the group schedule. Insert b_3 first, searching only $[4,5,6]$ before partner 7.
3. For key 3, middle 5 is larger, then remaining 4 is larger. The gap is before 4. Shift to make room and obtain $[3,4,5,6,7,8]$.
4. Insert $b_2=1$ before partner 6 by searching $[3,4,5]$. Inspect 4 then 3; both are larger, so insertion at the front gives $[1,3,4,5,6,7,8]$.
5. Finally $b_4=2$ searches the six-item prefix $[1,3,4,5,6,7]$ before partner 8. Repeated middle comparisons find the gap between 1 and 3. Shift and insert to get $[1,2,3,4,5,6,7,8]$. This group permits at most seven searched items; this search has six.

How much work and room it needs

The main saving is key comparisons, particularly smaller terms below the $n \log n$ scale. Comparisons are $\Theta(n \log n)$ for n records, without being optimal for every n . Array insertion still shifts records, so a straightforward implementation can take $O(n^2)$ worst time. Teaching implementations commonly use $O(n)$ partner/chain metadata and $O(\log n)$ recursive stack. Ordinary versions are unstable. Expensive comparisons and cheap moves can justify the management overhead.

Why it is fast or slow

Binary insertion into at most $2^q - 1$ items needs at most q comparisons. The special insertion schedule exploits those thresholds, at the cost of bookkeeping.

When it is useful

Small datasets with very expensive comparisons and research on comparison counts.

Details that make the procedure work

A partner's identity matters more than a fixed position number. Track its current location after every insertion. Sorting larger members while forgetting their partners loses the reason search ranges can be restricted.

References [81]

43 Indirect sorting

Type Supporting technique Aliases or related names index sorting; pointer sorting; tag sort in the index sense

First understand the small actions

A record may be large, holding photos, addresses, and many fields. Moving it repeatedly can be expensive. Instead assign a small position number and move those numbers. Index 0 means read original position 0; it does not mean the key is zero.

Comparing indices still reads their record keys. If index 0 points to 30 and index 1 to 10, index 1 belongs first. An index is a small direction sign while the large record stays where it was.

The idea in plain words

Sort small indices to obtain a reading-order list. Following the list gives an ordered view without moving records. If physical rearrangement is required, follow the resulting cycles once. This is a movement-saving technique that still needs an underlying sorter.

Carry it out one action at a time

1. Create index list $[0, 1, \dots, n-1]$ for n records. Each index uniquely identifies one original record.
2. Choose a known sorter such as merge sort. To compare indices i, j , inspect keys at original positions i, j . Exchange or shift the indices rather than entire records.
3. For an ordered view, read records through the sorted index list and stop. Original positions remain available for another independently ordered view.
4. For physical rearrangement, interpret $\text{index}[p]$ as the original record required at target p . Keep visited marks. For each unvisited start s , save its record and let the hole position be $p=s$.
5. Read $q=\text{index}[p]$. If $q=s$, fill p from the saved record, mark p visited, and finish the cycle. Otherwise move q 's record into p , mark p visited, set $p=q$, and continue through the original mapping. Both branches mark the position just filled, including the last position that closes the cycle. Do not destroy an unread mapping entry.
6. Complete every cycle. Comparing original positions after equal keys can make both the view and final arrangement stable.

Walk through the numbers

1. Keys $[30, 10, 20]$ occupy positions $0, 1, 2$. Start indices $[0, 1, 2]$, then sort by referenced keys to obtain $[1, 2, 0]$.
2. Reading original positions $1, 2, 0$ displays $10, 20, 30$. The large records have not yet moved.
3. For physical rearrangement, target 0 needs original position 1. Save 30 from position 0, move 10 from position 1 into 0, and let position 1 become the hole.
4. Target 1 needs original position 2, so move 20 into that hole. Target 2 needs original position 0, whose 30 is safely held. Fill the last hole with it.
5. Output is $[10, 20, 30]$. Cycle $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$ is complete. Mark its positions to avoid processing it again.

How much work and room it needs

Sorting indices still has to discover key order; it is not automatically a general $O(n)$ sort. A common comparison sorter takes $O(n \log n)$, with $O(n)$ index storage. Physical rearrangement adds $O(n)$ large-record moves, one temporary record, and visited information; indices can sometimes store marks without destroying needed mappings. The saving is repeated large-record movement. Indirect key reads may have poorer locality. Best, worst, and stability properties remain those of the selected sorter.

Why it is fast or slow

Moving indices is cheap compared with moving large records. Indirect key reads can hurt locality and offset some of that saving.

When it is useful

Large records, immutable original data, and several independently ordered views.

Details that make the procedure work

Decide whether an ordered view or a physically reordered array is required. State the mapping direction: confusing target-to-original with original-to-target causes incorrect moves.

References [\[82\]](#)

4 External parallel and restricted-input methods

These models cannot share one timing list with serial in-memory sorting. Count block-transfer passes for external data, total work and dependency chains for CPU/GPU tasks, and comparators and layers for networks. External, parallel, network, and restricted-input entries are separate model groups. Replacement selection generates runs rather than completing the sort.

44 External multiway merge sort

Type Algorithm family Aliases or related names k-way external merge sort

First understand the small actions

Imagine a table that holds a small pile of cards and a cupboard holding the rest. Memory is the table; disk is the cupboard. Working at the table is easier than repeatedly bringing cards from storage. External sorting handles data that cannot all fit in memory.

A run is a sequence of records already written in order. We merge runs by inspecting their current fronts. A buffer holds a batch read from disk, so the next record can be taken without another disk visit. Refill only when the batch is used up. Output is also collected into batches.

For many run fronts, a min-heap avoids searching every front again. Each heap item stores a key and the run it came from. Its root is smallest. After emitting that root, replace it with the next record from the same run and exchange downward with the smaller child until the parent rule holds.

Number array positions from zero. Children of position i are $2i+1$ and $2i+2$, but exist only below the current heap size. A non-root position has parent $\text{floor}((i-1)/2)$, where floor discards the fraction. Moving upward repeatedly compares with that parent. Moving downward chooses the smaller existing child and exchanges only when the current record is larger.

The idea in plain words

If the whole file does not fit on the table, sort manageable batches first. Then repeatedly emit the smallest remaining run front. Advance only the run that supplied it. Reading moves forward instead of searching the entire file for each output.

Carry it out one action at a time

1. Let M be available memory measured in records and B a disk block measured in records. To merge F runs, reserve F input buffers, an output buffer, and heap metadata within M . Choose F at least two and small enough for that budget.
2. Read a manageable batch in original file order, sort it with an in-memory method, and write an ordered run. Continue until the source file ends. Several ordered runs still do not form one globally ordered file.
3. Choose at most F runs and read their first blocks. Put each front record and its run identity in a min-heap. One simple construction inserts items at the end and swaps each upward while it is smaller than its parent.
4. Emit the root into the output buffer. Replace it with the next item from its own run, then exchange downward with the smaller child when needed. If that run is exhausted, fill the root from the last heap item, reduce the heap size by one, and restore the rule by the same downward comparisons. With zero or one remaining item, stop without reading nonexistent children.
5. Refill an input buffer only after its records are consumed. Write output when its buffer fills. At the end, also write the final partly filled output block.
6. Merge the resulting longer runs in further batches until only one remains. The number of runs decreases each round. Include an incomplete final batch too.

Walk through the numbers

1. For a visible example, choose batches of three. Split [7,2,5,8,1,4] into [7,2,5] and [8,1,4], then sort them into [2,5,7] and [1,4,8]. A real batch size also respects the buffering budget.
2. The fronts are 2 and 1. Emit 1; that run advances to 4, so the candidates are 2 and 4. Emit 2; its run advances to 5, giving candidates 5 and 4.
3. Emit 4, leaving 5 and 8. Emit 5, leaving 7 and 8. Emit 7; its run is empty. Append the remaining 8. Output is [1,2,4,5,7,8].
4. Every chosen front is the smallest record still available in its own run. Choosing the smallest of those fronts therefore cannot miss a smaller record hidden later in a run.

How much work and room it needs

Each merge round reads and writes the complete data, using roughly n/B block transfers, where n counts records. Larger usable fan-in usually means fewer rounds. With memory M fully used, the standard external-memory bound for $n > M$ is $O((n/B) \log_{(M/B)}(n/B))$ block transfers. That counts storage traffic, not seconds. Conventional comparison work is bounded by $O(n \log n)$; memory uses $O(M)$, with temporary disk files for new runs. Stability requires stable run generation and consistent original-order ties during every merge.

Why it is fast or slow

Large sequential transfers and fewer merge passes save time. Increasing the fan-in helps only while adequate buffers and file handles remain available.

When it is useful

Large files, database sorting, and logs that exceed RAM. Compare it with other external methods using the same memory and disk system.

Details that make the procedure work

Skip empty runs. When batches are consecutive in the original file and merged in that order, favoring earlier runs on equality preserves record order. Arbitrary run reordering is safer with original-position tie keys. Account for stream limits and the last partly filled block.

References [\[83\]](#) [\[84\]](#)

45 Balanced multiway external merge

Type Important variant Aliases or related names balanced k -way merge

First understand the small actions

Merging tells us how to combine runs. File scheduling answers another question: where do old runs live, and where can new runs be written safely? Separate input and output groups keep new output from overwriting unread data.

A file may contain several runs. Their lengths or end markers must be stored. A file holding [1,7] followed by [2,8] is not one ordered four-item run: the boundary matters.

The idea in plain words

Spread old runs across one file group, merge them in batches, and rotate new runs across a second group. After a round, exchange the groups' jobs. Balanced refers to run distribution for the next round, rather than a new comparison operation.

Carry it out one action at a time

1. Choose fan-in F at least two. Prepare F input streams and F output streams, record every run boundary, and fit the needed buffers and front-record metadata in memory.

2. Take the next run from each input stream that still has one. Merge this batch by comparing fronts, emitting the smallest, and advancing its source. For many streams, use a min-heap for the fronts.
3. Write the first resulting run to the first output stream, the next to the second, and so on cyclically. Keep each new run boundary distinct from the following run.
4. Finish the round when all old runs have been consumed. Reopen or rewind streams so the output group becomes input and the old input group can hold new output.
5. Continue until one run remains. Merge incomplete batches normally. If the whole job already consists of one run, it is finished.

Walk through the numbers

1. With fan-in two, A contains runs [1,7] and [2,8]; B contains [3,6] and [4,5]. C and D begin empty.
2. Merge A's first run with B's first. Choose 1 before 3, then 3 before 7, then 6 before 7, and append 7. Write [1,3,6,7] to C.
3. The second pair produces [2,4,5,8], written to D. Four short runs have become two longer runs, one per output stream.
4. Read C and D in the next round. Front comparisons emit 1,2,3,4,5,6,7,8. Run counts have changed from four to two to one.

How much work and room it needs

Starting with R runs, a round reduces their count by roughly a factor F . That gives about $\text{ceil}(\log_F R)$ merge rounds; ceil rounds a fractional result upward. Each round transfers $\Theta(n/B)$ blocks for n records and B records per block. Conventional total comparison work is bounded by $O(n \log n)$. Buffer memory has scale $O(FB)$, plus front metadata; temporary disk uses $O(n)$, with commonly $2F$ work files. Ordered keys alone do not remove rounds unless longer initial runs are recognized.

Why it is fast or slow

A high fan-in reduces passes without a separate redistribution pass. The trade-off is the second set of files and the resources needed for concurrent streams.

When it is useful

External sorting on systems that can support enough working files. Compare against polyphase merging with the same stream limit.

Details that make the procedure work

Balanced run counts do not guarantee equal run byte sizes. Stability needs ordered ties at run creation and merging. If distributing runs changes their original relative order, retain original positions as tie keys.

References [\[83\]](#)

46 Polyphase merge sort

Type Important variant Aliases or related names polyphase external merge

First understand the small actions

A stream is a sequence read or written forward, much like a tape. An input has unread runs; an output needs safe empty space. Three streams can merge with two reading and one writing, provided their run counts allow useful role changes.

Here is a concrete three-stream plan. For R real runs, begin with counts $(1,1)$ and repeatedly replace (a,b) by $(a+b,a)$ until their sum reaches R : $(1,1)$, $(2,1)$, $(3,2)$, $(5,3)$, and so on. Put a runs in A and b in B ; leave C empty. Fill surplus run slots with dummy runs containing an end marker but no records. These counts schedule runs; they say nothing about key values.

The idea in plain words

Rotate which streams read and write as runs are consumed. When an input runs out, it becomes the next empty output, and newly produced runs become input. A phase is the sequence of merges performed while those roles stay fixed.

Carry it out one action at a time

1. Create ordered runs from manageable batches. Distribute them into A and B using the count pair above, adding empty dummy runs as needed. Keep run boundaries and two-input, one-output block buffers.
2. Take one run from each input and merge into the output. Choose the smaller front and advance its source. A dummy contributes nothing, so its partner can pass through unchanged.
3. Continue pairing runs until one input exhausts its runs. End this phase without copying the output back to a fixed original input.
4. Use the exhausted input as new output. The old output and the remaining old input now supply runs, beginning at their next unread run boundaries.
5. Repeat role changes until one real run holds all records. Additional streams need the corresponding generalized count schedule rather than this three-stream pair sequence.

Walk through the numbers

1. A has $[2,7]$ and $[1,8]$; B has $[3,6]$; C is empty. Counts $(2,1,0)$ match the three-stream plan.
2. In phase one, merge $[2,7]$ with $[3,6]$. Emit 2, then 3, then 6, then 7 into C . A retains $[1,8]$, while B has no runs left.
3. Let B write in phase two. Choose A 's 1 before C 's 2, then emit 2,3,6,7 and finally A 's 8. B now holds $[1,2,3,6,7,8]$.
4. A and C are empty and B contains every original record in one run. Dummy runs would add no values and remove no values.

How much work and room it needs

The aim is to avoid redistributing files on devices with few streams. Traffic depends on which runs meet and on their lengths, so polyphase does not automatically beat balanced merging. Conventional comparison work is bounded by $O(n \log n)$, with n records. F input ways normally use $F+1$ streams and $O(FB)$ buffering for B -record blocks, plus run metadata; temporary disk has scale $O(n)$. Here $F=2$. Scheduled counts control phases, but unequal run sizes and dummy placement still affect actual transfers.

Why it is fast or slow

Changing stream roles saves redistribution when the number of sequential devices is tightly limited. That advantage may matter less on modern random-access storage.

When it is useful

Tape sorting and systems with very few sequential streams; also a useful lesson in scheduling external merges.

Details that make the procedure work

Phases may combine runs that were not adjacent in the original file. For stability, retain original positions and compare them after equal keys, including during initial run sorting. Always preferring a particular stream is not enough.

References [\[85\]](#) [\[86\]](#)

47 Replacement selection for run generation

Type Supporting phase Aliases or related names replacement-selection run generation

First understand the small actions

A run need not be as short as memory capacity. If you can hold three cards, removing one creates room to read another. A new key at least as large as the last output can still belong to the current increasing run. A smaller key must wait for a later run.

Frozen does not mean deleted. It means temporarily excluded from this run's choice of smallest record. Current and frozen records share one capacity of m items. Use a run label or two regions within that storage. A min-heap organizes current candidates.

Number array positions from zero. Children of position i are $2i+1$ and $2i+2$, but exist only below the current heap size. A non-root position has parent $\text{floor}((i-1)/2)$, where floor discards the fraction. Moving upward repeatedly compares with that parent. Moving downward chooses the smaller existing child and exchanges only when the current record is larger.

The idea in plain words

Emit the smallest currently allowed key and refill its space from the file. A key compatible with the current output remains eligible; an incompatible key waits. The run can grow beyond the number of records held at once, but all resulting runs still need merging.

Carry it out one action at a time

1. Read up to m initial records and make a current min-heap. No record has yet been output in this run, so every initial key is allowed. The frozen region starts empty.
2. Remove the root as the next output and remember its key, last. Fill its place from the final current-heap item, reduce the current heap size by one, and exchange downward with the smaller existing child until the rule holds. For zero or one remaining item, stop without reading children. Frozen records do not belong to this current-heap size.
3. If unread input remains, read one record into the released capacity. If its key is at least last, insert it into the current heap, adjusting upward. Otherwise store it frozen and exclude it from this run.
4. When the current heap empties, end the run. Make all frozen records current, build their heap, and start another run with no previous output restriction.
5. After the input ends, stop refilling but drain the current run and any frozen run. Merge the generated runs afterward to sort the whole file.

Walk through the numbers

1. With $m=3$, read $[4,1,3]$ from $[4,1,3,2,6,0,5]$. Candidates are 4,1,3 and the root is 1. Candidate lists here show membership, not a sorted heap-array layout.
2. Emit 1 and read 2. Since $2 \geq 1$, current candidates become 2,3,4. Emit 2 and read 6; candidates become 3,4,6.
3. Emit 3 and read 0. Since $0 < 3$, freeze it. Current candidates are 4,6 and frozen candidates contain 0: still three stored records. Emit 4 and read 5, which joins the current heap.
4. Input has ended. Emit 5 and 6, completing $[1,2,3,4,5,6]$. Unfreeze 0 to create run $[0]$.
5. Concatenation would be wrong because 0 is smaller than the first run. Merging the two runs finally gives $[0,1,2,3,4,5,6]$.

How much work and room it needs

Let n be the total number of records, m the record capacity in memory, and B the number of records per disk block. Each record is read once and written into one run. A heap of at most m records takes about $\log_2(m+1)$ adjustment levels per entry or removal; adding one also covers the constant work when $m=1$. Run generation is bounded by $O(n \log(m+1))$, with $O(m)$ memory and $\Theta(n/B)$ sequential block transfers. Count the final partial block too: $\Theta(\text{ceil}(n/B))$ is the precise form for nonempty input, and empty input needs no transfer. Final merging is separate. Under an independent random continuous-key model, average run length is about $2m$; this is not an all-input guarantee. Ordered input can form one long run, while reversed data can give runs near m . Original-position ties can preserve stability.

Why it is fast or slow

Longer runs may remove a whole merge pass. The gain must be weighed against frequent heap updates and less local memory access.

When it is useful

External sorting when memory is small and reading or writing the file is expensive.

Details that make the procedure work

Allow equality: a new key must be at least the last output, rather than strictly larger. Frozen records still consume capacity. Do not allocate m current records plus another m frozen records while claiming an m -record memory limit.

References [\[83\]](#)

48 External distribution sort

Type Algorithm family Aliases or related names external bucket or range-partition sort

First understand the small actions

Imagine dividing cards into boxes for keys below 4, keys from 4 through 6, and keys at least 7. Every key in an earlier box precedes every key in a later box, although each box may be unordered. External distribution sorting uses disk files as those boxes.

Boundaries can come from a known range or from a small sorted sample. Spaced sample values estimate useful divisions but do not guarantee balanced bucket sizes. Each bucket uses a small write buffer to collect a block before a disk write.

The idea in plain words

First send each record to the file for its key range. Sort a bucket in memory if it fits; divide it further if it does not. Once every bucket is internally ordered, concatenate them in range order. The boundaries already guarantee order between buckets.

Carry it out one action at a time

1. Choose disjoint ranges covering every key and assign boundary values consistently, for example $x < 4$, $4 \leq x < 7$, and $x \geq 7$. Fit the bucket write buffers and metadata within memory.
2. Scan the input once, placing each record in exactly one bucket. Write full buffer blocks and the last partial blocks. Record each bucket's actual size.
3. Load a fitting bucket and sort it with a complete in-memory sorter. Classification into a bucket is not itself internal ordering.
4. Subdivide an oversized bucket, checking progress. All-equal keys can finish directly. If sample boundaries send everything to one bucket, isolate records equal to the observed minimum and put larger records in another bucket, so the unfinished task really shrinks.
5. Read completed buckets from lowest range to highest. No cross-bucket merge comparisons are needed because the ranges already determine their order.

Walk through the numbers

1. Scan [7,2,5,8,1,4] with ranges $x < 4$, $4 \leq x < 7$, $x \geq 7$. Send 7 to bucket three, 2 to one, 5 to two, 8 to three, 1 to one, and 4 to two.
2. The files hold [2,1], [5,4], and [7,8]. The first two are still unordered. Sort each pair to obtain [1,2], [4,5], and [7,8].
3. Write 1,2, then 4,5, then 7,8. Result: [1,2,4,5,7,8]. A first-bucket key is below 4 and a second-bucket key is below 7, so appending ranges cannot create a cross-bucket inversion.
4. Repeatedly splitting [5,5,5,5] by the same ineffective boundaries would make no progress. An all-equal check finishes it while retaining all four records.

How much work and room it needs

One distribution level reads and writes every record, using $\Theta(n/B)$ block transfers for n records and B -record blocks. F bucket buffers use roughly $O(FB)$ memory plus metadata. Balanced comparison-based divisions can reach the same transfer scale as standard external merging. Fixed-digit external radix variants instead count digit passes. Poor divisions cause many repeat scans; without progress checks a naive version may not terminate. Temporary files use about $O(n)$ per level, with reclamation reducing peak storage. Stability requires order-preserving distribution and stable bucket sorting.

Why it is fast or slow

Balanced partitions are independent and can be processed in parallel. Poor splitters and frequent equal keys concentrate work in a single bucket.

When it is useful

Large range-key datasets and external integer sorting, after checking distribution and bucket sizes.

Details that make the procedure work

All equal keys must follow consistent boundary rules. A sample estimate is not the actual bucket size: count the records and decide the next step from those counts.

References [83] [84]

49 Parallel merge sort

Type Important variant Aliases or related names parallel mergesort

First understand the small actions

Parallel means workers do independent actions at the same time. Two people can sort different piles simultaneously. They cannot safely write different cards into the same output place, so ownership of positions must be settled first.

During a merge, choose a record x and count how many records in the two ordered rows belong before it. That count determines x 's output position. Binary search finds the other row's boundary by repeatedly halving candidates. Merging below and above x then uses disjoint output ranges.

Equal records affect that boundary. If x comes from the original left row, equal right-row records follow it. If x comes from the right row, equal left-row records precede it. This is the stable tie rule.

The idea in plain words

Sort separate pieces concurrently, then divide the merging work too. More workers help only when each task knows which positions it owns and which earlier results it must wait for.

Carry it out one action at a time

1. Split the unsorted range in half and create two sorting tasks. They split further until a small task uses stable insertion sort on one worker: hold the next record, shift strictly larger predecessors right, and fill the gap without passing equal records. Wait for both ordered halves before merging them. Stable leaf sorting and the merge tie rules below are both required to retain the original order of equal records.
2. For merging, copy the other row if one is empty. For a tiny total, use the ordinary front-by-front merge rather than creating many small tasks.
3. Otherwise choose x at the middle of the longer row. Binary-search the other row: find its first key at least x for a left-row pivot, or its first key greater than x for a right-row pivot.
4. Add the lengths before x and before the other boundary. That many records precede x , so place it immediately after their reserved output range. Exclude x from further tasks.
5. Give the two before-boundary pieces to one merge task and the two after-boundary pieces to another. Assign separate left and right output ranges so they can run concurrently.
6. Wait until both complete before the parent uses the result. Unwritten output positions are not ready data.

Walk through the numbers

1. Two tasks turn $[7,2,5,8,1,4]$ into left $[2,5,7]$ and right $[1,4,8]$. We begin inspecting their merge after those tasks finish.
2. Choose left middle 5. In the right row, middle 4 is smaller, so search to its right. The next candidate 8 is at least 5, so the boundary is before 8: $[1,4] \mid [8]$.
3. One left record, 2, precedes 5; two right records precede the boundary. Three records must come before 5, so write 5 into the fourth position.
4. The left task writes only positions one through three. Merge $[2]$ and $[1,4]$: take 1 before 2, take 2 before 4, then append 4. Result: $[1,2,4]$.
5. The right task writes positions five and six, merging $[7]$ and $[8]$. It can proceed independently. After both finish, output is $[1,2,4,5,7,8]$.

How much work and room it needs

Here n is the total number of records to sort. Work W counts the actions of all workers together. Span S counts the longest chain that must happen in order. Total work is bounded by $O(n \log n)$. This binary-recursive merge has $O(\log^2 n)$ span; across about $\log n$ sorting levels, sorting span is $O(\log^3 n)$. If merging remains serial, span can instead be $\Theta(n)$, leaving workers waiting. A usual output array uses $O(n)$ space, with task stacks counted too. On p workers, both W/p and S limit elapsed time, before scheduling and transfer overhead. The stated tie rules preserve stability.

Why it is fast or slow

Large arrays offer enough independent work to repay task overhead. Synchronization and memory bandwidth eventually limit speedup.

When it is useful

Large stable sorts on multicore machines.

Details that make the procedure work

Every task needs both input boundaries and assigned output boundaries. Saying “merge both sides at once” without allocating positions does not prevent overwrites. Empty and single-item tasks must stop.

References [87]

50 Parallel quicksort

Type Important variant Aliases or related names parallel quick sort

First understand the small actions

A pivot is a reference key. Choosing 5 lets us label each record less than 5, equal to 5, or greater than 5. A group may remain unordered internally; only order between groups is known.

Workers must count before sharing output space. If the first worker has one small record and the second has two, the second starts at the small region’s second position. Adding earlier workers’ counts gives an offset. This running addition is a prefix sum: it simply calculates where each worker starts.

The idea in plain words

Sort the pivot’s two sides concurrently and divide the partitioning work as well. Workers classify their slices, write into allocated positions, then sort the small and large regions independently. Equal keys need no further key ordering.

Carry it out one action at a time

1. Choose an input key as pivot and give workers consecutive input slices. Each scans its own slice to count the three categories without discarding the original records.
2. Sum category counts to reserve three contiguous output regions. Within each category, add earlier-slice counts to assign every worker its own starting offset.
3. Rescan each slice in original order. Write a record into the next allocated position for its category and advance that local position. No other worker owns it.
4. Wait until partition output is complete. Sort the less and greater regions with separate tasks, repeating partitioning until ranges are empty, single-item, or small enough for a complete serial sorter.
5. Leave the equal region between the two results. A depth limit and guaranteed fallback can protect against repeatedly unbalanced partitions.

Walk through the numbers

1. Choose 5 for [7,2,5,8,1,4]. Worker one reads [7,2,5], counting one small, one equal, one large. Worker two reads [8,1,4], counting two small, no equal, one large.
2. The small region has length three, the equal region one, and the large region two. Worker one writes 2 into the first small slot. Worker two writes 1 and 4 into the next two. The equal slot receives 5.
3. The large region receives first worker's 7 followed by second worker's 8. State: [2,1,4] | [5] | [7,8]. No records overlap, but the small region is not yet ordered.
4. Further sorting tasks produce [1,2,4] and [7,8]. With 5 between them, output becomes [1,2,4,5,7,8].

How much work and room it needs

A level scans, counts, allocates positions, and writes roughly all records once. Balanced or good randomized pivots give expected $O(n \log n)$ total work. Repeatedly removing only one item can give $O(n^2)$ work without a fallback. Parallel scanning and prefix sums have $O(\log n)$ span, producing $O(\log^2 n)$ balanced sorting span; serial partitioning can leave $\Theta(n)$ span. This buffered version needs $O(n)$ extra record space. Original-slice offsets, input-order scatter, and stable leaf sorting together permit stability. Exchange-based in-place versions are usually unstable.

Why it is fast or slow

Balanced partitions keep workers busy. A chain of tiny and huge partitions concentrates almost all the work on one worker.

When it is useful

Large in-memory arrays, evaluated on the same hardware, thread count, and partition implementation.

Details that make the procedure work

More workers do not fix bad pivots. The equal region avoids repeatedly sorting duplicate keys. Child tasks must not read a region until its partition writes finish.

References [\[87\]](#)

51 Parallel samplesort and IPS4o

Type Algorithm family Aliases or related names in-place parallel super scalar samplesort

First understand the small actions

One pivot divides quicksort into a few regions. Several boundaries divide samplesort into many buckets. Sort a small sample and choose spaced sample keys as boundaries. Good choices make bucket sizes similar, but a sample is not a guarantee.

IPS4o is a specific member of this family. It gathers same-bucket records into blocks of b items before moving them. A classified block is not internally sorted; its records only share a destination bucket. Each worker has small per-bucket buffers.

Moving blocks within the input needs protection against overwrites. Hold one block aside, leaving a hole. Move the block that belongs in that hole, follow the new hole, and eventually place the held block. It is the same preservation idea as temporarily holding one card.

The idea in plain words

Workers classify by several key ranges and then sort the resulting buckets. A basic samplesort can write a separate array. IPS4o instead tries to arrange full blocks inside the original array, using buffers and destination metadata to protect records.

Carry it out one action at a time

1. Sort a sample and choose splitters. Arrange them as a decision tree: compare a middle boundary, select the appropriate half, and continue to a bucket. Equal-splitter keys need consistent treatment or equality buckets.
2. Each worker reads its slice into per-bucket buffers. Write a full buffer as a same-bucket block, only into safe already-read locations. Keep incomplete buffers for cleanup. Its bucket can be identified by classifying its first record through the splitters again, because all records in that block share a bucket. No permanent bucket label need be added to output records.
3. Add worker counts to find global bucket starts and ends. Bucket one begins first; bucket two begins after bucket one's total, and so on. This is another prefix-sum calculation.
4. Identify each full block's bucket from its first record and move these blocks into their target bucket regions. Save or exchange an occupied destination block before overwriting it, following hole cycles when needed. Concurrent movers must uniquely claim each block position.
5. Handle partial boundary blocks and leftover buffers. Classify their individual records and fill the still-empty target slots. Actual IPS4o keeps dedicated boundary metadata; those fragments cannot be ignored.
6. Assign buckets to recursive sorting tasks or small-array sorters. Ordered ranges plus internally ordered buckets finally give the whole result.

Walk through the numbers

1. For [7,2,5,8,1,4], use splitters 3 and 6. The first worker gets local buckets [2], [5], [7]; the second gets [1], [4], [8].
2. Totals are two per bucket. Reserve positions 1–2, 3–4, and 5–6, then arrange [2,1] | [5,4] | [7,8]. Sorting within each gives [1,2] | [4,5] | [7,8].
3. Inspect full-block movement separately with block size two. Suppose classified blocks stand as [8,7] | [2,1] | [5,4]: large, small, middle, instead of small, middle, large.
4. Hold [8,7], leaving the first block empty. Move [2,1] into that hole, move [5,4] into the resulting second hole, and place the held [8,7] into the third. State: [2,1] | [5,4] | [8,7].
5. This block example deliberately has no partial boundaries. Real data also needs the individual-record cleanup before bucket sorting. No block should be overwritten without saving its records.

How much work and room it needs

Work includes sampling, bucket decisions, block movement, boundary cleanup, and recursive sorting. Under the paper's random-sampling and implementation assumptions, total work is $O(n \log n)$ with high probability for n records. That is not a deterministic guarantee for every random choice. With p workers, k buckets, and b -item buffers, buffer storage has scale $O(pkb)$, plus metadata. In-place avoids a second full record array; it does not mean zero extra memory. Time depends on bucket balance, workers, and bandwidth. The usual implementation is unstable.

Why it is fast or slow

Whole-block movement and fewer unpredictable branches improve processor use. Sampling, synchronization, and cleanup can outweigh these gains on small inputs.

When it is useful

Large multicore array sorting where stability is not required.

Details that make the procedure work

Classification, safe block cycles, and boundary cleanup are all necessary. Real concurrent code also needs correct claims and synchronization for block positions. Ordinary samplesort is the family; IPS4o is a particular implementation, not an alternate name.

References [\[74\]](#) [\[88\]](#)

52 GPU radix sort

Type Important variant Aliases or related names parallel GPU radix sorting

First understand the small actions

A GPU is good at giving many small workers similar jobs. It does not automatically make every step faster: transferring data, launching tasks, and sometimes returning the result all cost time.

For the units digit, 21 and 11 share bucket 1. Stable distribution keeps the earlier 21 before 11. A later tens pass must preserve that previously established units order inside equal-tens groups.

A safe output position adds three quantities: where the bucket begins, how many same-bucket records earlier blocks own, and this record's local same-digit rank. Unique positions prevent two workers from overwriting one another.

The idea in plain words

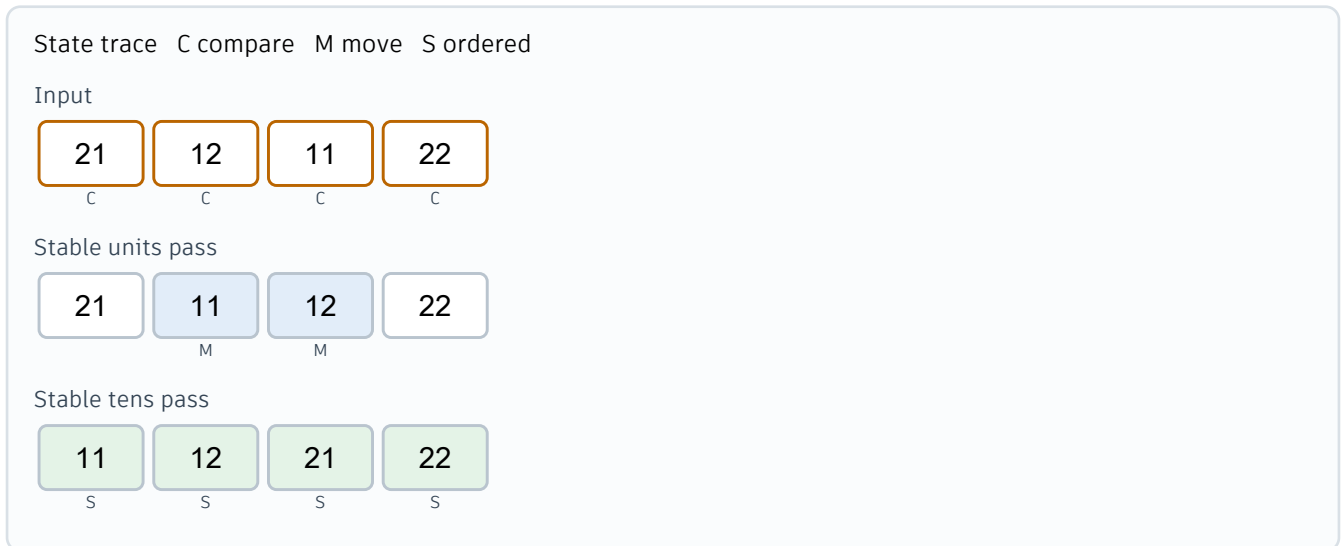
Many workers read the current digit, count categories, allocate positions, and scatter records into buckets. Repeat from low digits to high digits, preserving input order for equal current digits. The successive stable passes create full-key order.

Carry it out one action at a time

1. Choose an order-preserving key representation and digits per pass. Decimal digits below make the example readable; GPUs commonly use groups of binary bits. Signed and floating keys need suitable transformations.
2. Give thread blocks consecutive input slices. Each block counts occurrences of every current digit. This count table is a histogram: just category frequencies.
3. Add block counts to find total bucket sizes and cumulative starts. Within each bucket, add counts from earlier input blocks to assign disjoint block offsets.
4. Compute a same-digit local rank in input order. Write each record at bucket start plus block offset plus local rank, carrying its associated fields too. Each record gets exactly one position.
5. Use this output as the next pass's input and process higher digits. Stop only after all required digits, transferring the result to the CPU if it needs the data.

Walk through the numbers

1. Split [21,12,11,22] into block one [21,12] and block two [11,22]. Each block contains one units-1 record and one units-2 record.
2. The two units-1 records reserve positions one and two. Units-2 begins at position three. Block one gets the first place in each bucket; block two gets the next.
3. Write 21 to position one, 12 to three, 11 to two, and 22 to four. Output is [21,11,12,22]. Equal units retain their original order.
4. Now distribute by tens stably. Tens-1 receives 11 then 12; tens-2 receives 21 then 22. Output [11,12,21,22] is numerically ordered because units order survives within equal-tens groups.



How much work and room it needs

Each pass reads records, counts digits, calculates offsets, and writes records. With d passes, k buckets per pass, and h thread blocks, total work is $O(d(n+h \cdot k))$ for n records. Only when a fixed configuration has $h \cdot k = O(n)$ can the common $O(d(n+k))$ shorthand apply. Typical extra storage is $O(n+h \cdot k)$, including output and block counts. Stable LSD passes permit stable output. Bandwidth, launches, and transfers affect elapsed time; tiny examples usually do not justify GPU overhead. Treat d as constant only for fixed-width keys.

Why it is fast or slow

Many records can be counted and moved together. Keeping data on the GPU avoids transfer costs; small arrays may lose to launch overhead.

When it is useful

Large batches of integer or encodable keys already processed on a GPU.

Details that make the procedure work

Without local ranks or block offsets, equal-digit records can claim the same output place. Signed keys, floating special values, and the desired ordering policy must be settled before sorting. Official implementations supply device-resource and interface details.

References [\[89\]](#) [\[90\]](#)

53 Bitonic sorting network

Type Algorithm Aliases or related names bitonic sort; sometimes Batcher sort

First understand the small actions

A comparator is one small operation on two positions: read their keys and put the smaller at the designated low end and the larger at the other end. Equal keys may stay. A network fixes those position pairs in advance, regardless of the values.

Comparators using disjoint positions can run together. Comparing positions 1,2 and positions 3,4 is safe in one layer. A later layer uses their results, so layers must still occur in order.

A bitonic sequence rises and then falls, such as $[1,3,4,2]$, allowing a cyclic rotation of that shape. This structure lets half-distance comparisons separate a low half from a high half. First inspect the concrete two- and four-item cases.

The idea in plain words

Create a rise-then-fall shape, then make it fully increasing with predetermined comparisons. Ascending and descending use the same small operation with opposite choices for the low end. Repeated halving eventually settles every position.

Carry it out one action at a time

1. Use a length obtained by doubling one: 2,4,8,16, and so on. Pad other lengths with special items larger than every real key and remove them afterward. A single item stops.
2. Sort the first half upward and the second downward using this same sorting procedure. When both finish, their concatenation is bitonic. Descending tasks use corresponding descending merges.
3. To merge n bitonic items upward, compare pairs separated by $n/2$ positions. Keep the smaller of every pair in the first half and the larger in the second.
4. Bitonic-merge each half upward by repeating half-distance comparisons, then halving again. This merging stage does not rebuild rising and falling halves. Two items need one comparison; one needs none.
5. A downward merge keeps larger values in the first half. Execute independent pairs together if the hardware allows, waiting between dependent layers.

Walk through the numbers

1. For [3,1,4,2], sort the first pair upward: swap 3 and 1. Sort the second downward: 4 and 2 already fit. State: [1,3,4,2], which rises then falls.
2. For the four-item upward merge, distance is two. Compare 1 with 4 and keep them. Compare 3 with 2 and swap. State: [1,2,4,3].
3. Merge each two-item half upward. [1,2] stays; [4,3] swaps. Output is [1,2,3,4].
4. Six comparisons were scheduled. Even a comparison causing no exchange still takes place: the route depends on length and direction, not the values.

State trace C compare M move S ordered

Make a bitonic sequence



After distance-two comparisons



After within-half comparisons



How much work and room it needs

Ordered and unordered inputs follow the same route. For $n=2^q$, meaning q doublings from one, there are $q(q+1)/2$ layers with $n/2$ comparisons per layer. Total work is $\Theta(n \log^2 n)$, with $\Theta(\log^2 n)$ ideal parallel depth. Iterative serial execution can exchange in the original array with $O(1)$ management storage; recursion adds $O(\log n)$ stack. Hardware wires and comparators are separate resources. It is generally unstable because distant exchanges can reverse equal-record order.

Why it is fast or slow

Predictable routes fit parallel hardware. A serial processor usually pays for more comparisons than a good $O(n \log n)$ sort.

When it is useful

Fixed-size GPU blocks, hardware circuits, and secure computation with fixed access routes.

Details that make the procedure work

Half-distance merging relies on the bitonic shape. Applying only that merge to an arbitrary unsorted row is not a complete sorter. Batcher names several networks, so it is an ambiguous alternate label.

References [91]

54 Odd-even merge sorting network

Type Algorithm Aliases or related names Batcher odd-even mergesort; not odd-even transposition

First understand the small actions

Odd and even here refer to position numbers, not key values. A key of 4 can occupy the first position. Within an ordered half, its first, third, fifth items are ordered, and its second, fourth, sixth items are ordered too.

Merge the two halves' first set of positions and their second set separately. Interleave the two ordered results. Each stream is increasing, but a boundary between streams may still be reversed; selected adjacent comparisons close those gaps.

The idea in plain words

Sort two halves, merge their alternating positions separately, then repair the seams of the interleaved result. Length alone determines every comparator. This network differs from odd-even transposition sorting, which repeatedly sweeps neighboring pairs.

Carry it out one action at a time

1. Use a power-of-two length, padding with larger-than-real items if needed. Sort both halves upward recursively; a single item stops.
2. For merging, two total items need one comparison. Otherwise take first, third, fifth positions within each half and recursively merge those two shorter ordered streams.
3. Merge the second, fourth, sixth positions in the same way. Selecting positions describes participation; an implementation can store a start and stride instead of copying separate arrays.
4. Leave the first result in the combined range's odd positions and the second in its even positions. Compare positions 2,3; 4,5; and so on through the internal pairs, swapping when the left key is larger.
5. Return to the enclosing merge. Participating sequences shrink until only two-item comparisons remain, so the entire comparator route can be generated beforehand.

Walk through the numbers

1. For [3,1,4,2], sorting the halves swaps 3 with 1 and 4 with 2. State: [1,3] | [2,4].
2. The first-position stream contains 1,2. Compare and leave them in combined positions one and three. The second-position stream contains 3,4, which stays in positions two and four. State: [1,3,2,4].
3. The interleaved middle is still reversed. Compare positions two and three, swapping 3 and 2. Output: [1,2,3,4].

4. The five-comparator route is (1,2), (3,4), (1,3), (2,4), (2,3), with one-based positions. The first two pairs share a layer, the next two share another, and the final pair forms a third.

How much work and room it needs

Every input follows the same route. Total comparisons are $\Theta(n \log^2 n)$ and parallel depth $\Theta(\log^2 n)$, with n the padded length. Records can exchange directly in the input array. Recursive route generation needs $O(\log n)$ stack; storing all comparators needs $\Theta(n \log^2 n)$ route space. It is normally unstable. Some sizes use fewer comparators than bitonic networks, but hardware connections and execution still affect elapsed time.

Why it is fast or slow

It often uses fewer comparators than a same-sized bitonic network. Real performance also depends on wiring, synchronization, and access patterns.

When it is useful

Sorting circuits, fixed-size parallel blocks, and applications requiring fixed access routes.

Details that make the procedure work

Distinguish padding from real records, removing only the added items. Concurrent pairs must not share positions. The four-item structure also appears in the small-network lesson, where it is explicitly not counted as another independent invention.

References [\[92\]](#) [\[91\]](#)

55 Fixed small sorting networks

Type Supporting technique Aliases or related names small compare-exchange networks

First understand the small actions

Imagine four fixed lanes, each holding one key. A comparator joins two positions and sends the smaller to the designated left end and the larger right. A four-item input always follows these five comparisons, without choosing a new route from its values.

Disjoint pairs can operate together. Positions 1,2 and positions 3,4 share no record. A layer means one such independent batch; it does not mean all five comparisons can run simultaneously.

The idea in plain words

For four items, first order two pairs, then establish the smallest and largest ends, and finally order the middle. This is a small-network design example, identical to the four-item odd-even merge structure rather than another independent large-input algorithm.

Carry it out one action at a time

1. In layer one, compare positions 1,2 and positions 3,4, swapping when each left key is larger. Both pairs become internally ordered.
2. In layer two, compare positions 1,3, the smaller members of the pairs, to put their minimum first. Compare positions 2,4, the larger members, to put their maximum last. These comparisons are independent.
3. The first key is now globally smallest and the fourth globally largest. Compare positions 2,3 in layer three and exchange if reversed.
4. Finish after all five scheduled comparisons. A quiet earlier layer does not justify skipping later layers. Other lengths require their own verified network.

Walk through the numbers

1. Start [4,2,3,1]. Layer one swaps 4,2 and 3,1, giving [2,4,1,3].
2. Layer two swaps 2,1 and 4,3 across the specified positions. State: [1,3,2,4]. The end keys 1,4 are final.
3. Layer three exchanges middle 3,2 to produce [1,2,3,4]. Every change exchanges existing records rather than creating or deleting keys.
4. Test all four-item inputs made from 0 and 1: there are $2 \times 2 \times 2 \times 2 = 16$. If this comparator network sorts them all, it sorts any totally ordered keys. This is the zero-one principle. Intuitively, a reversed numeric output pair could be separated by a threshold, labeling the larger 1 and the smaller 0; the same route would then leave an unsorted binary pair, contradicting the tests.

State trace C compare M move S ordered

Input



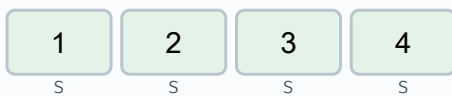
Layer one orders pairs



Layer two fixes extremes



Layer three orders middle



How much work and room it needs

Four fixed items always need five comparisons in three layers, using $O(1)$ temporary record space. That fixed-size count says nothing about general n . Other networks need their own comparator and depth counts. Avoiding recursion and dynamic pivots can help as a tiny base case inside a larger sorter. It is generally unstable: equal records can pass each other through different lanes without being directly exchanged.

Why it is fast or slow

Tiny networks remove recursion and bookkeeping, making useful base cases. Network size and construction difficulty increase as input size grows.

When it is useful

Small base cases inside larger sorts, vector instructions, and hardware blocks.

Details that make the procedure work

Input [2a,3,2b,1] becomes [2a,3,1,2b] after the first layer and [1,2b,2a,3] after the second. The equal middle keys stay in layer three, so their original order has reversed. Not swapping equal values does not by itself prove stability.

References [91]

56 Shear sort

Type Algorithm Aliases or related names snake-order mesh sorting

First understand the small actions

Place four keys in two rows and two columns. A snake path reads row one left to right, row two right to left, row three left to right, and so on.

A displayed second row [4,3] is read as 3,4 along that path. Even rows therefore need descending display order. Ordered rows alone are insufficient: the end of one row may exceed the start of the next. Column sorting moves small keys upward and large keys downward.

The idea in plain words

Alternate complete row and column sorts to shrink the disordered region. Rows follow the snake direction; columns increase downward. After enough rounds, read the snake path to obtain ascending output.

Carry it out one action at a time

1. Use a square grid with s positions per side and $n=s^2$ records. Place input row by row, retaining record identities.
2. Sort odd-numbered rows upward from left to right and even rows downward. Use s alternating neighbor phases. In the first phase compare positions (1,2),(3,4),... simultaneously; in the next compare (2,3),(4,5),... simultaneously, then keep alternating. Exchange when the left key is larger for an upward row or smaller for a downward row. Pairs within one phase share no position, so they can act together. Different rows can also work simultaneously.
3. Sort every column upward from top to bottom using the same s phases, with positions now numbered down the column. Exchange when the upper key is larger. Nonoverlapping pairs in one column phase act together, and different columns act together. This may disturb the previous row order, so another row round follows.
4. A conservative schedule uses $\text{ceil}(\log_2 s)+1$ row-column rounds, followed by one final snake-direction row sort. Ceil rounds upward; for $s=3$, $\log_2 3$ lies between one and two, so its ceiling is two.
5. Read along the snake, not left to right through every row. Non-square variants need their own boundary and round rules.

Walk through the numbers

1. Start with top row [4,1] and bottom [3,2]. Swap 4,1 for ascending top [1,4]. Descending bottom [3,2] already fits.
2. Columns are [1,3] and [4,2]. Keep the left column and swap the right. Rows become [1,2] and [3,4].
3. Sort rows again. The bottom must descend, so exchange 3,4 to get [4,3]. Grid: [1,2] / [4,3].
4. Both columns now ascend, so remaining conservative rounds preserve it. Read 1,2 across the top, then 3,4 right to left across the bottom. Output: [1,2,3,4].
5. Reading every row left to right would give [1,2,4,3]. The records are correct; that reading path is wrong.

How much work and room it needs

On an s -by- s nearest-neighbor processor mesh, an odd-even transposition row or column sorter needs $O(s)$ parallel time. $O(\log s)$ rounds give $O(s \log s) = O(\sqrt{n} \log n)$ time. Work added across all processors is $O(n^{3/2} \log n)$. Those are mesh-hardware conditions; a single processor cannot claim the parallel time while performing all actions alone. Grid exchanges can be in place, and the method is generally unstable. Other row/column sorters change the costs.

Why it is fast or slow

Local row and column communication suits a mesh. On one ordinary processor, many repeated comparisons make it unattractive.

When it is useful

Mesh-processing research and lessons about parallel grid algorithms.

Details that make the procedure work

Finish an entire row or column sort, rather than performing only one neighbor comparison per round. The square assumption, scheduled rounds, final row sort, and snake output all belong to the procedure.

References [\[93\]](#) [\[94\]](#)

57 Heap sorting for bounded displacement

Type Algorithm Aliases or related names k -displaced sorting; bounded-displacement heap method

First understand the small actions

The promise is precise. Compared with final $[1,2,3,4]$, input $[2,1,4,3]$ puts each record only one place away. This is maximum displacement one, not merely a row that looks nearly ordered.

With bound k , the final first record cannot begin at position $k+2$ or later: reaching first would move it more than k places. Search only the first $k+1$ candidates. After each output, reading one more item preserves the corresponding next-candidate guarantee. A min-heap selects from that small window.

The idea in plain words

When disorder is known to have a short range, keep a small candidate heap rather than all records. Repeatedly emit its minimum and read the next input. The displacement promise limits where the next answer can hide.

Carry it out one action at a time

1. Confirm the bound k from the data's rules. Do not choose a small k merely to make sorting faster. Without the promise, use a general sorter.
2. Read at most $k+1$ initial records. Insert into a min-heap by placing each at the end and exchanging upward while it is smaller than its parent. Short inputs simply all enter.
3. Emit the root into the next output position. Replace it with the heap's last item and exchange downward with the smaller child until the heap rule holds.
4. If unread input remains, insert one new record and emit the next minimum. Read one at a time because each output releases one candidate place.
5. After input ends, drain the heap. Original-position tie keys can preserve stability.

Walk through the numbers

1. For $[2,1,4,3,6,5]$, every record is displaced by at most one, so $k=1$ and only two candidates are needed. Read 2,1 and emit heap root 1.

2. Read 4; candidates 2,4 emit 2. Read 3; candidates 3,4 emit 3. Output is now [1,2,3].
3. Read 6 and emit 4. Read 5 and emit 5. Input ends; emit remaining 6. Result: [1,2,3,4,5,6].
4. Misusing $k=1$ for [3,2,1] would inspect only 3,2 and incorrectly emit 2 before the hidden 1 arrives.
The promise is required for correctness, not only efficiency.

How much work and room it needs

At most $k+1$ candidates give roughly $\log(k+2)$ adjustment levels per heap update. For n records the upper bound is $O(n \log(k+2))$, with $O(k+1)$ extra candidate storage. The $k+2$ form includes $k=0$: already ordered input still needs $O(n)$ reading and output. When k approaches n , the small-window advantage disappears. Breaking the promise may give a wrong result. Ordinary extraction is unstable; original-position ties can stabilize it.

Why it is fast or slow

A small heap needs short update paths. When k approaches n , the benefit over a full heap disappears.

When it is useful

Streams with a verified out-of-order window or datasets with a known positional displacement bound.

Details that make the procedure work

A complete sort performed only to discover k would already incur general-sorting cost. Useful promises usually come from known transmission or timestamp disorder limits.

References [\[46\]](#)

5 Higher-work and restricted-operation methods

Gap and full-pass methods appear first, with version-dependent total bounds. The quadratic comparison or movement tier follows, then high-degree and superpolynomial recursion. Cycle sort can reduce writes while retaining many comparisons. Pancake sorting charges for whole-prefix reversals. Order inside a tier follows learning dependencies rather than universal speed.

58 Comb sort

Type Algorithm Aliases or related names Comb sort; CombSort11 is a gap-rule variant

First understand the small actions

A gap is the distance between compared positions. Gap three pairs slot one with slot four, then slot two with slot five.

Round down by dropping a division's fractional part: $4/1.3$ becomes three. The gap must stay at least one.

The idea in plain words

Comb sort begins with distant comparisons, letting misplaced values travel several slots in one swap. It gradually shortens the distance and finishes by checking neighbors. These are coarse adjustments followed by fine cleanup. One large-gap pass does not claim that the whole array is already ordered.

Carry it out one action at a time

1. Begin with the array length as the gap. Before each pass, divide by about 1.3 and round down.
Replace zero with one. This entry uses that particular common rule.
2. Starting at the first slot, compare it with the slot one gap to its right. Swap if the left value is larger.
Move the pair one slot right; stop when its right member would be outside the array.
3. Shrink the gap for the next pass. Continue even after a quiet large-gap pass, since closer relationships have not yet been examined.
4. At gap one, keep that gap and repeat complete neighboring passes. Stop after an entire pass makes no swaps, proving every neighboring relationship is correct.

Walk through the numbers

1. For $[4,1,3,2]$, dividing length four by 1.3 gives gap three. The only pair is 4 and 2. Swap: $[2,1,3,4]$. The 2 crosses three slots at once.
2. Dividing three by 1.3 gives gap two. Compare 2,3 and then 1,4. Neither changes. The array is still $[2,1,3,4]$, so a quiet large-gap pass clearly does not prove order.
3. The next gap is one. Swap 2,1: $[1,2,3,4]$. Keep the remaining pairs 2,3 and 3,4.
4. Keep gap one and scan again. No exchanges occur anywhere, so stop.

How much work and room it needs

Even ordered input must pass through all prescribed gaps. With n items, roughly $\log n$ shrinking-gap passes each examine roughly n values, giving best time $\Theta(n \log n)$. The logarithm counts repeated proportional shrinkage. Neighboring cleanup can still make worst time $O(n^2)$. Extra space is $O(1)$. No single exact average formula applies to every gap policy.

Why it is fast or slow

Long swaps can save many one-slot moves, but cleanup can still repeat. Shellsort is different: it thoroughly insertion-sorts each gap-separated sequence rather than making just one pass.

When it is useful

Use it to study distant comparisons or as a simple experiment. Prefer stronger guarantees when predictable running time matters.

Details that make the procedure work

It is normally unstable: [2a,2b,1] can reverse the equal twos through a distant swap. CombSort11 changes gaps nine or ten to eleven. This remains a variant, and other starting gaps or shrink rules need their own stated conditions.

References [95]

59 Circle sort

Type Algorithm Aliases or related names Circlesort; distinct from other circular-sort proposals

First understand the small actions

Work inward from both ends: compare the first with the last, then the second with the second-last. Keep the smaller value on the left.

A complete pass includes the whole-array scan and every smaller-region scan. A quiet local scan does not mean the full pass was quiet.

Recursive half processing means waiting for the left task to finish, then the right task, before the current task ends. One item needs no comparison.

The idea in plain words

Compare pairs from the outside inward, sending smaller values left and larger values right. Apply the same rule to both halves. A complete pass can leave the array unordered, so start again at the whole array until every region makes no exchanges. The name describes the pairing pattern; the records need not form a physical circle.

Carry it out one action at a time

1. Put one pointer at each end of the current region. Exchange the values only if the left is larger. Move both pointers one slot inward and repeat while they refer to different slots.
2. An odd-length region leaves a middle item. In this version, also compare that item with its immediate right neighbor and exchange if the middle is larger. A one-item region returns without doing this.
3. Split into neighboring halves, assigning the middle item to the left half for odd lengths. Process the left half completely, then the right, using these same rules down to single items.
4. Track whether any exchange occurred anywhere in the full pass. Finish only when a pass starting at the whole array, including every smaller task, makes no exchanges. Otherwise start another full pass.

Walk through the numbers

1. For [4,1,3,2], compare outside values 4 and 2. Exchange them: [2,1,3,4]. Move inward and compare 1 and 3; they stay.
2. Process the left half [2,1]. Exchanging its endpoints gives the full array [1,2,3,4]. Its one-item tasks return. Process the right half [3,4], which needs no exchange.
3. The first pass did exchange records. Follow the stopping rule by doing another complete pass. Every whole-array and smaller-region comparison is now quiet, so finish.
4. For [1,3,2], the outside pair 1,2 is already ordered. The unpaired middle 3 still matters: compare it with its right neighbor 2 and exchange, giving [1,2,3].

5. The halves are [1,2] and [3]. The first stays and the second returns immediately. A later quiet full pass confirms completion. Leaving out the specified middle comparison would miss an essential relation.

How much work and room it needs

A full pass checks the region, both halves, their smaller halves, and so on. There are about $\log n$ halving levels, each doing about n total work. With n items, one pass costs $\Theta(n \log n)$; p passes cost $\Theta(p n \log n)$. Θ means the same growth order. Already sorted input needs one pass, giving best time $\Theta(n \log n)$. Waiting calls use $O(\log n)$ extra space. The author reports common $O(n \log^2 n)$ behavior, not a universal worst-case theorem.

Why it is fast or slow

Long exchanges move records far, and smaller tasks repair local relations. But a symmetric scan does not fully separate the halves; their repairs may require another whole-array pass.

When it is useful

Use it to study pairing, smaller tasks, and full-pass stopping rules.

Details that make the procedure work

It is unstable. Odd-middle variants differ. A quiet pass covers internal adjacent boundaries through smaller tasks and the halves' boundary through the central comparison. A very loose upper bound is $O(n^3 \log n)$: at most $O(n^2)$ exchanging passes, since each decreases the number of inverted pairs, times $O(n \log n)$ work per pass. This is not a speed prediction.

References [96]

60 Cycle sort

Type Algorithm Aliases or related names Write-minimizing comparison cycle sort; known-destination variants differ

First understand the small actions

A write changes the value stored in one slot. A swap normally writes two slots; placing a held value writes just its destination.

A destination is a value's position in the ordered result. If three values must precede it, its destination is slot four.

A square $\square$ marks a logical vacancy after its value has been picked up. Real memory can keep an old copy there; no blank marker needs to be written.

The idea in plain words

Cycle sort tries to write as few slots as possible. Pick up one value, work out where it belongs, and put it directly there. Save the value displaced from that destination, work out its destination, and continue. Eventually a held value belongs in the starting slot. Filling that vacancy closes the chain into a cycle.

Carry it out one action at a time

1. Choose a starting slot and hold its value temporarily. Count strictly smaller values in the suffix after that start. Positions before the start have already been dealt with. The destination is that many slots to the right of the start.
2. If this first destination is the starting slot itself, no later value is smaller. Leave the value there and try the next starting slot.
3. Otherwise examine the destination. If it already contains an equal value, skip it and move right past equal occupants. Replacing an equal value would achieve nothing and could keep returning the

same held value.

4. Save the destination's old value before writing. Put the held value there, then hold the saved old value. Only this destination is written; the logical starting vacancy remains.
5. Recount the destination for the newly held value in the same suffix. Skip equal occupants, place the value, and save its replacement again. When the destination returns to the start, fill it and end this cycle. Advance to later starts to handle any other cycles.

Walk through the numbers

1. Begin with [3,1,4,2]. Hold the first 3. Later values 1 and 2 are smaller, so its destination is slot three. Place 3 and hold the displaced 4: [□,1,3,2].
2. The later 1,3,2 are all smaller than 4. Place 4 in slot four and hold 2: [□,1,3,4].
3. Only 1 is smaller than the held 2. Place 2 in slot two and hold 1: [□,2,3,4]. No later value is smaller than 1, so fill the original vacancy: [1,2,3,4]. Four incorrect slots each received one write.
4. Now start at slot two. Its 2 has no smaller value after it, so no new cycle begins. The same is true of slot three; the last slot needs no separate work.
5. For duplicates, start with [2,1,2,1]. Holding the first 2 gives destination slot three because two ones are smaller. That slot already holds 2, so skip it. Write 2 in slot four and hold 1: [□,1,2,2]. Put 1 in the starting slot: [1,1,2,2]. Only the originally wrong first and fourth slots were written.

State trace C compare M move S ordered

Hold 3; count its destination



Write 3; hold 4



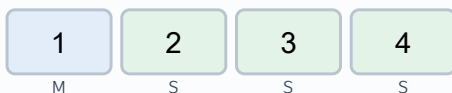
Write 4; hold 2



Write 2; hold 1



Write 1; close the cycle



How much work and room it needs

The cost of saving writes is repeated destination counting. With n items, this comparison-based version has $\Theta(n^2)$ best, typical and worst time: even ordered input scans many suffixes. Array writes total $O(n)$, aiming to write each incorrect value position once. $O(1)$ extra space means one held value and fixed-size control information. It is normally unstable. Special keys with directly computable destinations can support linear cycle methods, under different assumptions.

Why it is fast or slow

It optimizes writes, not every operation. Finding destinations repeats reads and comparisons, but direct placement avoids repeated two-slot swaps and long shifts. This tradeoff matters when writes are much more expensive.

When it is useful

Use it to understand permutation cycles or evaluate storage with costly writes. Estimate reading, comparison and writing separately.

Details that make the procedure work

Skipping equal occupants makes value sorting progress but does not preserve equal records' identities or order. Haddon's linear methods rely on restricted keys or known destinations. Their linear bound does not transfer merely because both approaches use the name cycle sort.

References [97] [98]

61 Rank / enumeration sort

Type Algorithm Aliases or related names Ranking sort; enumeration sort (not exhaustive permutation sort)

First understand the small actions

A record can have a value and an identity. Records 3a and 3b both have value three; their letters only help track them.

A rank starts at zero and counts records that should precede this record. Rank zero means the first output slot; rank one means the second.

The idea in plain words

Rank sort lets each record calculate its own destination. Count all records that should come before it, then write it into the output position given by that count. Equal values must still receive different slots. Original input position provides a clear rule: the earlier equal record belongs first.

Carry it out one action at a time

1. Keep the input available and prepare a separate result array of the same length. Other records still need to examine this input, so writing over it too early could destroy needed information.
2. Choose one record and start its count at zero. Compare it with every other input record. Add one whenever the other value is smaller.
3. For equal values, compare original positions. Add one if the other record originally appeared earlier. Do not count the record itself as preceding itself.
4. After all comparisons, write the record that many slots after the first result slot. Repeat for every input record. The tie rule gives distinct ranks, so output slots do not collide. Read the filled result or copy it back.

Walk through the numbers

1. Use [3a,1,3b,2]. The equal record 3a originally appears before 3b.
2. For 3a, count the smaller 1 and 2, but not the later equal 3b. Rank two places it in slot three: [□,□,3a,□].
3. For 1, no record should precede it. Rank zero gives [1,□,3a,□].
4. For 3b, count the earlier equal 3a as well as smaller 1 and 2. Rank three gives [1,□,3a,3b].
5. For 2, only 1 comes before it. Rank one fills the remaining slot: [1,2,3a,3b]. Ignoring the tie rule would give both threes rank two and overwrite one record.

How much work and room it needs

Every record compares with all other records. With n records, best, typical and worst time are $\Theta(n^2)$, or quadratic comparison work. A separate n -slot result needs $\Theta(n)$ extra space. Each record is written once to the result, so output writes are $O(n)$. Original-position tie breaking makes it stable. Parallel rank calculations can shorten elapsed time, but total comparisons remain quadratic.

Why it is fast or slow

Independent destinations make the work easy to distribute. The disadvantage is examining the same collection repeatedly instead of reusing sorted-region information.

When it is useful

Use it when ranks are useful results themselves, for hardware ranking, or for parallel-algorithm study. Sequential sorting usually has less costly choices.

Details that make the procedure work

Parallel costs also include summing counts and communication. Enumeration here means enumerating comparison partners, not all permutations. Counting sort counts a key range instead. Stability comes from the tie rule, not simply from having a separate output array.

References [99]

62 Selection sort

Type Algorithm Aliases or related names Straight selection sort

First understand the small actions

Number slots from left to right. Comparing values tells you which is smaller; it does not move either value.

To swap two slots safely, save the first value, copy the second into the first slot, then put the saved value in the second slot.

The idea in plain words

Find the smallest value and put it in slot one. Then find the smallest of the remaining values and put it in slot two. Keep going. Each search finishes one position permanently. The important point is that a remembered minimum becomes trustworthy only after every remaining candidate has been examined.

Carry it out one action at a time

1. Treat the first unfinished slot as the destination. Remember its value and position as your current smallest candidate. This is a temporary choice, not yet a final answer.
2. Visit each later slot. Whenever its value is smaller, remember this new position instead. Keep looking to the end; do not move values during the search.
3. Swap the final remembered minimum into the destination. If it is already there, leave it alone. It cannot be greater than any remaining value, so this slot is finished.
4. Move the destination one slot right. Search only that slot and the values after it. Leave the finished prefix alone. When only one unfinished slot remains, every other value is already placed, so the last one is correct too.

Walk through the numbers

1. Begin with [4,2,5,1,3]. Remember 4, replace it with 2, ignore 5, replace 2 with 1, then ignore 3. Only now is 1 confirmed as the minimum.

2. Swap 4 and 1: [1,2,5,4,3]. The 1 stays there permanently; the displaced 4 remains available in the unfinished region.
3. Search [2,5,4,3]. Nothing is smaller than its first 2, so leave the array as [1,2,5,4,3]. Two slots are finished.
4. Search [5,4,3]. The remembered candidate changes from 5 to 4 to 3. Swap the 5 and 3: [1,2,3,4,5].
5. Still compare the remaining 4 and 5. This basic method completes its prescribed searches even when the array happens to become ordered early.

How much work and room it needs

The repeated searches dominate the cost. With n items, they require $n-1$, then $n-2$, down to one comparison: $n(n-1)/2$ altogether. Best, typical and worst time therefore grow quadratically, written $\Theta(n^2)$. There are at most $n-1$ swaps. $O(1)$ extra space means only a fixed amount of temporary storage is needed.

Why it is fast or slow

It saves movement but repeats many comparisons. Even ordered input must be searched. This tradeoff can help when writing a value costs far more than comparing it.

When it is useful

Use it for tiny arrays, learning permanent placement, or a simple situation that especially rewards few writes. Large general inputs usually benefit from other methods.

Details that make the procedure work

The swapping version is unstable. For [2a,2b,1], moving 1 to the front gives [1,2b,2a], reversing equal records. A stable variant removes the first minimum and shifts intervening records right before inserting it, but may perform quadratic movement.

References [\[100\]](#) [\[31\]](#)

63 Double selection sort

Type Important variant Aliases or related names Min-max selection sort; bidirectional selection sort

First understand the small actions

A left and a right boundary mark the still-unfinished region. Values outside it have reached their final positions.

Remembering a minimum or maximum position means remembering where its value lives. A swap can change that address.

The idea in plain words

Double selection sort chooses two values per round: the minimum for the left end and the maximum for the right end. It then works inside those finished ends. This is selection sort with two destinations. A round still needs a complete search, because an unseen value might change either extreme.

Carry it out one action at a time

1. Mark the ends of the unfinished region. Scan every value between them, remembering the smallest and largest values and their current positions. Update either remembered position when appropriate.
2. Swap the minimum into the left boundary. This slot can now be finished because all other remaining values are at least as large.

3. Swap the maximum into the right boundary. First check whether it used to occupy the left boundary: the first swap then moved it to the minimum's old position. Use that new address.
4. Move both boundaries inward and repeat. When only one middle value remains, all values around it have been placed, so no further movement is needed.

Walk through the numbers

1. Start with [5,2,3,1,4]. A full search finds minimum 1 in slot four and maximum 5 in slot one.
2. Swap 1 with the leftmost 5: [1,2,3,5,4]. The maximum now lives in slot four. Using its old slot-one address would accidentally select 1.
3. Swap the current 5 with the rightmost 4: [1,2,3,4,5]. The outer values 1 and 5 are finished.
4. Search only [2,3,4]. Its minimum and maximum already occupy its ends, so make no swaps. Move inward again and leave the final middle 3 in place.

How much work and room it needs

Each round scans the whole remaining region. Shrinking by two instead of one still gives quadratic total search work. With n items, best, typical and worst time are $\Theta(n^2)$. At most two swaps per round gives $O(n)$ swaps altogether. $O(1)$ extra space means a fixed number of remembered positions and temporary values.

Why it is fast or slow

Fewer rounds do not eliminate repeated searching. Halving the round count does not guarantee twice the speed. Comparison and movement costs also matter.

When it is useful

Use it to learn how one scan can find two answers, or for very small arrays. Cocktail shaker sort is different: it moves values through neighboring comparisons.

Details that make the procedure work

The ordinary version is unstable because distant swaps can reverse equal records. Equal values need consistent handling, although an all-equal region needs no movement. Pairwise min-max searching can reduce comparison constants. The critical implementation detail remains updating a maximum moved by the first swap.

References [\[100\]](#)

64 Gnome sort

Type Algorithm Aliases or related names Garden gnome sort; originally also stupid sort, an ambiguous name

First understand the small actions

A position marker tells you which slot to examine. Moving the marker does not move a value.

After moving a small value left, check it against its new left neighbor. That neighbor might still be larger.

The idea in plain words

Gnome sort checks the current value and its predecessor. If they are ordered, walk right. If the left value is larger, exchange them and step left. A small item can therefore keep passing larger neighbors until it fits. Then the walk proceeds right again. This is insertion by neighboring swaps.

Carry it out one action at a time

1. Begin at slot two. A single first item needs no internal comparison, so it is the initial one-item ordered region.

2. If the predecessor is no greater than the current value, move the marker one slot right. Passing the array's end means completion.
3. If the predecessor is greater, swap the two values and move the marker one slot left. The smaller value just moved left now needs checking against an earlier neighbor.
4. At slot one there is no predecessor, so move right again. Continue applying these local rules. Equality permits forward movement without a swap.

Walk through the numbers

1. For [4,1,3,2], the marker starts at the second-slot 1. Swap 4,1: [1,4,3,2]. Step left to slot one, then right. The pair 1≤4 allows movement to slot three.
2. Swap 4,3: [1,3,4,2]. Step back to slot two. Check 1≤3, then 3≤4; both allow progress to slot four.
3. Swap 4,2: [1,3,2,4]. Step back to slot three, where 3>2. Swap again: [1,2,3,4], then step back to slot two.
4. The pairs 1≤2, 2≤3 and 3≤4 now permit successive rightward steps. The marker passes the end. The 2 found its place through two exchanges, with every original value preserved.

How much work and room it needs

Ordered input needs only a rightward walk, so best time is $\Theta(n)$, where n counts items. Disordered input can require many leftward crossings, giving typical and worst time $\Theta(n^2)$. Extra space is $O(1)$: a position marker and swap storage. Strictly inverted neighboring swaps preserve equal-record order, so it is stable.

Why it is fast or slow

It repairs local mistakes rather than blindly repeating whole passes. However, crossing every larger neighbor still costs a swap, so long misplaced journeys remain expensive.

When it is useful

Use it for teaching, tiny inputs or highly ordered data. Array insertion sort can shift a sequence with less overhead.

Details that make the procedure work

An optimized version remembers the furthest processed frontier. After inserting a value it returns directly there instead of walking across known order again. Both versions are gnome sort. The old name stupid sort is also used for Bogosort, so the name alone is not enough to identify the procedure.

References [\[101\]](#)

65 Bubble sort

Type Algorithm Aliases or related names Sinking sort; exchange sort is an ambiguous name

First understand the small actions

Neighbors occupy consecutive slots. A pass walks across a region and checks every neighboring pair in it.

One neighboring swap moves each involved value only one slot. Leave equal records alone if their original order matters.

The idea in plain words

Walk from left to right, swapping a pair whenever its left value is larger. A large value moved right becomes part of the next comparison, so it can keep traveling. By the end of a complete pass, the region's maximum has reached the right boundary. Leave it there and repeat on the shorter region.

Carry it out one action at a time

1. Begin with the leftmost neighboring pair. Swap if the left value is greater; otherwise keep both values where they are. This puts the pair's smaller value on the left.
2. Move the comparison one slot right. The previous right slot is now the new left slot, so a large value just moved right is examined again.
3. Continue to the current right boundary. The value carried along by these comparisons is the largest encountered, so it finishes at the end. Exclude that slot from the next pass.
4. Remember whether any swap occurred during the whole pass. No swaps means every neighboring relationship was checked and is correct. Otherwise return to the left and scan the shorter unfinished region. A one-item region needs nothing further.

Walk through the numbers

1. Start with [4,1,3,2]. Compare 4 and 1. Swap them: [1,4,3,2].
2. The next pair is the current 4 and 3. Swap: [1,3,4,2]. Then swap 4 and 2: [1,3,2,4]. The 4 traveled three slots and is now final.
3. Scan only the first three slots. Keep 1 and 3, then swap 3 and 2: [1,2,3,4]. The 3 finishes the shorter region.
4. Check the remaining pair 1,2. This complete pass makes no swaps, so stop. A single quiet comparison would prove only that one pair is ordered.

State trace C compare M move S ordered

Compare 4 and 1



4 moves one slot right



Keep moving 4 right



End of pass: 4 is final



Next pass swaps 3 and 2

**How much work and room it needs**

On ordered input, one pass takes roughly n comparisons, giving best time $\Theta(n)$. Here n is the item count. Typical and worst inputs can need roughly n passes of roughly n operations, giving $\Theta(n^2)$. Extra space is $O(1)$. Strict neighboring swaps preserve equal-record order, so the method is stable. A fixed-pass old version has quadratic best time too.

Why it is fast or slow

Large values can travel far right in one pass, but a tiny value at the far right generally moves left only one slot per pass. The pattern $[2,3,\dots,n,1]$ is therefore still expensive.

When it is useful

Use it to understand scanning, swaps and a correct stopping condition. Insertion sort is generally a better practical small-array choice.

Details that make the procedure work

A last-swap-position variant shortens later passes further. Some sources call bubble sort exchange sort, but that name is ambiguous. The separate direct exchange entry in this book gives a different, explicit process.

References [\[102\]](#)

66 Cocktail shaker sort

Type Important variant Aliases or related names Bidirectional bubble sort; shaker sort; double-direction bubble sort

First understand the small actions

A forward pass checks neighboring pairs left to right; a backward pass checks them right to left. Both put the smaller value on the left.

Moving a boundary inward means leaving a finished end slot alone. No item is removed from the array.

The idea in plain words

Cocktail shaker sort makes bubble sort walk both ways. A forward pass carries the maximum to the right end. A backward pass carries the minimum to the left end. A tiny value stuck on the right can therefore travel several slots left during one backward trip.

Carry it out one action at a time

1. Scan every neighboring pair in the unfinished region from left to right. Swap only when the left value is larger. The region maximum finishes at the right end; move that boundary left one slot.
2. Scan the shortened region backward. Start with its rightmost pair and move the pair left one slot at a time. Still swap only an inverted pair. A small value moved left participates in the next comparison.
3. At the left end, the region minimum is now final. Move the left boundary right one slot. Continue these trips through the shrinking middle region.
4. A complete directional pass with no swaps checks every neighboring relationship in that region and allows termination. Also stop when the boundaries leave at most one unfinished slot.

Walk through the numbers

1. Start with $[2,3,4,1]$. Going forward, keep 2,3 and then 3,4. Swap the final 4,1: $[2,3,1,4]$. The 4 is final.
2. Going backward through the first three slots, swap 3,1: $[2,1,3,4]$. The 1 just moved left and will be examined again immediately.
3. Swap 2,1: $[1,2,3,4]$. In this one backward trip, the 1 traveled two slots. The left end is now final.
4. The middle is $[2,3]$. Its next forward scan makes no swaps, so stop. Scanning backward changes the inspection direction; it still builds ascending order.

State trace C compare M move S ordered

Forward: compare 4 and 1



4 is final at the right



Backward: 1 passes 3



1 is final at the left



How much work and room it needs

Ordered input is confirmed in one pass, giving best time $\Theta(n)$, where n is the item count. Typical and worst cases can still require quadratically many neighboring operations, giving $\Theta(n^2)$. $O(1)$ extra space means only fixed-size position markers and swap storage. Equal neighbors are not swapped, so this version is stable.

Why it is fast or slow

Two directions help certain movement patterns, but moving an item across many other items still requires many one-slot exchanges. The improvement does not remove quadratic growth.

When it is useful

Use it to study how traversal direction changes movement, or for very small arrays. Insertion sort and efficient general methods usually serve software sorting better.

Details that make the procedure work

Cocktail, shaker and bidirectional bubble are aliases of this method. A variant remembers the last swap in each direction and tightens its boundaries. Boundary updates must remain correct when direction changes.

References [\[103\]](#)

67 Odd-even transposition sort

Type Algorithm Aliases or related names Brick sort; odd-even sort (not odd-even mergesort)

First understand the small actions

Pairs processed at the same time must not share a slot. Otherwise two operations could compete for the same value.

Number positions from one. One phase checks 1–2, 3–4; the other checks 2–3, 4–5. Odd and even describe positions, not whether the values are odd or even.

The idea in plain words

Order one set of neighboring pairs, then shift the pairing by one slot and order the other set. Alternating these phases lets small values travel left and large values right. The pairs within a phase do not overlap, which makes simultaneous comparisons possible.

Carry it out one action at a time

1. Run the first phase, pairing slots from slot one. For each pair, swap if its left value is larger. An unmatched last slot waits.
2. Run the second phase, pairing from slot two. This checks neighboring boundaries omitted by the first phase.
3. Treat both phases as one round. Remember whether either phase exchanged anything. If so, run another complete round.
4. Stop only when a whole round makes no swaps. Together its phases covered every neighboring boundary. One quiet phase alone cannot certify the array.

Walk through the numbers

1. For [4,1,3,2], the first phase swaps 4,1 and 3,2: [1,4,2,3].
2. The second phase checks only the middle 4,2 and swaps: [1,2,4,3]. The 4 travels one more slot right.
3. In the next first phase, keep 1,2 but swap 4,3: [1,2,3,4]. The following second phase keeps 2,3.
4. That round contained a swap, so run another complete round. Its pairs 1,2; 3,4; and then 2,3 all remain unchanged. Now stop. In [2,1,4,3], the middle 1,4 is already ordered although the outer pairs are wrong, illustrating why one phase is insufficient.

How much work and room it needs

On one processor, the early-stop version has best time $\Theta(n)$ and typical/worst time $\Theta(n^2)$, where n counts items. Extra space is $O(1)$. Strict inverted-pair swaps make it stable. A fixed n -phase network performs $\Theta(n^2)$ total comparison work. Enough simultaneous comparators reduce elapsed depth to $\Theta(n)$ phases; depth is different from total work.

Why it is fast or slow

Independent pairs are convenient for parallel hardware. Values still travel only one slot per phase, so long journeys require many phases.

When it is useful

Use it for linear processor chains, simple sorting networks and parallel-algorithm teaching. Sequential software usually has more efficient options.

Details that make the procedure work

Brick sort is an alias. Odd-even transposition is not Batchers odd-even mergesort. The latter connects comparisons through a different merging network; a similar name does not imply identical steps or costs.

References [\[104\]](#) [\[99\]](#)

68 Direct exchange / interchange sort

Type Algorithm Aliases or related names All-pairs exchange sort; terminology varies across sources

First understand the small actions

Keeping a slot fixed means using the same position as your comparison target. A swap may change the value inside it.

Pairs need not be neighbors. Slot one can exchange with slot five while the intervening slots stay put.

The idea in plain words

Fix the leftmost unfinished slot. Whenever a later value is smaller, swap it into that slot immediately. Continue comparing with the newly received value. After all later slots have been examined, the target contains the region minimum. Unlike selection sort, this method moves every improving candidate instead of waiting until the search ends.

Carry it out one action at a time

1. Choose the first slot as the target. Begin comparisons at the following slot.
2. If the target value is larger than the later value, exchange them immediately. Use the new, smaller target value for the next comparison. Otherwise leave both values alone.
3. Move the comparison partner one slot right until every later slot has been visited. They now all contain values no smaller than the target, so the target is finished.
4. Move the target right and begin at its next slot. Leave the finished prefix unchanged. Processing the second-last target finishes the array.

Walk through the numbers

1. For [4,1,3,2], fix slot one and compare 4,1. Swap: [1,4,3,2]. The target now holds 1; its comparisons with 3 and 2 need no changes. Slot one is finished.
2. Fix slot two, holding 4. Compare with the later 3 and swap: [1,3,4,2]. The new 3 is temporary because slot four has not been examined.
3. Compare that current 3 with the later 2 and swap again: [1,2,4,3]. Only now does slot two finish.
4. Fix slot three and swap 4,3: [1,2,3,4]. The second round made two swaps. Ordinary selection sort would first locate its minimum 2 and then make only one.

How much work and room it needs

Each target compares with every later slot, giving $n(n-1)/2$ comparisons for n items. Best, typical and worst time are therefore $\Theta(n^2)$. Best-case swaps can be zero, but worst-case swaps can also be quadratic. A saved value and two position markers give $O(1)$ extra space. Distant exchanges make the ordinary method unstable.

Why it is fast or slow

It still searches full suffixes, and one destination can repeatedly receive smaller candidates. Ordered input saves movement but not the prescribed comparisons.

When it is useful

Use it to understand immediate candidate movement versus remembering a position, or to read older short-region sorting routines.

Details that make the procedure work

For $[2a, 2b, 1]$, exchanging the first and third slots gives $[1, 2b, 2a]$, changing equal-record order. Sources use exchange and interchange inconsistently: sometimes they mean bubble sort or an entire family. Here they refer specifically to the nonadjacent nested-search procedure described above.

References [\[105\]](#)

69 Rotation-based in-place merge sorting

Type Algorithm family Aliases or related names In-place merge; this section uses single-item insertion rotations

First understand the small actions

A rotation exchanges neighboring pieces while keeping the order inside each: $[4, 7 \mid 2]$ becomes $[2 \mid 4, 7]$. Here the right piece has just one item.

In-place means that we avoid a full second record array. Holding one displaced item is still allowed.

Before a join, the left and right pieces must each be ordered.

Notation $a[i]$ means the item in slot i of the row named a . Brackets select a position; they do not mean multiplication.

The idea in plain words

If the right front is smaller than the current left item, insert it there. Hold that right item, move the intervening items one slot right, then fill the opening. Repeating this action joins the two pieces without a full copy buffer. This elementary version can move many items repeatedly.

Carry it out one action at a time

1. Use the bottom-up schedule: first ordered pairs, then joins of size-two pieces, then size four, and so on. For one join, let i identify the next left item, j the first remaining right item, and end the position just after both pieces. Positions are zero-based.
2. Continue while $i < j$ and $j < \text{end}$. If $a[i] \leq a[j]$, leave the left item where it is and advance i . Every remaining right item is at least its current front.
3. Otherwise save $x = a[j]$. Starting at $j-1$ and moving backward to i , move each item one slot right. Backward movement preserves values that have not yet been read. Write x into position i .
4. Advance both i and j . The inserted item is now confirmed for this range; the remaining left piece and the right boundary both shifted right.
5. Stop when one piece is exhausted. Its ordered counterpart needs no rearrangement. Repeat joins at larger sizes until the whole input is ordered.

Walk through the numbers

1. Begin with $[4, 1, 3, 2]$. For the first pair, hold 1, shift 4 right, and insert 1. Do the same with 3 and 2. The result is $[1, 4 \mid 2, 3]$.
2. For the final join, $i=0$, $j=2$, $\text{end}=4$. Keep 1 because $1 \leq 2$ and advance i to 1. Now $4 > 2$: hold the 2, move 4 from position 1 to 2, then write 2 at position 1. The row is $[1, 2, 4, 3]$, with $i=2$ and $j=3$.
3. Hold 3, shift 4 right, and put 3 at position 2. This gives $[1, 2, 3, 4]$. Now $i=j$, so the remaining left piece has ended. Every held item returned to the row.

How much work and room it needs

Let n count records. Without a skip check, favorable joins still scan at about $\log_2 n$ size levels: best $\Theta(n \log n)$. Checking whether the left last item already precedes the right first item can give $\Theta(n)$ on ordered input. Repeated long shifts allow typical and worst $\Theta(n^2)$ movement. Iterative size control needs $O(1)$ extra space; recursive control adds $O(\log n)$ stack space. The method is stable because equal left items remain first.

Why it is fast or slow

Saving a copy array trades memory for movement. Binary search can reduce position-finding comparisons, but cannot remove the shifts needed to make room.

When it is useful

Useful for small ranges with little memory and a stability requirement.

Details that make the procedure work

Do not apply the guarantees of sophisticated in-place merging to this single-item version. A general two-block rotation can use three reversals: reverse each piece, then their union; the pieces change places and recover their internal order.

References [\[106\]](#) [\[107\]](#)

70 Pancake sort

Type Algorithm Aliases or related names [Sorting by prefix reversal](#)

First understand the small actions

A prefix starts at slot one. In $[3,1,4,2]$, the first-three prefix is $[3,1,4]$.

Reversing a segment reverses its entire order. Swap its endpoints, then the next inward pair, and continue to the middle.

The idea in plain words

Pancake sort allows only prefix reversals, as though flipping the top of a pancake stack. To finish the current maximum, first reverse through its position to bring it to the front. Then reverse the whole unfinished prefix to carry it to the end. Each round finishes one large value and leaves a shorter prefix.

Carry it out one action at a time

1. Search the current prefix for its maximum. Remember the first value, update the remembered position when a larger value appears, and examine the whole prefix. If the maximum already occupies its end, skip this round's flips.
2. Otherwise reverse the prefix ending at the maximum. A value at that segment's end becomes its first value. Skip this flip if the maximum is already first.
3. Reverse the whole unfinished prefix. The maximum at the front becomes its final rightmost value.
4. Reduce the prefix length by one. Exclude the finished suffix from future flips. A final single-item prefix needs no movement.

Walk through the numbers

1. For $[3,1,4,2]$, maximum 4 is in slot three. Reverse $[3,1,4]$: swap its endpoints 3 and 4, leaving middle 1 unchanged. The array becomes $[4,1,3,2]$.
2. Reverse all four. Swap outer 4,2 to get the intermediate $[2,1,3,4]$, then inner 1,3 to complete $[2,3,1,4]$. The 4 is final.

3. In the first three slots, maximum 3 is second. Reverse the first two: [3,2,1,4]. Reverse the first three by swapping 3,1: [1,2,3,4]. The 3 is final.
4. The remaining prefix [1,2] already ends in its maximum 2, so skip flips. Only 1 remains. Every reversal stayed within the unfinished prefix and protected the final 4.

State trace C compare M move S ordered

Maximum 4 is in slot three



Flip first three: 4 reaches front



Flip four: 4 is final



Flip first two: 3 reaches front



Flip first three: 3 is final



How much work and room it needs

Each round searches a prefix of length n , $n-1$, and so on. For n items, this basic version therefore has $\Theta(n^2)$ best, typical and worst time. There are $O(n)$ flips, at most two per placement, but reversing length m takes about $m/2$ swaps. A long flip is not one computer operation. In-place reversal gives $O(1)$ extra space.

Why it is fast or slow

A flip changes many positions, but those values must actually move. The method is interesting because its allowed movement is restricted to prefixes.

When it is useful

Use it for the pancake problem, prefix-reversal constraints and teaching. General arrays without this restriction usually have faster choices.

Details that make the procedure work

This greedy procedure guarantees order, not the minimum number of flips. It is normally unstable because reversals can reverse equal records. Burnt pancakes additionally have orientations, requiring each included sign to change during a flip; that variation needs an extra placement rule.

References [108]

71 Burnt pancake sort

Type Important variant Aliases or related names Signed prefix-reversal sorting

First understand the small actions

Here size and orientation are separate. +3 is a size-three pancake facing correctly; -3 is the same size upside down. It is not ordinary numeric order in which negative three is smaller than positive one.

Flipping the first k items reverses their order and changes every sign. Flipping one item changes its orientation without moving its position.

The idea in plain words

Only a whole beginning stack can be flipped. The goal is increasing sizes with every sign positive. Each round sends the largest unfinished pancake to the end of the unfinished region with the correct orientation. Shorten that region and leave the completed large pancakes alone.

Carry it out one action at a time

1. Search the unfinished beginning for the greatest size, ignoring signs. If it is already at that region's end and positive, mark it finished.
2. Otherwise bring it to the first slot. If it is in slot k , flip the first k items: reverse their order and invert all their signs. Skip this move if it is already first.
3. The next flip will send it to the region's end and invert its sign again. Therefore it must be negative before that flip. If it is positive at the front, flip the first item alone.
4. Flip the whole unfinished beginning. The largest pancake arrives at its final end slot and becomes positive. Reduce the unfinished length by one and repeat. Check the last single item too: flip if negative; leave if positive.

Walk through the numbers

1. Start with $[-3, +1, -2]$, aiming for $[+1, +2, +3]$. Greatest size three is already first and negative, so flip all three.
2. Reverse the positions: -2 comes first, $+1$ remains in the middle, and -3 goes last. Invert all signs to obtain $[+2, -1, +3]$. The final $+3$ is complete.
3. Only $[+2, -1]$ remains unfinished. Greatest size two is first but positive. A direct two-item flip would send it to the end negative. First flip one item: $[-2, -1, +3]$.
4. Flip the first two, reversing their positions and signs: $[+1, +2, +3]$. Two is complete. The remaining $+1$ already has the right orientation, so finish.
5. If the final state were $[-1, +2, +3]$, one more single-item flip would still be required. Correct size order alone is insufficient.

How much work and room it needs

The basic method scans each unfinished region to find the largest size. Region lengths are $n, n-1, \dots$, so best, usual, and worst time are $\Theta(n^2)$, where n counts pancakes and Θ means the same growth order. Each placement uses at most three flips, making $O(n)$ flips overall. However, a long flip exchanges many records and changes many signs, so element work can still be $\Theta(n^2)$. A fixed number of temporary values gives $O(1)$ extra space. O states an upper bound; it does not make a long flip free.

Why it is fast or slow

A flip changes both position and direction. Preparing the front sign makes the final placement correct in both respects. A one-item flip repairs direction alone.

When it is useful

Use it to learn oriented arrangements and prefix reversals.

Details that make the procedure work

This workable greedy method does not minimize flips. Equal-size record identities can reverse, so it is normally unstable. Comparisons use absolute size; completion also checks positive signs.

References [108] [109]

72 Stooge sort

Type Playful method Aliases or related names Overlapping two-thirds recursive sort (descriptive name)

First understand the small actions

Recursion means using the same instructions on a shorter segment, then returning to the larger task where you paused. Remember the segment and the next instruction.

One item needs no sorting. Two items need one comparison and possibly a swap. These small cases are where the process reaches concrete actions.

Overlapping segments share slots. Changes made by a later task can disturb positions handled by an earlier one.

The idea in plain words

Stooge sort compares a segment's endpoints, then sorts its first two thirds, its last two thirds, and its first two thirds again. Each smaller sorting task follows exactly the same instructions until it reaches one or two items. The overlap helps carry values toward the correct ends, but it also creates enormous repetition.

Carry it out one action at a time

1. On receiving a segment, check its length. An empty or one-item segment returns unchanged. Otherwise compare the first and last values and swap if the first is larger. For two items, that completes the task.
2. For a longer segment, divide its length by three and round down to obtain t . The front task excludes the final t slots; the back task excludes the first t slots. Length six gives $t=2$; length four gives $t=1$.
3. Finish the front task first. If it has more than two items, it too compares endpoints and launches its own three shorter tasks. Complete those before returning to the paused outer task.
4. Next finish the back task in the same way. It pushes larger values right, but can bring a small value into the shared middle region.
5. Finally finish the front task again to move such values left. Only after all three tasks return is this segment complete. Initially ordered segments still execute this schedule.

Walk through the numbers

1. For $[2,3,1]$, compare and swap endpoints 2,1: $[1,3,2]$. Since $t=1$, the smaller tasks are the first pair and the last pair.
2. The first pair 1,3 stays unchanged. Swap the last pair 3,2: $[1,2,3]$. Check the first pair 1,2 again, without changing it. The three small tasks have now completed the larger one.
3. To see why the third task matters, use $[1,5,6,7,3,2]$. Endpoints $1 \leq 2$ stay put. Here $t=2$. The front-four task handles $[1,5,6,7]$; it is ordered but still completes its prescribed comparisons.
4. The back-four task handles $[6,7,3,2]$. Swapping its endpoints gives $[2,7,3,6]$. Its front-three task produces $[2,3,7,6]$; its back-three task produces $[2,3,6,7]$; its repeated front-three task leaves that order unchanged. The whole array is now $[1,5,2,3,6,7]$.

5. The back is ordered, but 5 still precedes smaller 2. The final front-four task turns [1,5,2,3] into [1,2,3,5], giving [1,2,3,5,6,7]. Omitting that third task would leave this input unordered.

How much work and room it needs

Every large task produces three tasks still about two thirds as long. Those tasks each produce three more. With n items, best, typical and worst time are $\Theta(n^{\log_{3/2} 3})$, approximately $\Theta(n^{2.71})$. The exponent describes length shrinking by $2/3$ while task count triples. It grows faster than quadratic work. About $\log n$ levels of paused tasks need $O(\log n)$ extra stack space. Distant endpoint swaps mean stability is not guaranteed.

Why it is fast or slow

The individual comparison is simple. The expensive part is sorting large overlapping regions again and again. Efficient divide-and-conquer methods split work into smaller pieces with less repetition; these three pieces remain relatively large.

When it is useful

Use it to study the actual order of recursive execution, overlap and growth. Small teaching inputs are enough; there is no practical reason to wait for a large run.

Details that make the procedure work

Some descriptions use the symmetric back-front-back schedule. This entry consistently uses front-back-front. The method does sort correctly. Correctness and efficiency are separate properties: an enormous amount of repeated work can still eventually produce the right answer.

References [\[110\]](#)

73 Slowsort

Type Playful method Aliases or related names Slowsort; not a synonym for Bogosort in this text

First understand the small actions

When a half is ordered, its last value is that half's maximum. The whole segment's maximum must be one of the two half-maxima.

A recursive call pauses the current task, completes the same procedure on a shorter segment, then returns to the paused instruction.

Excluding the last slot means protecting a finished value from later work. It does not delete that value.

The idea in plain words

Slowsort spends work ordering both halves, compares their maxima, and leaves the whole segment's maximum at the right end. Then it applies Slowsort again to almost the entire remaining region. This repeats much of the work just completed, but it eventually fixes the next maximum and continues until every value is placed.

Carry it out one action at a time

1. Return for an empty or single-item segment. Otherwise split near the middle, giving the left half the extra item for an odd length.
2. Complete the left-half task, then the right-half task. Each uses these same instructions. At length two, the halves are singletons: they return, their two maxima are compared and possibly exchanged, and the remaining singleton returns.
3. After both halves finish, compare their last values. If the left maximum is larger, swap those two slots. The right half's last slot is also the whole segment's last slot, and now contains the overall

maximum.

4. Protect that final slot and launch a new Slowsort task on the complete prefix before it. This region crosses the old half boundary and can still contain inversions.
5. That new task again sorts its halves, fixes its maximum, and sorts its shortened prefix. The outer task finishes only after these smaller tasks return. Ordered input follows the same calling schedule.

Walk through the numbers

1. Begin with [4,1,3,2], split into [4,1] and [3,2]. In the left task, singletons 4 and 1 return; comparing their maxima swaps them: [1,4,3,2]. Its remaining singleton 1 then returns.
2. In the right task, singletons 3 and 2 return. Swap them: [1,4,2,3], then finish its remaining singleton 2. The ordered halves are [1,4] and [2,3].
3. Their maxima are 4 and 3. Swap them: [1,3,2,4]. The final 4 is protected, but the earlier 3,2 remain inverted. Ordering both halves did not finish the whole array.
4. Launch the prefix-three task on [1,3,2]. Its left task [1,3] completes without a swap and still visits its remaining singleton 1. The right singleton 2 returns.
5. Compare this task's half-maxima 3 and 2, then swap: [1,2,3,4]. Its final 3 is finished, but the prescribed prefix-two task still runs: singletons 1 and 2 return, their comparison needs no change, then singleton 1 returns.
6. All paused tasks now complete. The outer task returns [1,2,3,4]. Returning does not move a value; it resumes the larger instruction after a smaller task finishes.

How much work and room it needs

Input order does not remove these calls, so best, typical and worst cases have the same kind of work growth. Let n be the length and $T(n)$ its work. The recurrence is $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + T(n-1) + \Theta(1)$. The first two terms order halves, the third reorders almost everything, and the last is fixed comparison work. The resulting class $n^{\Theta(\log n)}$ grows faster than any fixed polynomial power, but slower than ordinary c^n growth. Basic retained call frames require $O(n)$ extra space. It is normally unstable.

Why it is fast or slow

It pays to sort two halves just to finish one value, then pays again for nearly the same region. Splitting a task in half does not by itself make an algorithm efficient.

When it is useful

Use it to unfold recursive execution and compare work recurrences. It is a deterministic counterexample to efficient design, not a practical sorting choice.

Details that make the procedure work

Here $\log n$ describes repeated halving, and the exponent itself grows with input size rather than staying at two or three. The recurrence and growth class avoid claiming an unsupported exact exponent. Ordinary calls retain a long $n-1$ chain; deliberate tail-call elimination can change space usage. Slowsort is distinct from random-shuffling Bogosort.

References [\[111\]](#)

6 Special models and playful methods

Beads, rods, and timers rely on models outside ordinary comparison sorting. Permutation enumeration, random shuffling, and recursive retries usually become very slow on ordinary computers. Names do not relax correctness or record-preservation requirements. Finite exhaustive searches are distinguished from random methods with no finite worst-case bound.

74 Bead / gravity sort

Type Physical model Aliases or related names Abacus sort; gravity sort

First understand the small actions

A value is a number of beads: 3 means three beads. This representation directly supports nonnegative whole numbers.

Rows go across; columns go down. A bead can fall down its own column, but cannot move into another column. An empty cell contributes zero.

The idea in plain words

Write each value as a left-aligned row of beads. Let the beads in every column settle at the bottom. Upper rows now have fewer beads and lower rows have more. Counting each row from top to bottom gives ascending values. The beads rebuild the values; a complete original record does not travel intact through the process.

Carry it out one action at a time

1. Find the largest value, called M. Make M columns and one row per input value. In the row for x, fill its first x cells with beads.
2. For the first column, count its beads. Clear that column, then fill that many cells upward from the bottom. This simulates falling without changing the column total.
3. Do the same for every remaining column. Each column settles independently.
4. Count the beads in the top row and write that count. Continue down to the bottom row. An empty row gives zero.

Walk through the numbers

1. Start with [3,1,4,2]. Four is largest, so each row has four cells. Write 1 for a bead and 0 for an empty cell: the rows are 1110, 1000, 1111, 1100.
2. Column one has four beads, so all four cells stay filled. Column two has three, so its top cell becomes empty and its bottom three cells are filled. The first two columns now read 10, 11, 11, 11 down the rows.
3. Column three has two beads and fills only its bottom two cells. Column four has one and fills only its bottom cell. The complete rows are now 1000, 1100, 1110, 1111.
4. Their counts give [1,2,3,4]. If an upper row has a bead in a column, every cell below it in that column also has a bead. Therefore the row counts cannot decrease as we move downward.

State trace C compare M move S ordered

Input: four nonnegative integers



Each cell is a row: 1 bead, 0 empty



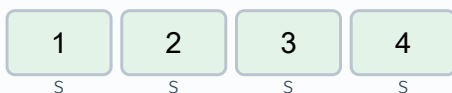
Count beads by column



Four rows after each column settles



Count each row: ascending output



How much work and room it needs

A full-grid simulation visits or fills the cells, so large values increase work even when there are few inputs. Let n be the item count and M the largest value. Time is $\Theta(nM+n)$, grid space $\Theta(nM)$, and output space $O(n)$. Θ means the same growth order; O gives an upper bound. Best, usual, and worst inputs have the same order for fixed n and M . A bead-by-bead model must also count the total number of beads.

Why it is fast or slow

Physical falling can happen in parallel because the apparatus supplies many working units. Large values still need more beads, width, or simulation work.

When it is useful

Use tiny whole-number examples to study physical computation. Counting or radix methods are normally more useful in software.

Details that make the procedure work

Negative and fractional values need extra encoding. Original record identities and stable equal-record order are not naturally preserved. The paper describes several models with different costs. A row such as 1110 is a picture of four cells, not the number eleven hundred and ten.

References [\[112\]](#) [\[113\]](#)

75 Spaghetti sort

Type Physical model Aliases or related names Spaghetti Analog Gadget (SAG)

First understand the small actions

Use length to represent a value: a three-unit rod represents 3. Assume positive lengths that can be made and distinguished accurately.

The bottom ends must share a flat surface. Raising a short rod would make its top falsely suggest a larger value.

The idea in plain words

Make one rod for each value, like a handful of uncooked spaghetti. With the bottoms aligned, the highest top belongs to the longest rod. Remove that rod, then the longest remaining rod. The readings arrive largest first. Reverse them to obtain ascending order.

Carry it out one action at a time

1. Choose a length unit and prepare the rods. Add labels if the original records have identities that must survive.
2. Stand the rods upright with every bottom on the same flat surface. A lifted or tilted rod can give a misleading height.
3. Find the highest top, remove its rod, and record its length. That rod represents the largest remaining input.
4. Continue until no rods remain. Reverse the recorded sequence, or write each removal into the last available output slot and work backward.

Walk through the numbers

1. For [3,1,4,2], prepare rods of lengths three, one, four, and two units. Their original positions do not affect their lengths.
2. After aligning the bottoms, the four-unit rod is highest. Remove it: the partial reading is [4], and the remaining lengths are 3,1,2.
3. Three is highest next, giving [4,3]. Then remove two, giving [4,3,2], and finally one, giving [4,3,2,1].
4. To reverse the readings, exchange the outside values 4 and 1: [1,3,2,4]. Exchange the middle values 3 and 2: [1,2,3,4].
5. Each removal is justified because all remaining bottoms are still aligned. Highest top then means greatest length. Without that alignment, the argument fails.

How much work and room it needs

Suppose an ideal model counts preparing a rod, aligning it, and selecting the highest top as unit-cost actions. Preparation and reading each take about n actions: $\Theta(n)$ work and $\Theta(n)$ rods, where n is the item count and Θ means the same growth order. Real work also depends on length, material, precision, and handling. A computer that scans every remaining value to find each maximum makes $n+(n-1)+\dots$ checks and takes $\Theta(n^2)$ time.

Why it is fast or slow

The apparatus exposes many lengths at once and supplies a special selection operation. Making the apparatus and reading its answer still require work. Nearly equal lengths are especially difficult to measure.

When it is useful

Use it to discuss analog computation and why speed claims need an operation model. It is not a free linear-time operation available to ordinary software.

Details that make the procedure work

Equal rods have no automatic stable order. Labels and an original-order rule are needed for ties. Negative values need an offset, and fractions need enough measurement precision. An aligned physical state is not the same as a zero-cost completed sorting job.

References [\[114\]](#)

76 Sleep sort

Type Playful method Aliases or related names Delay-based output (descriptive name)

First understand the small actions

A waiting task becomes ready after its delay. Ready does not mean it immediately receives processor time or access to output.

Several tasks can exist concurrently without starting or printing at exactly their planned times.

The idea in plain words

Create a waiting task for each value. Small values wait briefly; large values wait longer. The hope is that printing in wake-up order gives ascending values. That hope requires timing conditions real systems do not generally promise, so this is a demonstration trick rather than a reliable general sort.

Carry it out one action at a time

1. Restrict values to nonnegative numbers and choose a scale, such as one second per value unit. All delays should refer to a common starting time.
2. Arrange one task per value. For x , the planned print time is the common start plus x times the scale. Separate task-creation times are not automatically a common start.
3. Collect the values actually printed and wait for every task to finish. Actual order depends on when each task can run and write.
4. If actual output order differs from planned time order, the result is wrong. A normal sleep or timer call does not by itself provide a sorting guarantee.

Walk through the numbers

1. Take $[3,1,2]$. Assume a common start at zero, one second per unit, and no delays. One prints at one second, two at two seconds, and three at three seconds: $[1,2,3]$.
2. No pair of values was compared and no array positions were exchanged. Values were encoded as planned waiting times.
3. Now take $[1,2]$. The task for one becomes ready at one second, but a busy processor delays its output until 2.5 seconds. The task for two prints at two seconds. The collected result is $[2,1]$.
4. Starting the tasks at different times creates another problem. A task for three can start much earlier than a task for one. A shared starting time fixes that issue, but does not remove later scheduling delays.

How much work and room it needs

In an ideal delay-free model, launching and collecting n tasks requires $\Theta(n)$ work. Elapsed waiting is roughly the scale times the largest value M . Here n counts inputs, M is their maximum, and Θ means the same growth order. Task resources are usually $O(n)$, an upper bound proportional to n . A maximum represented with b bits can approach 2^b , producing enormous waiting. Real scheduling generally provides neither guaranteed output order nor a universal delay bound, so ordinary best, usual, and worst sorting times cannot be ranked using n alone.

Why it is fast or slow

It replaces numeric size with real waiting instead of removing that cost. Shortening the scale increases sensitivity to timer resolution and delays.

When it is useful

Use it to learn timers, concurrency, and the difference between becoming ready and actually running. Do not use it when order must be correct.

Details that make the procedure work

Equal records have uncertain order. Offsets for negatives and finer scales for fractions do not fix scheduling. A dependable ordered event queue may already use a heap or another ordering structure whose work must be counted.

References [\[115\]](#)

77 Exhaustive permutation sort

Type Playful method Aliases or related names Permutation sort; sometimes deterministic Bogosort

First understand the small actions

A permutation is one arrangement of the same records. No record is added or lost: $[3,1,2]$ and $[1,3,2]$ are different arrangements.

Choose a record for the first slot, then one of the remaining records for the second. Completing all choices gives a candidate. After failure, return to a choice and try another option.

The idea in plain words

Try possible arrangements in a fixed, systematic order. Check each complete candidate for ascending order and stop when one passes. The method eventually finds an answer, but failed candidates provide almost no useful guidance for the next attempt.

Carry it out one action at a time

1. Keep a current candidate and marks showing which records are used. Try the options for its first slot in input order.
2. After choosing a record, mark it used. Try only unused records for the next slot. Continue until all slots are filled.
3. Check neighboring values from left to right. If each value is no greater than its right neighbor, return the candidate. A larger left value makes it fail.
4. After failure, return to the latest choice that has another option. Release the records used after that choice, select the next option, and fill the later slots again. If a choice has no options left, move back farther. This avoids missing arrangements.

Walk through the numbers

1. Start with $[3,1,2]$, trying records in input order. Put 3 first, then 1, then the only remaining record, 2. Candidate $[3,1,2]$ fails because $3 > 1$.

2. Return to the second slot. Release 1 and try 2 there, leaving 1 for the last slot: [3,2,1]. It fails too. All candidates beginning with 3 have now been tried.
3. Return to the first slot and choose 1. The remaining records are 3 and 2, so the next candidate is [1,3,2]. It fails because $3 > 2$.
4. Change the second choice to 2 and put 3 last: [1,2,3]. Both neighboring pairs pass, so stop. The two arrangements beginning with 2 need not be generated.

How much work and room it needs

With n different records, there are $n!$ arrangements. The exclamation mark means $n \times (n-1) \times \dots \times 1$. With a complete $n-1$ -pair test for each candidate, best time is $\Theta(n)$, and worst time $\Theta(n \cdot n!)$; Θ means the same growth order. If the target position is roughly uniform, usual waiting is also factorial-scale. Keeping only the current choices needs $O(n)$ extra space, a linear upper bound. Stopping a check at its first error can save checks, but does not remove the huge candidate count.

Why it is fast or slow

Adding one distinct record multiplies the possibilities by another growing factor. Guaranteed eventual success does not imply practical speed.

When it is useful

Use tiny examples to learn exhaustive search or to check another method's answer.

Details that make the procedure work

Equal records may exchange identities. Comparing both value and original position defines a stable target. This permutation enumeration differs from rank/enumeration sort, which calculates positions from pairwise comparisons.

References [\[116\]](#)

78 Bogosort

Type Playful method Aliases or related names Stupid sort; monkey sort; random permutation sort

First understand the small actions

A neighboring pair is in order when its left value is no greater than its right value. All neighboring pairs must pass.

A shuffle rearranges all records without losing any. A uniform shuffle gives every permutation the same chance; a single arbitrary swap does not do that.

The idea in plain words

Check the current arrangement. If it is ordered, finish. Otherwise shuffle the whole array and check again. It throws away useful order and hopes to land on the answer by chance. The final check ensures correctness; the difficulty is how long success takes.

Carry it out one action at a time

1. Check neighboring pairs from left to right. Stop if every pair has its smaller or equal value on the left.
2. After failure, shuffle uniformly. One method starts at the last slot: choose a slot uniformly from the first through the current slot and exchange their records. Move the current slot one place left and repeat through the second slot.
3. Check the complete array again. No prefix or suffix remains protected after a shuffle.

4. Repeat after every failure. A demonstration may impose an attempt limit, but reaching it means unfinished, not sorted.

Walk through the numbers

1. Start with [3,1,2]. The pair 3,1 fails. One possible shuffle exchanges the last record 2 with the first record 3, giving [2,1,3]. It then exchanges the first two slots, giving [1,2,3].
2. Another run might instead produce [2,3,1], which fails at $3 > 1$, then shuffle back to [3,1,2]. Apparent progress can disappear.
3. Only [1,2,3], followed by checks of $1 \leq 2$ and $2 \leq 3$, makes this example stop. These are possible random outcomes, not a script predicting future attempts.
4. Three distinct values have six arrangements. A uniform shuffle succeeds with probability one sixth. An average of six attempts is not a promise to finish on attempt six, or even sixty.

How much work and room it needs

For sorted input, one check gives best time $\Theta(n)$, where n counts records and Θ means the same growth order. Distinct records have $n!$ permutations, with $!$ meaning multiplication from n down to 1. Independent uniform trials need $n!$ attempts on average. Linear-cost shuffling and checking give expected time $\Theta(n \cdot n!)$. In-place shuffling needs $O(1)$ extra space: a fixed number of temporary values. There is no finite worst-case completion bound. Repeated values reduce the possibilities: [1,1,2] has three value arrangements, not six.

Why it is fast or slow

The comparisons supply direction, but the method ignores it and draws another arrangement. Probability-one eventual termination does not supply a finite guaranteed deadline.

When it is useful

Use very small examples to explain averages, probability, and worst cases.

Details that make the procedure work

It is normally unstable. If equal-value counts are $c_1, c_2, \dots$, the number of value arrangements is $D = n! / (c_1! c_2! \dots)$, giving success probability $1/D$. The randomness assumptions matter. Randomly swapping just two slots defines Bozosort. The alias stupid sort can also mean gnome sort.

References [\[117\]](#) [\[118\]](#)

79 Bozosort

Type Playful method Aliases or related names Random-transposition sort

First understand the small actions

To exchange two records safely, save one, move the other into its slot, then put the saved record into the other slot. Both records survive.

Random position choices ignore the values. After the exchange, the whole neighboring-pair test must still pass, not merely the changed pair.

The idea in plain words

Choose two random positions, exchange their records, and check whether the array is ordered. Unlike a whole-array shuffle, this changes only two positions at a time. Because the choices ignore values, an exchange may help or undo an earlier improvement.

Carry it out one action at a time

1. First check every neighboring pair. Stop if they are all ordered. Otherwise prepare a random exchange.
2. Here choose two different positions uniformly and exchange their complete records. Another version allows choosing the same position twice, creating an idle attempt.
3. Start the order check again from the beginning. Every pair must pass. Any larger left value means another attempt is needed.
4. Choose another random pair. All positions remain eligible; an accidentally correct first slot is not permanently finished.

Walk through the numbers

1. Start with [3,1,2]. The check fails at $3 > 1$. Suppose the chosen positions are one and two. Save 3, move 1 into the first slot, and place 3 in the second: [1,3,2].
2. Check again: $1 \leq 3$ passes, but $3 > 2$ fails. Suppose positions two and three are chosen next. Their exchange gives [1,2,3]. Every neighboring pair passes, so finish.
3. A different run can reach [1,3,2], then choose positions one and two again and return to [3,1,2]. The first slot looked finished but had no protection.
4. It is also possible to keep exchanging the same pair and travel back and forth. Each attempt follows the rule without guaranteeing success within a fixed number of attempts.

How much work and room it needs

Already ordered input needs one scan: best time $\Theta(n)$, where n is the record count and Θ means the same growth order. Each swap uses a fixed number of actions and $O(1)$ temporary space. Waiting is the main cost. Standard random-transposition models have factorial-scale waiting; exact expectations depend on pair selection, starting arrangement, and checking rules. A complete check adds $\Theta(n)$ work per attempt. Stopping at the first error can reduce that check. There is no finite worst-case completion bound, and equal records are normally not stable.

Why it is fast or slow

A cheap swap does not necessarily point toward the answer. It can repair one relation while breaking another, or the next attempt can reverse it.

When it is useful

Use three or four records to compare random pair swaps with complete shuffles.

Details that make the procedure work

Exchanging distinct positions alternates permutation parity but does not prevent eventual reachability. Allowing identical choices adds idle steps. Ideal randomness can give probability-one eventual success without a fixed deadline.

References [\[119\]](#) [\[118\]](#)

80 Bogobogosort

Type Playful method Aliases or related names Recursive-check Bogobogosort; definitions vary

First understand the small actions

A copy is separate data. Changing it does not automatically change the original array. Keep track of both.

A prefix is the beginning part of an array. Once that part is ascending, its last value is its maximum. The full array is ordered if the final value is at least that maximum.

Recursion asks the same method to handle a smaller input. The larger task waits, then resumes where it stopped. Zero or one item returns immediately.

The idea in plain words

This method deliberately makes checking expensive. Instead of checking neighboring pairs, it copies the original, sorts the beginning of the copy using this same method, and tries to obtain an ordered copy. It then compares that copy with the original. If they differ, it discards the useful ordered copy and shuffles the original again.

Carry it out one action at a time

1. Zero or one item needs no work. Otherwise, to check original array A, make an independent copy C. Leave A untouched during this check.
2. Ask this method to sort the first $n-1$ items of C, where n is the current item count. That smaller task also copies, checks, and sometimes shuffles. Wait until it finishes.
3. The prefix of C is now ordered. Compare its last value with the final value of C. If the final value is smaller, shuffle the whole copy C, sort its prefix again, and repeat this comparison until C is ordered.
4. Compare ordered C with A, slot by slot. Equality everywhere means A was already ordered and the check passes. Any difference makes it fail.
5. After failure, discard the check result, shuffle A, and restart with a fresh copy. Do not replace A with ordered C: that would define a different, more sensible method.

Walk through the numbers

1. Let $A=[2,1,3]$, and copy it to $C=[2,1,3]$. Sorting the first two items pauses the three-item task and starts a two-item task on $P=[2,1]$.
2. That task copies P to $Q=[2,1]$. Its one-item prefix [2] returns immediately. The final 1 is smaller than 2, so Q is not ordered. Suppose shuffling Q gives [1,2]. Its prefix [1] returns, and $2 \geq 1$ makes Q ordered.
3. But $Q=[1,2]$ differs from $P=[2,1]$, so the two-item check fails. Suppose P now shuffles to [1,2]. A new copy, one-item return, final comparison, and equality test all pass. The two-item task finally returns [1,2].
4. Resume the three-item task: $C=[1,2,3]$. Since $3 \geq 2$, C is ordered, but it differs from $A=[2,1,3]$. The outer check fails. A has not yet changed.
5. Suppose A shuffles to [1,3,2]. Its copy has an ordered prefix [1,3], but final $2 < 3$. Shuffle that copy. A possible result [2,1,3] requires the two-item work above, eventually giving $C=[1,2,3]$. C still differs from A, so the outer check fails again.
6. Suppose A next shuffles to [1,2,3]. Its copy passes the prefix check, the final comparison, and equality with A. Only now does the outer task finish. These chosen shuffle outcomes illustrate a possible path, not a fixed waiting time.

How much work and room it needs

Even already ordered input creates and checks copies of lengths $n, n-1, \dots$. Its best no-shuffle path takes $\Theta(n^2)$ work; n counts items and Θ means the same growth order. Waiting parent tasks retain their copies, so a basic implementation needs $O(n^2)$ live extra space. Random retries have no finite worst-case completion bound. Definitions vary, and there is no universal established exact expected-time formula.

Why it is fast or slow

It replaces a short check with nested sorts and can discard their completed work. Inner and outer retries compound the waste.

When it is useful

Use only tiny examples to study recursive waiting and copies.

Details that make the procedure work

This version uses at least, rather than strictly greater, to support duplicate maxima. Shuffling can reorder equal record identities, so stability is not assured. The original author gives complexity guesses. Prefix-growing variants should not inherit this version's precise claims.

References [\[120\]](#) [\[121\]](#)

Implementation checks and practice

1. Checking ascending order is not enough. Compare input and output multisets, including multiplicities. For records, compare identities as well as keys.
2. A stability test such as $[2a, 1, 2b, 1c]$ should produce $[1, 1c, 2a, 2b]$, where the unlabeled 1 is the original record. Adding original positions can stabilize an unstable method, but its costs must be counted.
3. Test empty and single-item inputs, sorted and reversed data, all-equal keys, many duplicates, negatives, extreme keys, and lengths that are not powers of two. Specialized methods also need domain, width, and sentinel checks.
4. After quicksort partitioning, verify region relations and shrinking recursive ranges. A Hoare boundary is not a final pivot position. A merge needs ordered input runs and an earlier-left tie rule. Heap extraction must restore the heap rules; bucket placement alone is not a completed sort.
5. Hold input size, key type, stability requirements, worker count, and included transfer costs fixed when measuring speed. Start each run from a copy of the same original data. Report repeated-run medians rather than comparing a later sorted input with an earlier random one.
6. Practice 1 Trace insertion, Lomuto quicksort, and merge sort on $[5, 2, 4, 1, 3]$. Practice 2 Apply LSD radix sort to $[31, 12, 21, 11]$ and explain why the tens pass must be stable. Practice 3 Label equal records and find a selection-sort stability counterexample. Practice 4 Verify all four-item zero-one inputs against the five-comparator network.

Practice hints

The LSD units pass gives $[31, 21, 11, 12]$; the tens pass gives $[11, 12, 21, 31]$. For selection sort, $[2a, 2b, 1]$ becomes $[1, 2b, 2a]$ after its first exchange. A four-wire zero-one network check covers $2^4=16$ inputs.

References

Sources checked on 5 October 2026. Bounds apply to the versions described. Original papers and author implementations supply advanced details. Titles and URLs are clickable; lesson citations link to these entries. Examples and diagrams were constructed for this manual.

[1] NIST DADS: Counting sort

<https://xlinux.nist.gov/dads/HTML/countingsort.html>

[2] NIST DADS: Pigeonhole sort

<https://xlinux.nist.gov/dads/HTML/pigeonholeSort.html>

[3] Sedgewick and Wayne: String sorts

<https://algs4.cs.princeton.edu/51radix/>

[4] Princeton LSD implementation

<https://algs4.cs.princeton.edu/51radix/LSD.java.html>

[5] Princeton MSD implementation

<https://algs4.cs.princeton.edu/51radix/MSD.java.html>

[6] McIlroy, Bostic, McIlroy: Engineering radix sort

https://www.usenix.org/publications/compsystems/1993/win_mcilroy.pdf

[7] NIST Radix Sort

<https://xlinux.nist.gov/dads/HTML/radixsort.html>

[8] Princeton AmericanFlag implementation

<https://algs4.cs.princeton.edu/51radix/AmericanFlag.java.html>

[9] NIST DADS: Bucket sort

<https://xlinux.nist.gov/dads/HTML/bucketsort.html>

[10] MIT 6.006: Linear-time sorting

<https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/resources/lecture-7-counting-sort-radix-sort-lower-bounds-for-sorting/>

[11] Karl-Dietrich Neubert: The FlashSort algorithm

<https://www.neubert.net/FSOIntro.html>

[12] Neubert: Flashsort implementations

<https://www.neubert.net/Flacodes/FLACodes.html>

[13] Sedgewick and Wayne: Tries

<https://algs4.cs.princeton.edu/52trie/>

[14] Princeton TrieST implementation

<https://algs4.cs.princeton.edu/52trie/TrieST.java.html>

[15] Sinha and Zobel: Cache-conscious sorting of large sets of strings with dynamic tries

<https://people.eng.unimelb.edu.au/jzobel/fulltext/acmjea04.pdf>

[16] Sinha and Zobel: Efficient trie-based sorting

<https://people.eng.unimelb.edu.au/jzobel/fulltext/acsc03sz.pdf>

[17] Boost.Sort: Spreadsort documentation

https://www.boost.org/doc/libs/1_69_0/libs/sort/doc/html/sort/single_thread/spreadsort.html

[18] Steven Ross: Original Spreadsort paper

https://www.boost.org/doc/libs/1_61_0/libs/sort/doc/papers/original_spreadsort06_2002.pdf

[19] Bentley & Sedgewick: Fast Algorithms for Sorting and Searching Strings

<https://sedgewick.io/wp-content/themes/sedgewick/papers/1997StringsSODA.pdf>

[20] Princeton: Quick3string implementation

<https://algs4.cs.princeton.edu/51radix/Quick3string.java.html>

[21] NIST: multikey quicksort aliases

<https://xlinux.nist.gov/dads/HTML/multikeyQuicksort.html>

[22] Munro and Wild: Nearly-optimal mergesorts

<https://www.wild-inter.net/publications/munro-wild-2018.pdf>

[23] Princeton: Mergesort

<https://algs4.cs.princeton.edu/22mergesort/>

[24] Tim Peters and contributors: CPython listsort notes

<https://github.com/python/cpython/blob/main/Objects/listsort.txt>

[25] Auger, Jugé, Nicaud, Pivoteau: On the worst-case complexity of TimSort

<https://arxiv.org/abs/1805.08612>

[26] CPython 3.10 classic collapse implementation

<https://github.com/python/cpython/blob/3.10/Objects/listobject.c>

[27] Munro and Wild: Nearly-optimal mergesorts

<https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2018.63>

[28] Authors' Powersort implementations

<https://github.com/sebawild/powersort>

[29] Gelling, Nebel, Smith, Wild: Multiway Powersort

<https://arxiv.org/abs/2209.06909>

[30] Munro and Wild: Nearly-optimal mergesorts, Peeksort algorithm

<https://drops.dagstuhl.de/opus/volltexte/2018/9526/pdf/LIPIcs-ESA-2018-63.pdf>

[31] Princeton: Elementary sorts

<https://algs4.cs.princeton.edu/21elementary/>

[32] Princeton Insertion implementation

<https://algs4.cs.princeton.edu/21elementary/Insertion.java.html>

[33] NIST DADS: Binary insertion sort

<https://xlinux.nist.gov/dads/HTML/binaryinsort.html>

[34] Bender, Farach-Colton, Mosteiro: Insertion sort is $O(n \log n)$

<https://arxiv.org/abs/cs/0407003>

[35] Aldous and Diaconis: Longest increasing subsequences, from patience sorting

<https://escholarship.org/uc/item/09j2v8d0>

[36] Aldous and Diaconis: Patience sorting paper

<https://www.stat.berkeley.edu/users/aldous/Research/OP/patience.pdf>

[37] Stanford Aldous and Diaconis research report

<https://statistics.stanford.edu/technical-reports/longest-increasing-subsequences-patience-sorting-baik-deift-johansson-theorem>

[38] Dijkstra: Smoothsort, an alternative for sorting in situ, EWD796a

<https://www.cs.utexas.edu/~EWD/transcriptions/EWD07xx/EWD796a.html>

[39] musl libc qsort: Smoothsort implementation

<https://git.musl-libc.org/cgit/musl/tree/src/stdlib/qsort.c>

[40] NIST DADS: Strand sort

<https://xlinux.nist.gov/dads/HTML/strandSort.html>

[41] Levcopoulos and Petersson: Adaptive heapsort

<https://www.sciencedirect.com/science/article/abs/pii/S0196677483710217>

[42] Goodrich: Cartesian-tree priority-queue sort lecture

<https://ics.uci.edu/~goodrich/teach/cs263/notes/09-Maxima.pdf>

[43] Merge.java

<https://algs4.cs.princeton.edu/22mergesort/Merge.java.html>

[44] Sedgewick & Wayne: MergeBU implementation

<https://algs4.cs.princeton.edu/22mergesort/MergeBU.java.html>

[45] MergeBU API and complexity

<https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/MergeBU.html>

[46] Sedgewick & Wayne: heaps and heapsort

<https://algs4.cs.princeton.edu/24pq/>

[47] Princeton: Heap implementation

<https://algs4.cs.princeton.edu/24pq/Heap.java.html>

[48] Edelkamp & Wegener: weak-heapsort analysis

<https://eccc.weizmann.ac.il/report/1999/028/download/>

[49] Edelkamp et al.: Weak heaps engineered

<https://www.sciencedirect.com/science/article/pii/S1570866713000592>

[50] Katajainen: weak-heap variants and applications

<https://hjemmesider.diku.dk/~jyrki/Myris/EEK2012bJ.html>

[51] Sahni: winner-tree sorting implementation

<https://www.cise.ufl.edu/~sahni/dsaaj/instruct/exer/c14/e8.htm>

[52] NIST: tournament sort and original report

<https://xlinux.nist.gov/dads/HTML/tournamentSort.html>

[53] Musser: original introsort paper

[https://onlinelibrary.wiley.com/doi/10.1002/\(SICI\)1097-024X\(199708\)27:8%3C983::AID-SPE117%3E3.0.CO;2-%23](https://onlinelibrary.wiley.com/doi/10.1002/(SICI)1097-024X(199708)27:8%3C983::AID-SPE117%3E3.0.CO;2-%23)

[54] Musser: generic algorithms and introsort

https://www.cs.rpi.edu/~musser/gp/index_1.html

[55] orlp: pdqsort design and guarantees

<https://github.com/orlp/pdqsort/blob/master/readme.md>

[56] orlp: pdqsort source

<https://github.com/orlp/pdqsort/blob/master/pdqsort.h>

[57] Lammich: verified introsort and pdqsort

<https://pmc.ncbi.nlm.nih.gov/articles/PMC7324064/>

[58] Moffat, Eddy & Petersson: Splaysort

[https://onlinelibrary.wiley.com/doi/10.1002/\(SICI\)1097-024X\(199607\)26:7%3C781::AID-SPE35%3E3.O.CO;2-B](https://onlinelibrary.wiley.com/doi/10.1002/(SICI)1097-024X(199607)26:7%3C781::AID-SPE35%3E3.O.CO;2-B)

[59] Sleator & Tarjan: self-adjusting trees

<https://www.cs.cmu.edu/~sleator/papers/self-adjusting.pdf>

[60] Carnegie Mellon: zig, zig-zig, zig-zag

<https://www.cs.cmu.edu/afs/cs.cmu.edu/academic/class/15451-f97/www/splay.html>

[61] Atkinson Sack Santoro Strothotte Min-Max Heaps and Generalized Priority Queues

<https://people.scs.carleton.ca/~santoro/Reports/MinMaxHeap.pdf>

[62] Grailsort author implementation

<https://github.com/Mrrl/GrailSort>

[63] WikiSort author: in-place blocks, buffers, tags

<https://github.com/BonzaiThePenguin/WikiSort/blob/master/Chapter%203.%20In-Place.md>

[64] WikiSort author implementation

<https://github.com/BonzaiThePenguin/WikiSort/blob/master/WikiSort.c>

[65] MIT OpenCourseWare: Quicksort lecture

<https://ocw.mit.edu/courses/6-046j-introduction-to-algorithms-sma-5503-fall-2005/resources/lecture-4-quicksort-randomized-algorithms/>

[66] Aumüller et al.: Lomuto-based partitioning analysis

<https://arxiv.org/abs/1810.12047>

[67] Princeton: original Hoare implementation

<https://algs4.cs.princeton.edu/23quicksort/QuickHoare.java.html>

[68] Sedgewick & Wayne: original Hoare exercise

<https://algs4.cs.princeton.edu/23quicksort/>

[69] Princeton: Quick3way implementation

<https://algs4.cs.princeton.edu/23quicksort/Quick3way.java.html>

[70] Wild: Quicksort with many equal keys

<https://arxiv.org/abs/1608.04906>

[71] Princeton: dual-pivot teaching implementation

<https://algs4.cs.princeton.edu/23quicksort/QuickDualPivot.java.html>

[72] Wild & Nebel: Yaroslavskiy analysis

<https://arxiv.org/abs/1310.7409>

[73] Frazer & McKellar: original samplesort paper

<https://research.ibm.com/publications/samplesort-a-sampling-approach-to-minimal-storage-tree-sorting>

[74] Axtmann et al.: in-place parallel superscalar samplesort

<https://arxiv.org/abs/1705.02257>

[75] Sedgewick & Wayne: binary search trees

<https://algs4.cs.princeton.edu/32bst/>

[76] Sedgewick & Wayne: balanced search trees

<https://algs4.cs.princeton.edu/33balanced/>

[77] Princeton: Shell implementation and Knuth bound

<https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/Shell.html>

[78] Poonen: Shellsort bounds and Pratt increments

<https://math.mit.edu/~poonen/papers/shell.pdf>

[79] Frigo et al.: Cache-Oblivious Algorithms, MIT

https://ocw.mit.edu/courses/6-895-theory-of-parallel-systems-sma-5509-fall-2003/7e5331aae765482e87251ba75d9aeabe_cach_oblivs_focs.pdf

[80] Prokop: Cache-Oblivious Algorithms thesis, MIT

https://ocw.mit.edu/courses/6-895-theory-of-parallel-systems-sma-5509-fall-2003/6dc7de52dcf13b53cebf2fe10ae6752a_cach_oblivs_thsis.pdf

[81] Stober Weiß On the Average Case of MergeInsertion

<https://arxiv.org/abs/1905.09656>

[82] Princeton Sorting Applications Index Sorting

<https://algs4.cs.princeton.edu/25applications/>

[83] OpenDSA External Sorting

<https://opensa-server.cs.vt.edu/ODSA/Books/CS3/html/ExternalSort.html>

[84] Yufei Tao External Memory Model and Sorting

<https://www.cse.cuhk.edu.hk/~taoyf/course/wst621/notes/lec1.pdf>

[85] NIST Polyphase Merge Sort

<https://xlinux.nist.gov/dads/HTML/polyphmrgsrt.html>

[86] NIST Optimal Polyphase Merge Sort

<https://xlinux.nist.gov/dads/HTML/optimalPolyphaseMergeSort.html>

[87] Carnegie Mellon Parallel Sorting

<https://www.cs.cmu.edu/afs/cs/academic/class/15210-f15/www/tapp.html>

[88] Authors IPS4o implementation

<https://github.com/ips4o/ips4o>

[89] Satish Harris Garland Designing Efficient Sorting Algorithms for Manycore GPUs

<https://www.nvidia.com/docs/io/67073/nvr-2008-001.pdf>

[90] NVIDIA CUB DeviceRadixSort official implementation and documentation

https://github.com/NVIDIA/cccl/blob/main/cub/cub/device/device_radix_sort.cuh

[91] NIST Bitonic Sort and Batcher Reference

<https://xlinux.nist.gov/dads/HTML/bitonicSort.html>

[92] Hans Werner Lang Odd-even mergesort

<https://www.inf.hs-flensburg.de/lang/algorithmen/sortieren/networks/oemen.htm>

[93] University of Pittsburgh Shear Sort assignment

<https://people.cs.pitt.edu/~bmills/docs/teaching/cs1645/hw6.pdf>

[94] Scherson Sen Shamir Two nearly optimal sorting algorithms using shear-sort

[https://doi.org/10.1016/0743-7315\(89\)90047-6](https://doi.org/10.1016/0743-7315(89)90047-6)

[95] NIST DADS: comb sort

<https://xlinux.nist.gov/dads/HTML/combSort.html>

[96] Hans Bezemer: Circle sort, original author discussion

<https://sourceforge.net/p/forth-4th/wiki/Circle%20sort/>

[97] Haddon: Cycle-Sort: A Linear Sorting Method (author-uploaded paper)

https://www.researchgate.net/publication/220459798_Cycle-Sort_A_Linear_Sorting_Method

[98] Manipal: Parallelization of Cycle Sort Algorithm

<https://researcher.manipal.edu/en/publications/parallelization-of-cycle-sort-algorithm/>

[99] Colorado State University: Enumeration Sort with duplicate tie breaking

<https://www.cs.colostate.edu/~cs475/f17/Lectures/Lec10Sorting.pdf>

[100] NIST DADS: selection sort

<https://xlinux.nist.gov/dads/HTML/selectionSort.html>

[101] NIST DADS: gnome sort

<https://xlinux.nist.gov/dads/HTML/gnomeSort.html>

[102] NIST DADS: bubble sort

<https://xlinux.nist.gov/dads/HTML/bubblesort.html>

[103] NIST DADS: bidirectional bubble sort

<https://xlinux.nist.gov/dads/HTML/bidirectionalBubbleSort.html>

[104] ETH Zürich: Principles of Distributed Computing, Odd/Even Sort

https://disco.ethz.ch/courses/fs12/podc/lecture/podc_2on1.pdf

[105] UCAR CESM official source: interchange_sort

https://docs.cesm.ucar.edu/models/cesm2/config/old/cesmBbrowser/html_code/cam/quicksort.F90.html

[106] WikiSort author: merging techniques

<https://github.com/BonzaiThePenguin/WikiSort/blob/master/Chapter%202.%20Merging.md>

[107] WikiSort author: rotation and other tools

<https://github.com/BonzaiThePenguin/WikiSort/blob/master/Chapter%201.%20Tools.md>

[108] Anupam Gupta: Sorting Pancakes lecture notes

<https://cs.nyu.edu/~anupamg/251-notes/pancakes.pdf>

[109] Cohen and Blum: On the problem of sorting burnt pancakes

<https://www.sciencedirect.com/science/article/pii/0166218X94000093>

[110] NIST DADS: stooge sort

<https://xlinux.nist.gov/dads/HTML/stoogesort.html>

[111] Broder and Stolfi: Pessimal Algorithms and Simplicity Analysis

<https://ic.unicamp.br/~stolfi/EXPORT/papers/by-tag/papers/math/pessimal-algorithms/download.pdf>

[112] Arulanandham, Calude and Dinneen: Bead-Sort: A Natural Sorting Algorithm

https://www.researchgate.net/publication/37987842_Bead--Sort_A_Natural_Sorting_Algorithm

[113] Calude: University of Auckland publication list

<https://www.cs.auckland.ac.nz/~cristian/papers.html>

[114] A. K. Dewdney: Computer Recreations, June 1984

<https://www.scientificamerican.com/article/computer-recreations-1984-06/>

[115] Kevlin Henney: Need Something Sorted? Sleep On It!

<https://accu.org/journals/overload/31/175/henney/>

[116] NIST DADS: permutation sort

<https://xlinux.nist.gov/dads/HTML/permutationSort.html>

[117] NIST DADS: bogosort

<https://xlinux.nist.gov/dads/HTML/bogosort.html>

[118] Gruber, Holzer and Ruepp: Sorting the Slow Way

https://link.springer.com/chapter/10.1007/978-3-540-72914-3_17

[119] NIST DADS: bozo sort

<https://xlinux.nist.gov/dads/HTML/bozoSort.html>

[120] David Morgan-Mar: Bogobogosort definition and discussion

<https://www.dangermouse.net/esoteric/bogobogosort.html>

[121] Lucas Werkmeister: recursive-check implementation

<https://github.com/lucaswerkmeister/Miscellaneous/blob/master/Sorting/src/Bogobogosort.java>